

分类号 TN492
UDC 621.3

学校代码 10590
密 级 公开

硕士学位论文

用于锂离子电池 SOC 估计的 数字芯片设计

学位申请人姓名 彭一闻

学位申请人学号 2110436222

专业（领域）名称 集成电路工程

学 位 类 别 电子信息硕士

学院（部、研究院） 电子与信息工程学院

导 师 姓 名 黎冰

二〇二四年六月

深圳大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文 用于锂离子电池 SOC 估计的数字芯片设计 是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律后果由本人承担。

论文作者签名：彭一闻

日期：2024 年 5 月 20 日

深圳大学 学位论文使用授权说明

本学位论文作者完全了解深圳大学关于收集、保存、使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权单位属深圳大学。学校有权保留学位论文并向国家主管部门或其他机构送交论文的电子版和纸质版，允许论文被查阅和借阅。本人授权深圳大学可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（涉密学位论文在解密后适用本授权书）

论文作者签名：彭一闻

导师签名：



日期：2024 年 5 月 20 日

日期：2024 年 5 月 20 日

摘 要

电动汽车是目前消费市场备受瞩目的产品,同时具有较高关注度的是其主要部件锂离子电池。锂离子电池渗透在我们生活的方方面面,是我们离不开的储能装置,然而受限于目前的材料技术,锂离子电池仍有不少缺点,准确估计锂电池的 SOC (State of Charge, 荷电状态) 是提升锂离子电池使用体验的关键。本文以高准确度、高鲁棒性地估计锂离子电池 SOC 为目标,研究了相关算法及硬件实现方式。

本文首先从电池的工作原理出发,探究了最能反映电池容量的度量单位,并在此基础上展开算法研究。构建一个能准确描述锂离子电池动态特性的电池模型是分析电池行为的常用手段,因此本文在对比分析不同模型在各种工况下的表现后,选择了二阶电路模型进行电池建模。随后,本研究创新性地提出了一种结合安时积分法和电池模型的特征提取算法,将提取的特征矩阵输入 LSTM (Long Short-Term Memory, 长短时记忆网络) 估计 SOC。通过与其他算法模型的效果对比,验证了该算法的有效性,该算法在噪声参数 σ 大小分 0.01、0.001、0.0001 的三种情况下的平均 RMSE (Root Mean Square Error, 均方误差) 分别为 1.571%、0.912%、1.174%。

本文进一步探讨了实现此算法的数字电路设计方案,提出了一种结合上述算法和开路电压法的双模式顶层架构,并详细阐述了各个模块的实现方法,该电路设计对定点模型的还原度为 99.9%,对浮点模型的还原度为 96.62%。在完成源码的编写和仿真后,将该设计部署到 FPGA 上,结合锂离子电池、充放电模块、前端采集板等部件搭建实物测试平台。此外,继续完善了 DC 综合、ICC 布局布线等后端流程,芯片布局规划后的面积为 0.17mm^2 ,在 50MHz 的工作频率下完成一次 SOC 估计的运算时间为 2.86ms。

关键词: 电池建模, SOC 估计, LSTM 网络, 数字芯片

ABSTRACT

Electric vehicles are currently a highly anticipated product in the consumer market, and the primary component garnering significant attention is the lithium-ion battery. Lithium-ion batteries permeate various aspects of our lives and are indispensable energy storage devices. However, constrained by current material technology, lithium-ion batteries still have many drawbacks. Accurately estimating the State of Charge (SOC) of lithium-ion batteries is crucial to improving the user experience. This paper aims to estimate the SOC of lithium-ion batteries with high accuracy and robustness, studying relevant algorithms and hardware implementation methods.

Firstly, the paper explores the most representative metrics of battery capacity based on the working principle of batteries and, on this basis, conducts algorithm research. Constructing a battery model that accurately describes the dynamic characteristics of lithium-ion batteries is a common method for analyzing battery behavior. Therefore, after comparing the performance of different models under various working conditions, the second-order circuit model was chosen for battery modeling. Subsequently, this study innovatively proposes a feature extraction algorithm that combines the ampere-hour integral method and the battery model, the extracted feature matrix is input into an LSTM (Long Short-Term Memory) network to estimate the SOC. By comparing the effectiveness of this algorithm with other algorithm models, the validity of the proposed algorithm is verified. The average RMSE (Root Mean Square Error) of this algorithm under noise parameters σ of 0.01, 0.001, and 0.0001 are 1.571%, 0.912%, and 1.174%, respectively.

Furthermore, the paper discusses the digital circuit design scheme for implementing this algorithm, proposing a dual-mode top-level architecture that combines the aforementioned algorithm and the open-circuit voltage method. The implementation methods of various modules are detailed. The fidelity of this circuit design to the fixed-point model is 99.9%, and to the floating-point model is 96.62%. After completing the source code and simulation, the design was deployed on an FPGA,

and a physical testing platform was constructed by combining components such as lithium-ion batteries, charging/discharging modules, and front-end acquisition boards. Additionally, the back-end processes such as DC synthesis and ICC layout routing were further refined, with the chip layout area planned to be 0.17mm^2 . At a working frequency of 50MHz, the computation time for one SOC estimation is 2.86ms.

Key words: Battery modeling, SOC Estimation, LSTM Network, Digital chip

目 录

摘 要	I
ABSTRACT.....	II
第一章 绪论	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	4
1.2.1 SOC 估计算法研究现状.....	4
1.2.2 SOC 估计芯片研究现状.....	6
1.3 论文研究内容.....	7
1.4 论文章节安排.....	8
第二章 电池建模	10
2.1 本章引言.....	10
2.2 相关知识.....	10
2.2.1 锂电池工作原理	10
2.2.2 电池 SOC.....	11
2.2.3 数据来源介绍	14
2.3 电池模型的建立.....	15
2.3.1 电池模型种类	15
2.3.2 电池模型参数辨识.....	17
2.3.3 参数辨识的结果分析	18
2.4 本章小结.....	22
第三章 SOC 估计算法.....	23
3.1 本章引言.....	23
3.2 特征提取算法.....	24
3.2.1 数据引入噪声	24
3.2.2 特征矩阵构造	25
3.2.3 归一化	28
3.3 深度学习算法.....	29
3.3.1 网络结构	29
3.3.2 激活函数	30
3.2.3 超参数的设置	31
3.4 算法效果分析.....	32

3.4.2 估计结果分析	32
3.4.1 量化指标分析.....	36
3.5 网络所占用存储空间.....	39
3.6 本章小结.....	40
第四章 电路设计	41
4.1 本章引言.....	41
4.2 定点化.....	41
4.2.1 定点化方法	41
4.2.2 定点数乘法	42
4.3 整体架构.....	43
4.4 各模块设计.....	44
4.4.1 SPI 通信模块.....	44
4.4.1 模式选择模块	45
4.4.2 特征构造模块	46
4.4.3 神经网络模块	46
4.4.4 开路电压模块	50
4.5 本章小结.....	51
第五章 FPGA 验证与后端设计.....	52
5.1 本章引言.....	52
5.2 逻辑仿真.....	52
5.3 原型验证.....	54
5.3.1 测试平台搭建	54
5.3.2 FPGA 综合	56
5.3.3 验证结果	60
5.4 后端设计.....	61
5.5 本章小结.....	63
第六章 总结与展望	64
6.1 总结.....	64
6.2 展望.....	65
参 考 文 献	66
致 谢	70
攻读硕士学位期间的科研成果.....	71
附：指导教师对研究生学位论文的学术评语	
答辩委员会决议书	

第一章 绪论

1.1 研究背景及意义

近年来，以 EV（Electric vehicle，电动车）为主流的新能源汽车毫无疑问是消费市场的聚焦重点^[1]。图 1-1 显示 2015 至 2025 年国内汽车总销量、新能源车销量和新能源车销量占比，其中 2024 与 2025 年为预测数据^[2,3]。可以看到自 2020 年以来，新能源汽车在销量上呈现指数级增长，从占比不足 5% 快速增长到了 30%，而预计的 2025 年新能源车销量占比将达到 45%。

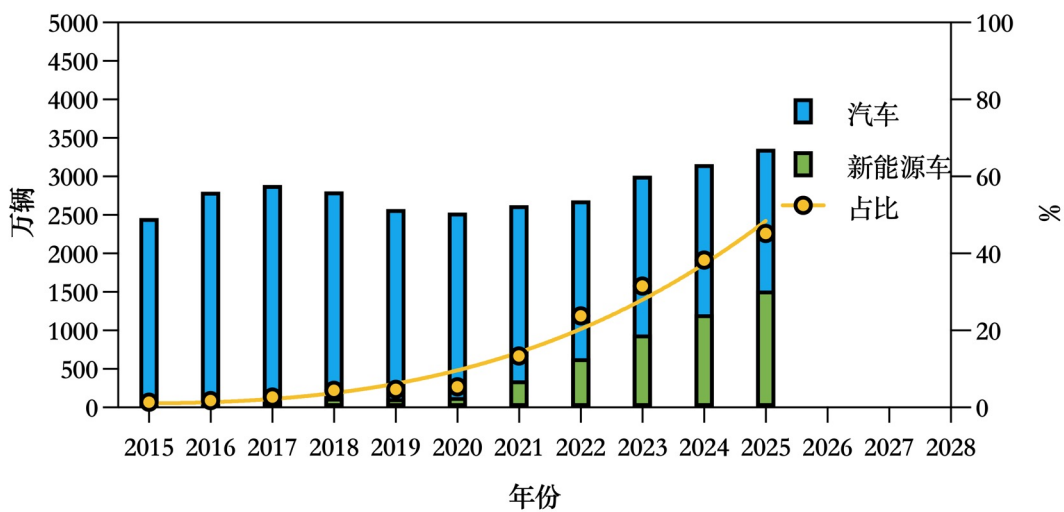


图 1-1 2015-2025 国内汽车销量

电动车带来的热度之大，以至于磷酸铁锂、三元锂这些电池领域的专业词汇都已经称得上家喻户晓，宁德时代、比亚迪等生产锂离子电池的大公司也为人们所熟知。对部分 EV 来说，电池包的成本能够占到整车的 40% 以上，是车上无可争议的“大件”，燃油车时代人们购买汽车关注发动机的缸数、变速箱的类型，而如今，人们对于新能源汽车更关注充电速度、续航里程等，这些都反映了电池对于电动车的重要性。

表格 1-1 对比了轴距接近的两台轿车：比亚迪汉 EV 与奥迪 A4L，在长度与重量上，汉 EV 略大于 A4L，功率上汉 EV 略高于 A4L^[4]，但在行使成本上，汉 EV 仅有 A4L 的五分之一不到，电价为南方电网充电站的日间电价 0.87 元/kWh，油价为 95 号汽油 8.61 元/L。可以说，EV 的高性价比正是越来越多消费者选择购买的原因。

表 1-1 电动车与油车参数对比

	轴距	重量	功率	百公里耗能	行驶成本	售价
	mm	kg	kW	kWh/L	元/km	万元
比亚迪汉 EV	2920	2295	150	13.2	0.11	17.98
奥迪 A4L	2908	2080	140	6.9	0.59	32.18

而电池的发展可以说刚刚起步,图 1-2 显示了锂离子电池近年来的成本变化,在能够预见的未来,电池的成本还会大幅下降^[5],这也意味着,在整车成本方面,电动车还有大幅下探空间,而燃油车技术已经十分成熟,成本不大有缩减的可能。抛开成本因素,新能源汽车还对环境更加友好,促进新能源发展是各国家节能减排的重要举措之一。因此,不管是 EV 还是锂电池,未来都有非常大的市场前景。

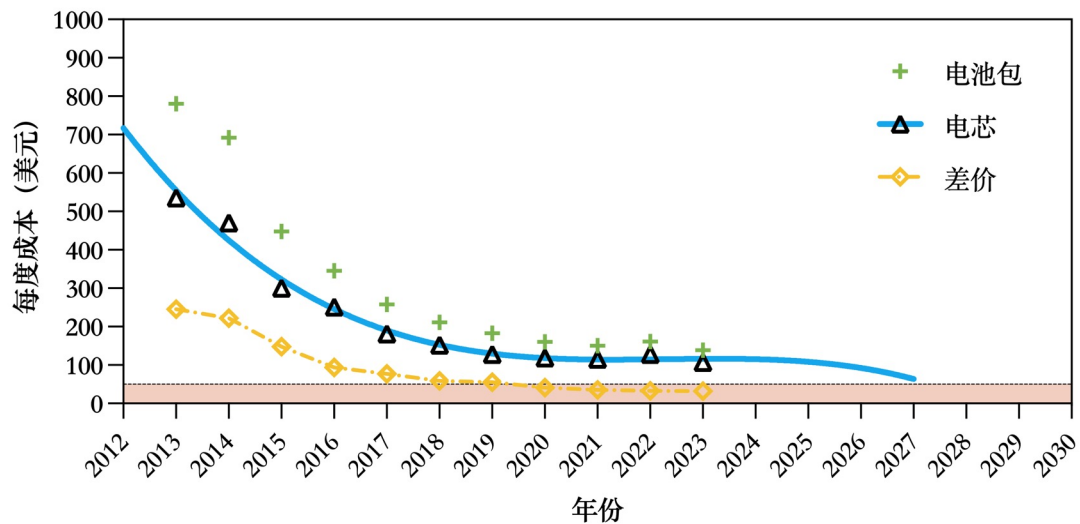
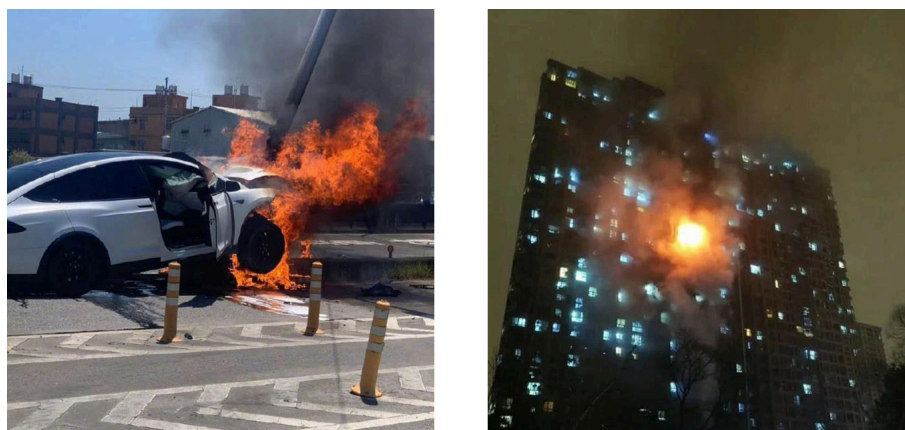


图 1-2 2013-2023 锂离子电池成本

事实上,在电动车如雨后春笋般出现之前,锂电池就已经大规模普及,最早可以追溯到 1970 年索尼就推出了第一款商用锂电池^[6],但电动车使得其真正进入大众的视野,获得如此高的关注度。集成电路的发展使得计算机等芯片在小规模的尺寸上实现成为可能,便携式电子设备应运而生并且不断发展,如今智能手机、笔记本电脑以其高性能、长续航、便携性等诸多优点,成为我们生活中必不可少的工具。而锂离子电池作为移动储能的最优选择,早已与我们的日常息息相关,且影响将进一步加深。

诚然锂电池具有非常好的市场预期,但其目前并非毫无缺点,锂离子电池的安全性问题无疑是最令人担忧的,尤其是对电动车用户,诸如图 1-3 所展现的电动车自燃或电瓶车引发火灾等新闻,使得不少人对电动车望而却步。



(a) 电动车事故后着火

(b) 电动自行车引发火灾

图 1-3 电动车带来的安全担忧

尽管锂离子电池由于高能量密度广受欢迎,但热失控所带来的风险也成为限制其发展的瓶颈。因而对于电池来说,BMS (Battery management system, 电池管理系统)是必不可少的。图 1-4 显示了 BMS 一般会行使的职能^[7-9],通过监控电流、电压及温度等信息,BMS 在必要时刻采取措施,为用电安全提供保障。除此之外,BMS 对电池本身也提供必要的保护。



图 1-4 电池管理系统职能

先进的 BMS 能诊断的信息非常多,包括电池健康度(SOH)、放电深度(DOD)、能量状态(SOE)、功率状态(SOP)、功能状态(SOF)、健康状态(SOH)、剩余使用寿命(RUL)、剩余放电时间(RDT)、平衡状态(SOB)和温度状态(SOT)等^[10-13]。

众所周知,过度充电或者过度放电都会对电池寿命造成损害,因此对电池的充放电进行管理是有必要的。电池的 SOC (State of charge, 荷电状态)可以认为是电池剩余电量的百分比,更确切的定义将在第二章详细阐述,掌握电池的 SOC 情况是进行充放电管理必不可少的环节。通过监控电池 SOC 来管理充放电行为,

可以最大化电池的使用效率、延长电池的寿命。

准确地获知 SOC 对 BMS 的重要性不言而喻^[14-16],但 SOC 并不能直接测量,只能通过电压、电流、温度及其他信息间接估计,BMS 中用以估计 SOC 的模块有时称作 BFG (Battery Fuel Gauge, 电量计),这与本文中的 SOC 估计芯片是一个意思。估计所使用的算法效果直接影响到整个 BMS 的工作效率,因此一款准确高效的 SOC 估计芯片对如今越来越庞大的锂电池市场所带来的影响将是深远的,这也是本课题的选题背景及研究的意义所在。

1.2 国内外研究现状

1.2.1 SOC 估计算法研究现状

电池 SOC 作为反映电池状态的重要指标,对其估算方法的研究非常多,本节探讨估计电池 SOC 估计算法的研究情况。纵观各文献所使用的方法,大致可以分为三类:直接测量法、基于模型的方法、基于机器学习的方法。下面详细阐述各方法的优缺点。

1.2.1.1 直接测量法

电池可以直接测量的量是其端电流和端电压,直接测量的方法就是用这两个测量值进行估算。

1. 使用电流估算的方法为安时积分法,这也是目前的电量计产品中使用最为广泛的方法^[17-20],计算的方式为初始的电量加上变化的电量,公式为(1-1)。实际上,不管是 SOC 还是电流 I 都不是时间的确定函数,因此积分只能用多项和来近似。假设电流的采样率为 f ,那么采样时间间隔 $\Delta t = \frac{1}{f}$ 则是最小的求和单元,公式则改写为式(1-2)。

$$soc(t) = soc(t_0) + \frac{\int_{t_0}^t i(t) \cdot t dt}{Q_{max}} \quad (1-1)$$

$$soc(t) = soc(t_0) + \frac{\sum_{n=0}^{(t-t_0)f} i(n) \cdot \Delta t}{Q_{max}} \quad (1-2)$$

式中 soc 代表 SOC, i 代表电流, Q 代表电荷量, n 代表第 n 个采样点。安时积分法有时也被称作 CC (Column Count, 库伦计数) 算法^[21-24],安时与库伦同为电荷量单位,换算关系为 $1Ah=3600C$ 。该方法优点是算法简单,且具有较好的准确

性，适合对精度要求不是特别高的场合。缺点也较为明显，从公式中看 soc 取决于初始值 $soc(t_0)$ 、总容量 Q_{max} ，以及电流的采样值 i_n 。精度受这三个变量的影响较大，任何一个产生误差都会导致结果的偏差，如果没有纠正的算法辅助，这个偏差不会随着时间消除，且会越来越大，即鲁棒性较差^[25,26]。

2. 使用电压估算的方法为开路电压法。已经有研究表明，对于任何一款电池，其OCV（Open circuit voltage，开路电压，指电池在没有负载的情况两端的电压）在电池静置状态下与电池的SOC具有固定的函数关系^[27-29]，当然这种关系是一种非常复杂的非线性函数，只能通过实验获取一一对应的值从而进行推断。这种方法的优点在于十分简单，在已知SOC-OCV曲线的情况下，只需要查表就能立即得到结果，并且准确度较高。缺点则是电池一旦接入负载，其开路电压就无法通过测量的手段获取，要等到负载为0且电池恢复较长时间才能再次测量到较为准确的开路电压。

1.2.1.2 基于模型的方法

常用的电池模型是将电池用戴维南等效电路近似，戴维南电路中的RC回路模拟了电池的一些非线性效应，更高阶的戴维南电路往往有更为接近的效果。但不管是上述哪类模型，都无法真正的反映电池的特性，常用的方法是使用模型搭配一些算法来估计SOC^[30-33]。

目前各研究学者们使用较多的方法是电池模型配合KF（卡尔曼滤波）及其衍生算法^[34-37]，以EKF（扩展卡尔曼滤波）为例，它通过对系统的测量噪声和观测噪声进行调节，大大的提高了电池SOC的预测精度，并且计算过程相对比较简单，但其性能容易受噪声影响^[38,39]。

1.2.1.3 基于机器学习的方法

机器学习的方法由数据驱动，包括SVM（支持向量机）、神经网络等，这些算法模型可以认为是一个“黑盒模型”，用大量数据训练得到模型的各个参数。随着计算机算力的提高，如今这种方法在各个领域都更多地被采用。这种模型其实与前面的电路模型也有相似之处，只不过比起具体的电路元件，这类模型本身的结构更为抽象。支持向量机回归是一种经典的回归算法^[40-44]；神经网络类型很多，有卷积神经网络、循环神经网络等^[45-49]。

这类方法优点是可以建立大量的隐藏特征，寻找特征与输出之间的潜在联系，

非常适合高非线性度的系统。但由于电池的采样数据是一个时间序列，其有效特征又难以获取，预测的时候存在不稳定性^[50-54]。

1.2.2 SOC 估计芯片研究现状

本节探讨用于锂离子电池 SOC 估计的芯片相关研究，与算法研究不同，这部分研究包含 ASIC（专用集成电路）设计或 FPGA 设计。相比于 SOC 估计算法研究，对 SOC 估计芯片的研究较少一些。

在 ASIC 设计方面，高丽大学的 Junwon Jeong 等提出了 ILE（瞬时线性外推）算法^[55]，该算法根据需求基于电流传感器的电流推算 EMF（电动势，大约等同 OCV），从而估算 SOC，并设计了适用于微型物联网电池的 SOC 指示器系统，该系统基于 180nm CMOS 工艺，功耗为 42nW，可以适配容量低至 2 μ Ah 的电池，最大误差 1.7%。罗马大学的 Filippo Neri 等设计了基于 CC 算法的低功耗电池电量计^[56]，该电量计基于 40nm CMOS 工艺，适用于移动电话，功耗为 135 μ W，不超过待机功耗的 5%。BFG 中最为著名的是美国 TI（Texas Instrument，德州仪器）公司的 bq 系列 IC，该系列 IC 部分基于 CC 算法，如 bq2631，该 IC 功耗为 385 μ W；更高端的 IC 则基于 Impedance Track（阻抗跟踪）算法^[57]，该算法通过安时积分估算电池的阻抗，进一步计算电池的 OCV，从而估计 SOC，如 bq27426，该 IC 功耗为 180 μ W。

在 FPGA 设计方面，韩国电子研究所的 Minjoon Kim 等提出了基于二阶电路模型的 EKF 算法^[58]，相比传统 EKF，该算法针对硬件进行优化，牺牲允许范围内的准确度从而降低了 70% 的复杂度，最大误差不超过 2%，并部署到了 ZYNQ 上，工作频率为 100Mhz，计算耗时 0.65 μ s。帕尔马大学的 Mattia Stighezza 等提出了 SVM 算法^[59]，通过蚁群优化方法进行训练，误差 1.4% 左右，该算法无需对特定电池进行建模，具有一定通用性，并在 FPGA 上实现，工作频率为 100MHz。巴纳斯塔利大学的 Bijender Kumar 等提出了 ANN（人工神经网络）结合模糊逻辑的算法^[60]，将 U、I 等输入 ANN，输出 SG（电解液比重），该参数与 SOC 存在一定的相关性，再用模糊器将 SG 映射到 SOC，文中提到准确率为 99.9%，DSP 占用为 40%，IO 占用为 52%。

综上，在相关 ASIC 的研究中多使用了直接测量的方法或电池模型的方法，或者对两者进行结合，而在 1.2.1 中被广泛研究使用的 KF 算法及机器学习算法

则少有涉及，原因可能是这类算法较为复杂，电路实现的面积及功耗较大、成本高昂。在 FPGA 的研究中相反，多使用较为复杂的算法^[61]，这体现了 FPGA 相比 ASIC，能够快速且低成本实现复杂设计的优势。

1.3 论文研究内容

本文的研究目标是设计一款具有高精度、高鲁棒性的锂离子电池 SOC 估计芯片，其功能及其产品定位如图 1-5 所示，该芯片输入前端 ADC 采样的电池信息，通过算法计算后输出剩余电量、剩余使用时间、电池寿命等多项信息供用户参考，或者为电池管理系统（BMS）分析电池的运行情况提供帮助。在图中，芯片被放在了电池包（灰色框）外，这样比较符合本文的设计环境，在实际产品中也有做的做法是，将该芯片也纳入电池包的范围（大虚线框）。

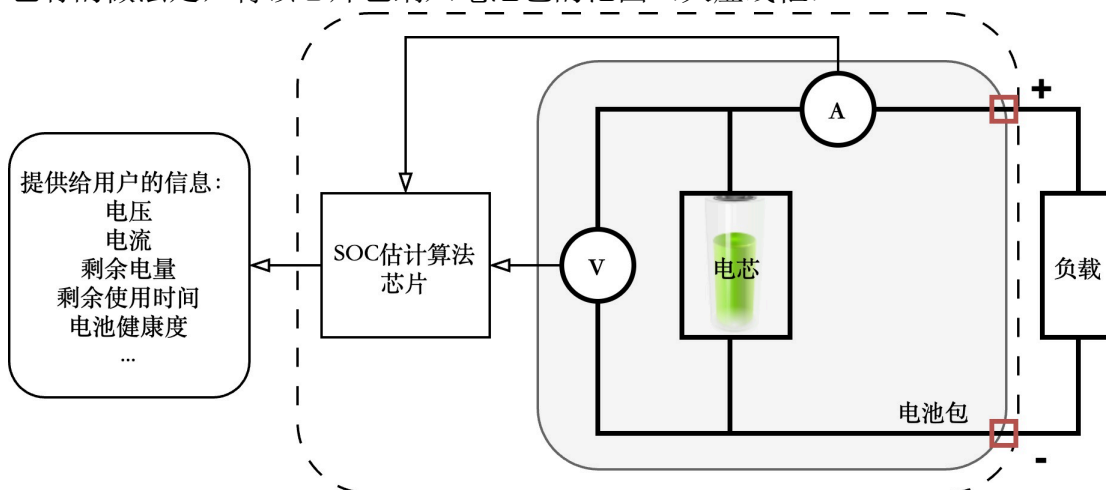


图 1-5 本设计在产品中的位置

基于研究目标，本文内容主要分成以下两大部分：

1. SOC 估计算法的研究。在各大方法的基础上，本文提出了改进的算法。所提出的算法结合电池模型与安时积分法构造特征矩阵，输入核心神经网络估算出 SOC，在硬件层面还结合了开路电压法组成双模式，两种算法取长补短。

2. 电路系统的设计。基于所提出的算法模型，提出了低并行度、多单元复用、多流水线的电路设计方案。电路设计包含多个模块，采用自底向上的设计方式，每个模块都通过逻辑仿真验证功能。设计还部署到了 FPGA 上，配合电池与前端，搭建完整的电池电量检测系统。最后对设计进行综合与布局布线。

1.4 论文章节安排

本文的章节安排基于 ASIC 芯片的开发流程，按照需求分析、行为级建模、RTL（寄存器传输）级设计、逻辑仿真、FPGA 验证、后端设计的顺序行文，全文共计六章。图 1-6 展示了论文的主要工作内容。

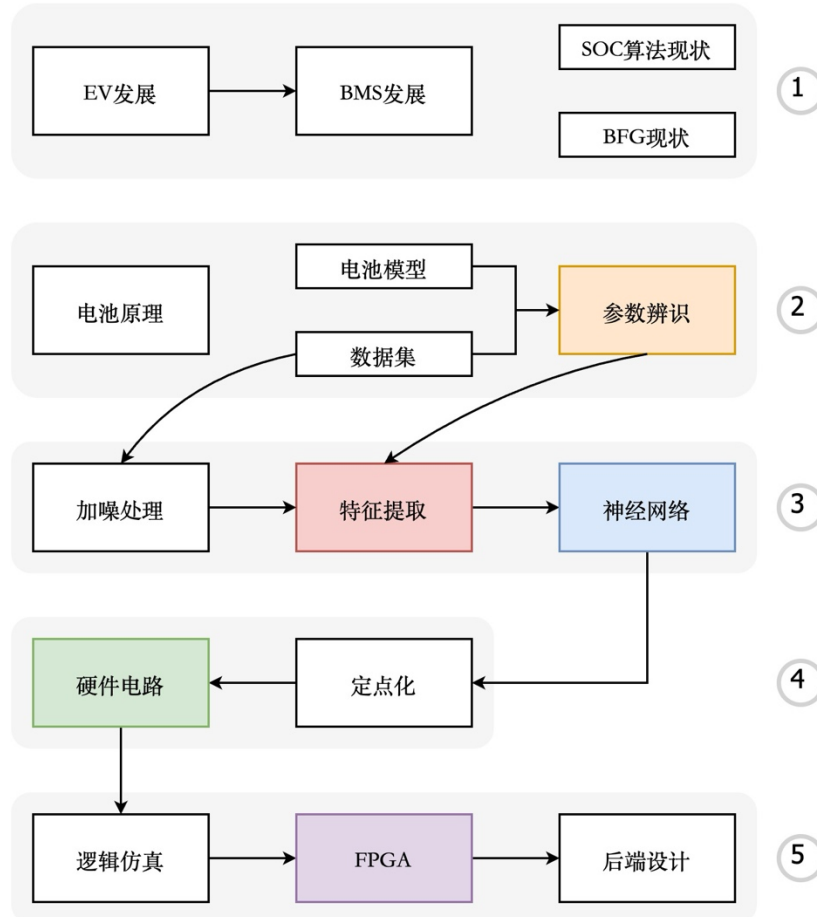


图 1-6 论文内容简介

第一章是绪论，内容包括介绍本课题的背景及选题的意义，从相关算法研究和相关芯片研究两个方面梳理了锂离子电池 SOC 估计这一课题的国内外研究现状，简要介绍本文的研究内容，以及总结各章节的内容安排。

第二章是相关知识以及电池建模，相关知识包括对研究对象的特性分析以及研究采用的数据集介绍，属于需求分析阶段。电池建模部分包括各种模型的介绍、建模与相应参数辨识的方法、各种模型的拟合效果分析，属于行为级建模的前半部分。

第三章是锂离子电池 SOC 估计算法研究，这一章提出了数据集的处理方法、介绍了特征矩阵构建的方法、剖析了神经网络框架，最后对比分析了不同算法的

预测效果，属于行为级建模的后半部分。

第四章是芯片的电路设计，内容主要是介绍算法硬件化的处理方式及阐述顶层到每个模块的架构设计、逻辑时序等，这一章属于 RTL 级设计。

第五章是验证与后端设计。该章节主要内容有，展示 RTL 的逻辑仿真结果，介绍 FPGA 原型验证从平台搭建到结果分析的整个流程，以及后端设计。后端设计主要是综合结果的分析以及布局布线结果分析。

第六章是总结与展望，本章对第 2-5 章的工作进行总结，归纳所取得的研究成果、提出工作中存在的不足与初步的未来改进设想。

第二章 电池建模

2.1 本章引言

本章的主要工作是对锂电池建立模型，目的是为算法的设计提供除电压、电流、温度等可以直接测量的属性以外，更高维度、更能反映电池内部状态的信息。在此之前，2.2 节对研究对象锂电池及 SOC 做必要相关知识的拓展，2.3 节对本章以及后面章节用到的数据来源进行说明。

2.2 相关知识

2.2.1 锂电池工作原理

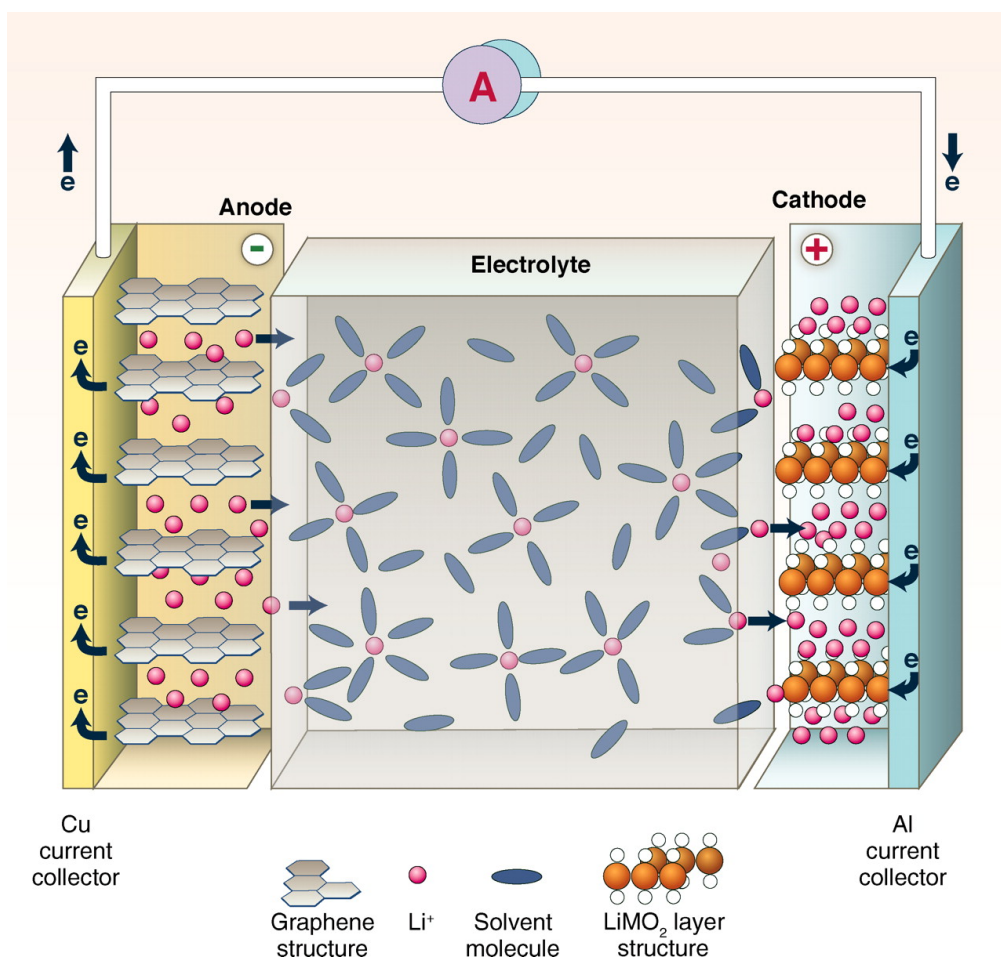


图 2-1 锂电池工作原理

首先明确本文的研究对象是 LIB (Li-ion battery, 锂离子电池)，常简称为锂电池或者锂电，但严格来说 LIB 并不是唯一一种以 Li (Lithium, 锂) 为材料的

电池，其他的 LB（Li-based Battery，锂基电池）也能称作锂电池。

LIB 是 LB 多次迭代后的一种技术，其放电时的工作原理如图 2-1 所示，该图为 Bruce Dunn 等在 2011 年提出^[62]。LIB 通常以石墨作为负极，锂合金氧化物作为阴极正极。放电时，电子从负极经外电路流向正极；失去电子的锂离子则经电解液进入正极。从化学角度来看，负极发生的反应为氧化反应，方程如式（2-1）所示，因此也被称为阳极（Anode）；正极发生的反应为还原反应，称为阴极（Cathode），如式（2-2）所示。



LMB（Li Metal Battery，锂金属电池）是另一种被广泛研究的 LB，它使用金属 Li 作为阳极材料，LMB 最大的优点是能量密高，表 2-1 列出了目前主流研究中的两种 LMB：Li-air（锂空气）电池、Li-S（Li-Sulfur，锂硫）电池与 LIB 的能量密度对比，以 LIB 为基准值 1，Li-S 的相对值达到了 10.4，Li-air 则达到了 20.84，其能量密度远远高于 LIB。且金属 Li 本身还具有极负的电势，相对标准氢电极为-3.04V，电势是一个相对值，讨论电极的电势都是相对标准氢电极而言，即该电极与氢电极组成电池时的电位差。但由于锂金属过于活泼，它们可能存在安全风险（尤其是充电时），这导致 LMB 诞生至今也未广泛商业化。但目前的研究在消除锂枝晶方面取得了一些进展，因此 LMB 被普遍认为是最有前途的 LB 之一^[63,64]。

表 2-1 LIB 与两种 LMB 能量密度对比

电池种类	LIB	Li-air	Li-S
能量密度	250	5210	2600
相对值	1	20.84	10.4

总的来说，鉴于目前锂离子电池占据了市场的主流，本文的讨论范围仅限 LIB，文中所提到的锂电池、电池、锂电都默认指代锂离子电池。

2.2.2 电池 SOC

2.2.2.1 电量的单位

关于本文所研究的电池 SOC（荷电状态），有必要先了解其定义，电池的 SOC 是指电池中储存的可用电量与其额定电量之间的比例。它表示了电池当前已经充

电或放电的程度，通常以百分比的形式表示。SOC 的范围通常从 0%（完全放电）到 100%（完全充电）。

SOC 代表的是电量的比值，便需探讨如何衡量电量的大小。通常，带锂电的设备上标注的电池容量单位多为 mAh（毫安时）或 Wh（瓦时），后者更常见于大容量设备。这两种单位在量纲上的主要区别在于，mAh 作为电荷量单位，表示电流与时间的乘积；而 Wh 作为电能量单位，代表功率与时间的乘积。从衡量电池电量的角度看，mAh 更多关注于锂电池内部的化学反应，Wh 则侧重于锂电池的外部能量输出。在针对电池 SOC 的研究中，国内外文献通常采用 mAh 作为电量的单位，本文亦遵循此惯例，具体理由将在下一小节详述。

2.2.2.2 能量转换效率与库伦效率

讨论 Wh 与 mAh 哪个单位更加适合计量，等同于讨论对电能进行计量还是电荷进行计量更精准，由此引出能量转换效率和电荷转换效率两个概念。

能量转换效率 ETE（Energy Transform Efficiency，能量转换效率）定义为可放出电能与充入电能之比，公式为式（2-1）。在放电的过程中，电池的电势能除了转化为用来做功的能量，还有一部分以热能形式损耗，充电过程中同理，因此用 ETE 来衡量损耗的多少。

$$ETE = \frac{W_{discharge}}{W_{charge}} \quad (2-3)$$

电荷转换效率，也称库伦效率（CE，Column Efficiency），定义为电池可放出的电量与充入电量的比值。前面提到电池的充放电就是锂离子在正负极之间来回移动的过程，但实际放电时并非所有的锂离子都移入负极，充电时也并非所有锂离子都移入正极。

$$CE = \frac{Q_{discharge}}{Q_{charge}} \quad (2-4)$$

效率越低，说明转换过程中以无法统计的形式耗散的量更多，则计量时误差更大。下面采用增量 OCV 实验数据计算 ETE 与 CE。增量 OCV 实验是用以确定 OCV 曲线的实验，实验方式为先充满电，再以 0.1C 的电流放电 10 次，其中每次放电 10min（分钟），接着静止 100min。放电完成后以同样的方式充电 10 次。实验过程的电压与电流曲线如下图 2-2 所示。

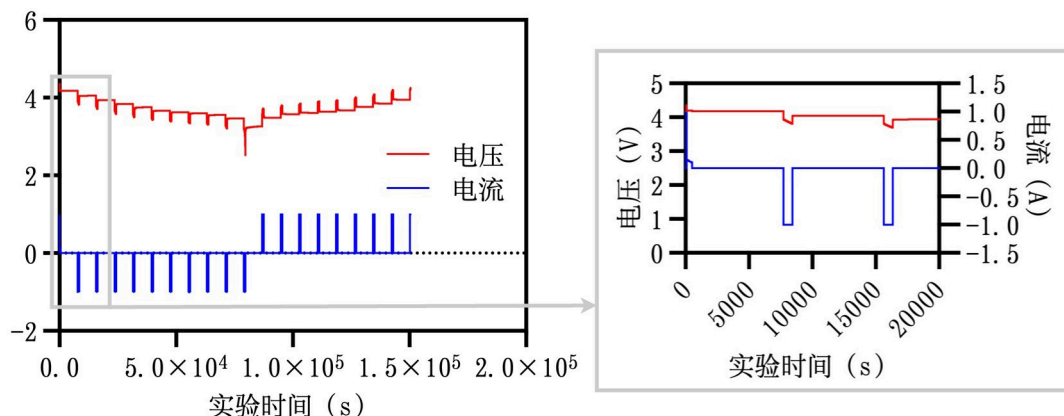


图 2-2 增量 OCV 实验

增量 OCV 实验中电量的变化情况如下图 2-3 所示，在图中，以 mAh 计算的电量用 soc 表示，以 Wh 计算的电量用 soe 表示。根据 1.2.1 节所提到的 OCV 准则，电池的静置开路电压与电池的电量状态具有对应关系，因此可以根据实验结束时刻 t_2 (150000s) 的电压为 3.947V，与第二次放电后时刻 t_1 (22000s) 的静置开路电压基本相同，推断这两个时刻具有相同的电量。 t_2 的 soc 为 0.806，也与 t_1 的 0.8 基本一致； t_2 的 soe 为 0.908，与 t_1 的 0.78 相差 0.128，误差达到了 14.1%，这一数据代表了在 OCV 增量实验条件下，电池每充 1Wh 的电量，实际能使用的能量为 0.86Wh 左右。

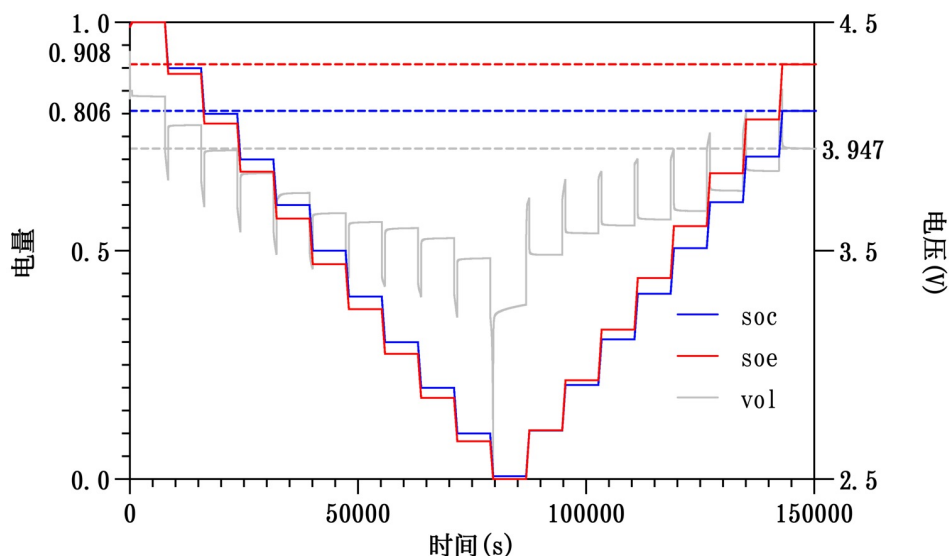


图 2-3 增量 ocv 测试过程中 soc 的变化

从以上分析可以得出结论，由于能量转换效率远低于库伦效率，若使用 Wh 作为电量单位存在较大的误差，用 mAh 作为电量单位计量更加精准。

2.2.2.3 小结

使用 mAh 为单位计量的 SOC 用以反映电池状态较为精准，且一般来说，SOC 可以与日常使用的电子设备所显示的电量剩余百分比划等号。但从严谨的角度来说，这两者存在一定差异，因为电量以 mAh 作为单位计量时无法准确反映设备的剩余使用时间，比如电池 SOC 从 90%~80%与 30%~20%同样是放电 10%，但其对外做功大小不同，因为电池的电压会随着电量的减少而降低。对于使用者来说，使用 Wh 作为电量单位则更合适，因为使用者希望电量所代表的应是使用时间，相同的 Wh 能够代表相同的使用时间（同功率下）。

2.2.3 数据来源介绍

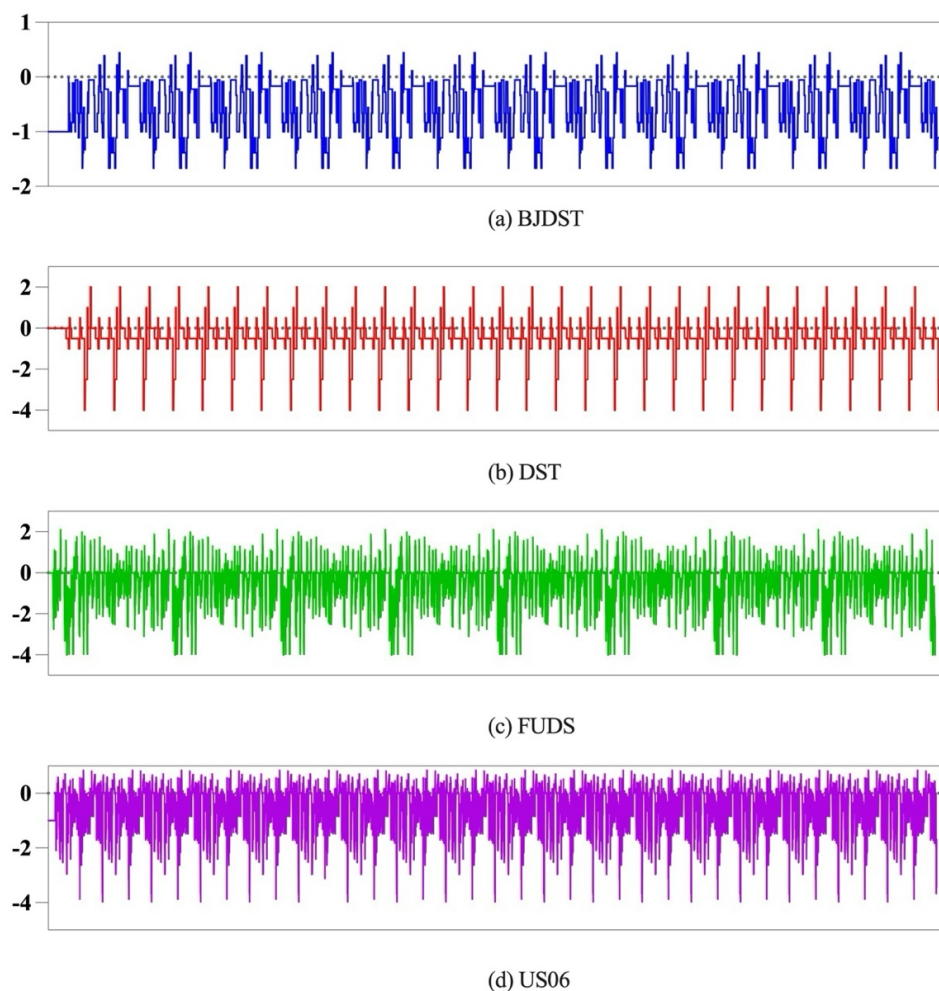


图 2-4 四种不同工况的电流变化曲线

做算法研究，数据集是不可或缺的，数据包括算法的输入与输出，前面已经提到，输入是可测量的属性，输出是不可测量的属性，由于自行采集这些数据不

但需要耗费大量的资金购买高精度设备,还需要大量时间来获得足够的样本数量,不利于研究的进展,鉴于本文的研究重心不在数据的采集上,直接采用已有的数据库中的数据可以大大缩短开发时间。

本文所采用的数据来源于马里兰大学电池实验数据库的公开数据。该团队公开这些数据方便研究人员用于 SOC 估计、电池寿命预测、电池建模分析等^[65]。该 LIB 实验测试数据中包括不同化学成分和形状的电池,本文选取了型号为 INR18650-20R 的电池数据,该电池额定容量为 2000mAh,正极材料为 LiNiMnCo。实验数据包含循环测试、动态测试等,循环测试主要是用来测电池的开路电压、阻抗等特性,动态测试则是模拟电动车的典型使用环境。动态测试包括了四种典型工况:BJDST、DST、FUDS、US06,四种工况下的电流变化曲线如上图 2-4 所示。

2.3 电池模型的建立

2.3.1 电池模型种类

对电池的建模分两种,一种是电化学模型,一种是电路模型。尽管电化学模型可以反映电池内部的电化学过程,包括离子传输、电荷转移、化学反应等,从而揭示电池性能和行为背后的物理和化学机制。但电化学模型通常基于大量的数学方程和假设,较为复杂和抽象,难以直观理解。还有电化学模型中涉及到的一些参数,如反应速率常数、传输系数等,往往需要通过实验或拟合来确定,而这些参数可能在不同条件下变化较大,导致模型的准确性受到影响。而等效电路模型由我们常见的电路元件组合,且只需要掌握基础的电路原理便能够理解,且参数识别较为容易、准确度尚可,被广泛应用于 LIB 及相关领域。

值得一提的是,电池模型仅能近似模拟电池的特性,其精度无法支持直接对 SOC 求解,一般作为各种算法的辅助手段,如卡尔曼滤波算法(KF)。本文选用电路模型对电池进行建模,以模型电路元件参数作为特征支持后续算法研究。

电池本身是一个非线性元件,在电路模型建模中,我们将其用多个随 SOC 变化的线性电路等效。对于某一 SOC 状态下的电池,可以将其等效为一个两端开路的线性有源电路,根据戴维南定理,任何线性、双端口、稳态的电路网络都可以用一个电压源和一个串联的等效阻抗来代替,从外部观察来看,该等效电路

与原始电路在负载端的行为是相同的。一般线性电路所包含的元件有电阻、电感以及电容，而电池的电压曲线中表现出了容性阻抗的特性，因此选用电阻和电容对其进行等效，在具有暂态过程的电路中，我们会提到电路的阶数这一概念，阶数大致可以解释为电路中所含有独立电感或者电容的个数，其表征了求解该电路的微分方程的阶数。下面对三种不同阶数的电路模型 RINT、PNGV、n 阶 ($n \geq 1$) 分别做简单介绍。

1. RINT 模型如图 2-5 所示，阻抗跟踪法便采用了该模型。其简明性不言而喻，但显然该模型无法解释电池的弛豫现象，电池的弛豫现象是指在电池放电或充电过程中，电池电压或电流不立即达到稳定状态的现象。在电池状态发生变化时，电池内部的化学反应需要一定的时间来适应新的工作条件，导致电池的电压或电流呈现出一种缓慢变化的过程。

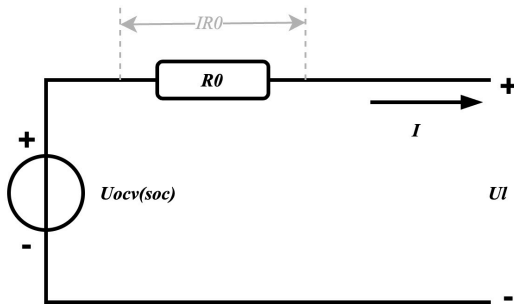


图 2-5 Rint 模型

2. PNGV 模型见图 2-6，由理想电压源与串联电阻、串联电容、极化电阻、极化电容四个元件组成。极化电容与极化电阻组成一个 RC 回路，RC 回路是一种常见的电路配置，用于各种电子和电路应用中，在 RC 回路中，电阻和电容连接在一起，形成一个闭合的电路。电阻的作用是限制电流的流动，而电容则可以储存和释放电荷，这样便可以描述电池的弛豫现象。

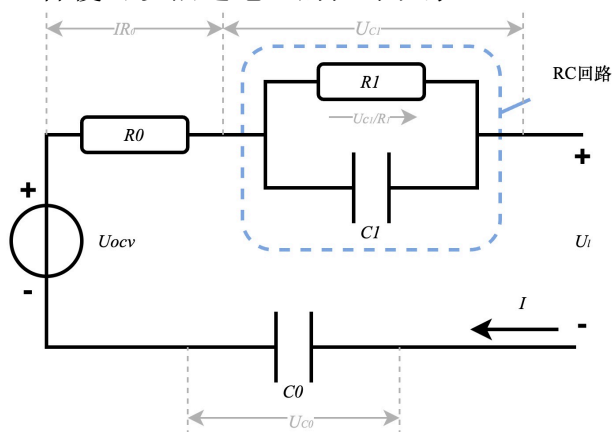


图 2-6 PNGV 模型

3. n 阶模型见图 2-7，由理想电压源与串联电阻以及 n 个 RC 回路组成，事实上，前面所提到的 R_{int} 模型，我认为也可以归于这一类，即 $n=0$ 时的特殊情况。一般来说， n 越大，拟合的精度就越高。

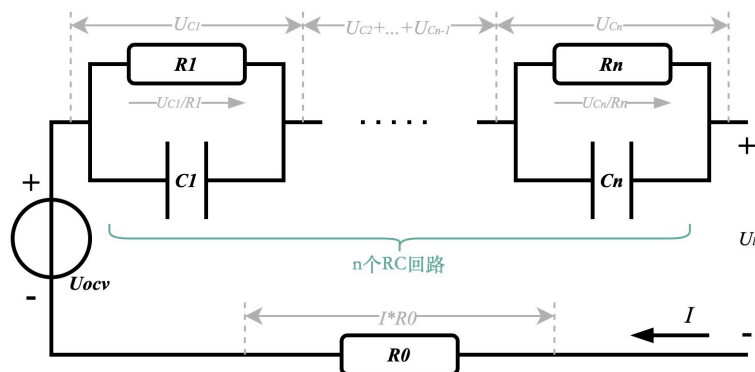


图 2-7 n 阶模型

为了兼顾精确度和复杂度，本文采用 2 阶电路模型。从模型图可以看出，该模型总共有 6 个变量： U_{ocv} 、 R_0 、 R_1 、 C_1 以及 R_2 、 C_2 。在电路分析中，一个 RC 回路决定了一个变量 τ （Tau，时间常数），因此参数 C 可以用 τ 来替代。下文对该模型的 U_{ocv} 、 R_0 、 R_1 、 τ_1 、 R_2 、 τ_2 六个参数进行拟合（辨识）。

2.3.2 电池模型参数辨识

对模型的参数辨识常用的方法有最小二乘法、最大似然估计、遗传算法等，本文所提出的算法中，模型的参数辨识属于离线辨识而非在线辨识，作为前期的数据准备工作，参数辨识直接使用工具软件 Simulink 的扩展 app 完成，Simulink 是 Mathworks 公司旗下的建模仿真软件。

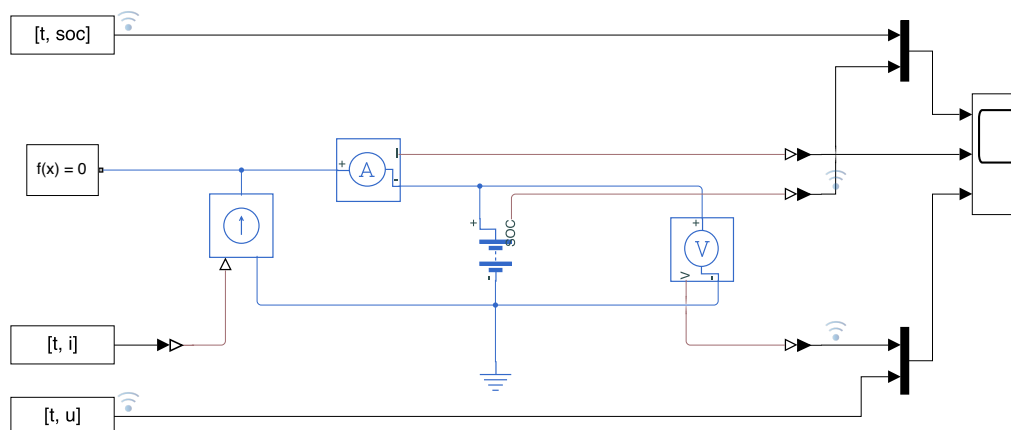


图 2-8 Simulink 建模图

对电池的建模使用 `simscape` 库，对参数的辨识使用 `parameter estimator` 这一 app，Simulink 中搭建的模型如图 2-8 所示。辨识所采用数据集为 OCV 增量实验数据，电流 I 作为激励，电压 U 作为响应，辨识工具通过不断修改参数使得模型的响应 U_{model} 与真实电压值 U_{real} 之间的误差缩小，最终完成优化。前面已经说明，电路模型是随着 SOC 变化的，所以辨识的 6 个结果均为 SOC 的函数，而辨识过程中的 SOC 变化为 `simscape` 库的电池模型自动计算，模型的 SOC 与数据集的实验数据 SOC 对比情况如图 2-9 所示。

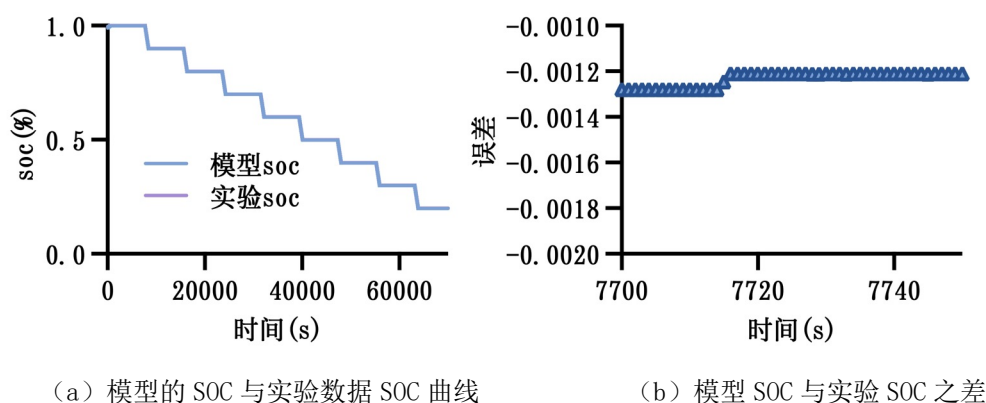


图 2-9 模型与实验的 soc 对比图

这里需要声明实验数据并没有直接给出 SOC，图中的实验 SOC 为实验数据通过安时积分法计算得到。从图 2-9 (a) 可以看出两条曲线完全重合，图 2-9 (b) 给出两曲线之差较为典型的一段，仅为 -0.0012 (0.12%)，而这一误差可能与初值的设置有关，因此可以推断 Simulink 中电池模型对 SOC 的计算方式也是安时积分。

2.3.3 参数辨识的结果分析

由于 `simscape` 库仅提供 n 阶电路模型，考虑到 PNGV 模型与 1 阶电路模型较为接近，因此对 0 阶 (RINT)、1 阶、2 阶三个模型进行参数辨识并对比分析。辨识完成后，各阶模型的响应与实际值对比见图 2-10。0 阶、1 阶、2 阶三个模型均有较良好的拟合效果，但 1 阶和 2 阶相对 0 阶有明显的提升，2 阶相对 1 阶提升幅度则有限。

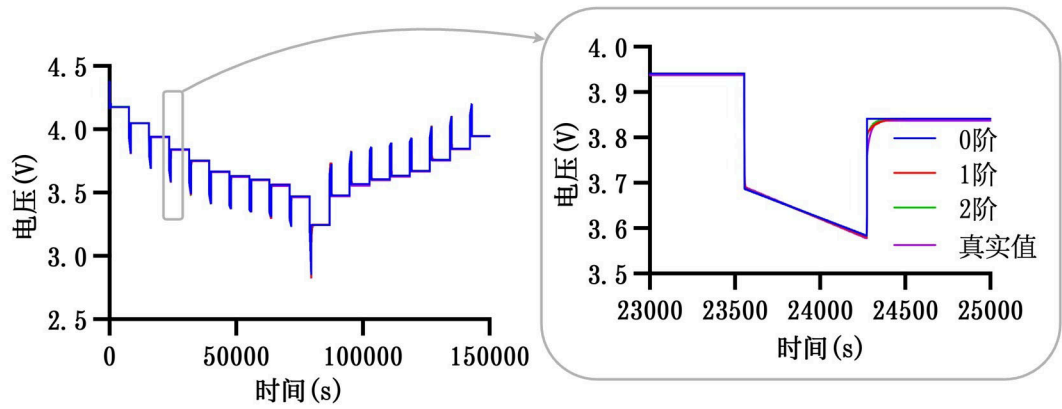


图 2-10 辨识后各阶模型响应

除了在用来说明参数的数据上查看拟合效果，本文将其他四种工况下的实验数据也输入三种模型查看误差情况。表 2-1 列出了这四种工况的仿真设置。

表 2-1 不同工况的仿真设置

工况	初始 SOC (%)	总容量 (mAh)	仿真时长 (s)
DST	78.9	2000	29000
FUDS	0	2000	37000
US06	0	2000	22000
BJDST	0	2075	23000

图 2-11 展示了三种模型在未经拟合的四种工况下的响应误差表现。从图中可以看出，在复杂的工况下，三种模型的准确度相差不多，在 DST 工况和 BJDST 工况下 0 阶模型误差相对较大，而在 US06 工况与 FUDS 工况，三个模型表现较为接近。并且可以看到与真实值相比，存在一定差异，但能够较为贴切的反应真实值的变化情况。

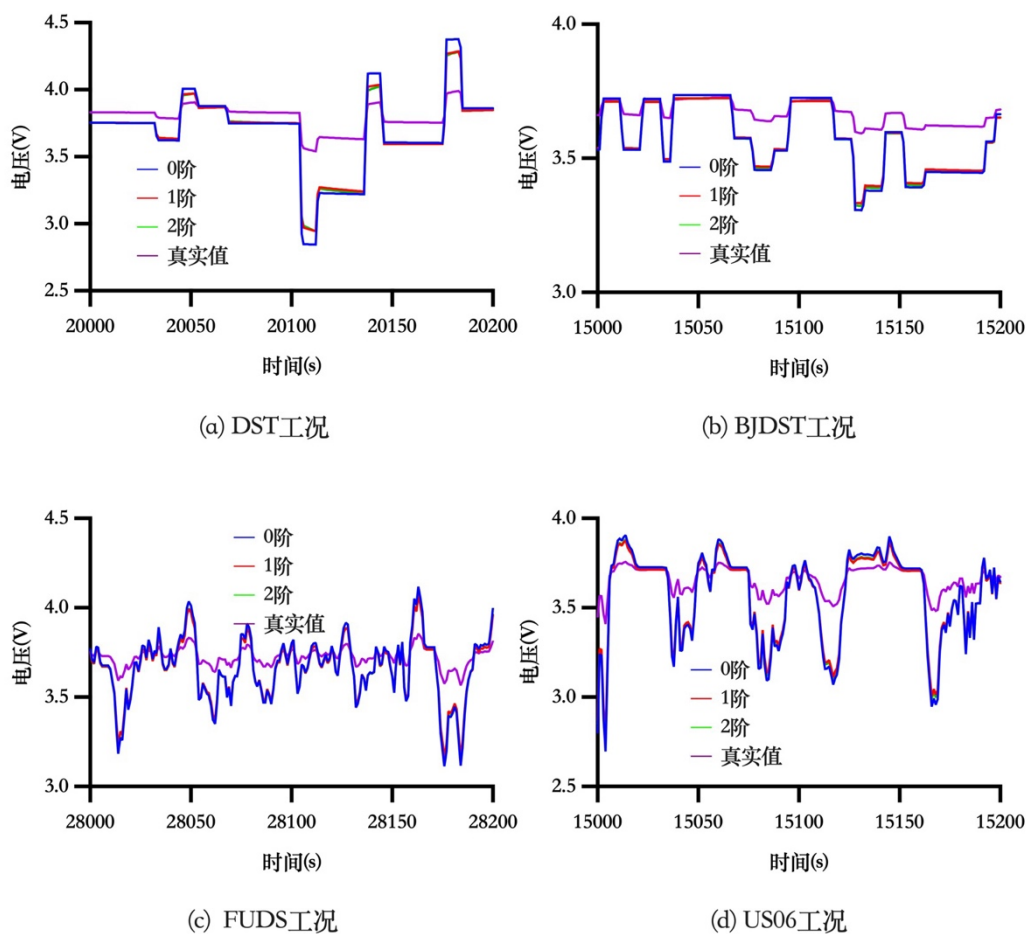


图 2-11 四种工况下模型响应与真实电压值对比

图 2-12 对比了三个模型共同拥有的参数 E_m 以及 R_0 ，可以看到三个模型的 E_m 几乎完全相同，但 R_0 差别较大，阶数越高的电路拥有的极化电阻越多，因此主电阻 R_0 的阻值越小。

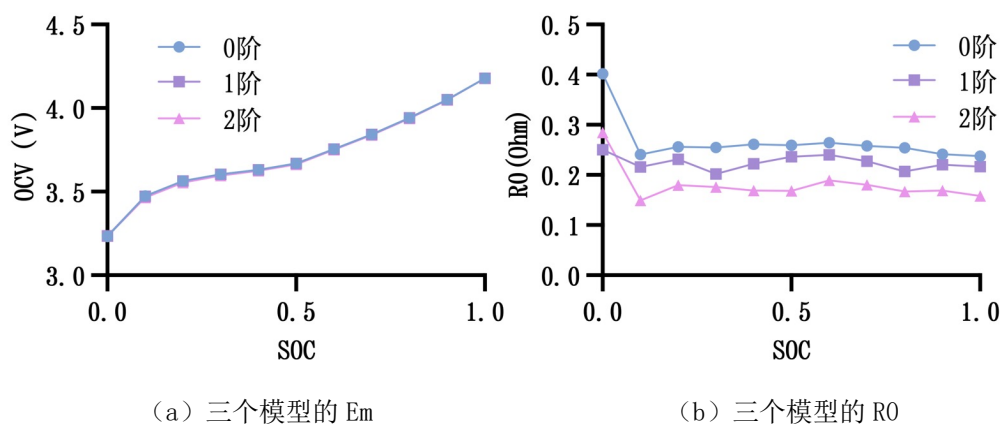
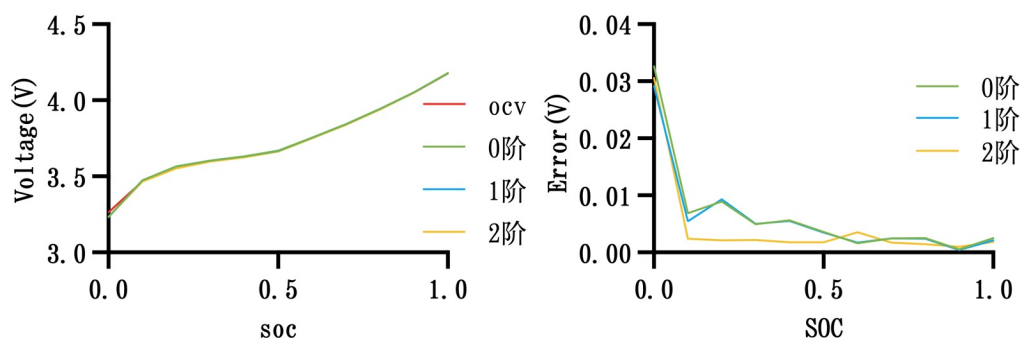


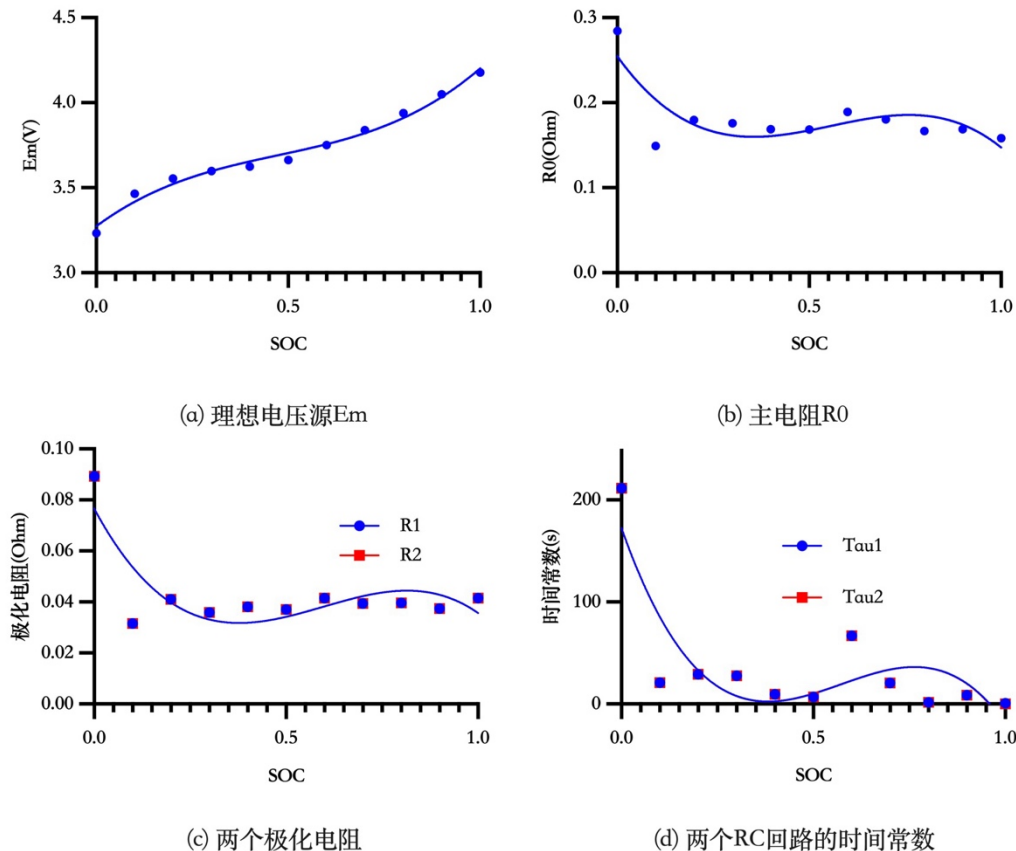
图 2-12 三种模型共同参数对比

这里有必要说明，电路模型参数的 E_m 与开路电压 OCV 在概念上相同，但

在数值上并不相同， E_m 是电路模型对真实 OCV 的近似拟合。获取真实 OCV 的曲线的方式是选取增量 OCV 实验中的数据，10 次放电过程中每次静置后的电压，共计 11 个点，与模型相同。三个模型的 E_m 与 OCV 的对比及绝对误差图如下 2-13，可以看出，0 阶与 1 阶 E_m 与 OCV 差距略大，2 阶则明显好于两者。

(a) 模型 E_m 与OCV对比(b) 模型 E_m 与OCV之差图 2-13 模型 E_m 与 OCV 实验数据对比

下图 2-14 展示了二阶电路模型的辨识参数及通过三次样条插值得到的拟合曲线，由于两个极化电阻以及时间常数在数值上比较接近，因此用同一张图展示。

(a) 理想电压源 E_m (b) 主电阻 R_0

(c) 两个极化电阻

(d) 两个RC回路的时间常数

图 2-14 二阶电路模型的辨识参数拟合曲线

表 2-2 列出了二阶电路模型的辨识参数。从表中可以看出， R_1 与 R_2 的值完全相同，因此可以合并为一个特征。

表 2-2 辨识参数列表

SOC(%)	E_m	R_0	R_1	τ_{u1}	R_2	τ_{u2}
0	3.234	0.284	0.089	211.547	0.089	211.548
10	3.465	0.149	0.032	20.808	0.032	20.815
20	3.553	0.180	0.041	29.084	0.041	29.085
30	3.597	0.176	0.036	27.736	0.036	27.736
40	3.624	0.169	0.038	9.415	0.038	9.415
50	3.663	0.168	0.037	6.977	0.037	6.977
60	3.750	0.189	0.042	66.871	0.042	66.871
70	3.838	0.181	0.039	20.662	0.039	20.662
80	3.939	0.167	0.040	1.662	0.040	1.668
90	4.049	0.169	0.037	8.751	0.037	8.749
100	4.178	0.158	0.041	0.828	0.041	0.003

2.4 本章小结

本章围绕研究课题，首先介绍 LIB 的工作原理，然后通过分析实验数据阐明了以 mAh 作为单位估计 SOC 的原因，接着介绍了本文所使用的数据集，并在该数据集的基础上对电池进行了建模与参数辨识，最后对比了不同模型的效果，选定二阶电路模型作为后文中将使用的模型，并列出了相关的数据。

第三章 SOC 估计算法

3.1 本章引言

本文提出的算法框架如下图 3-1 所示。

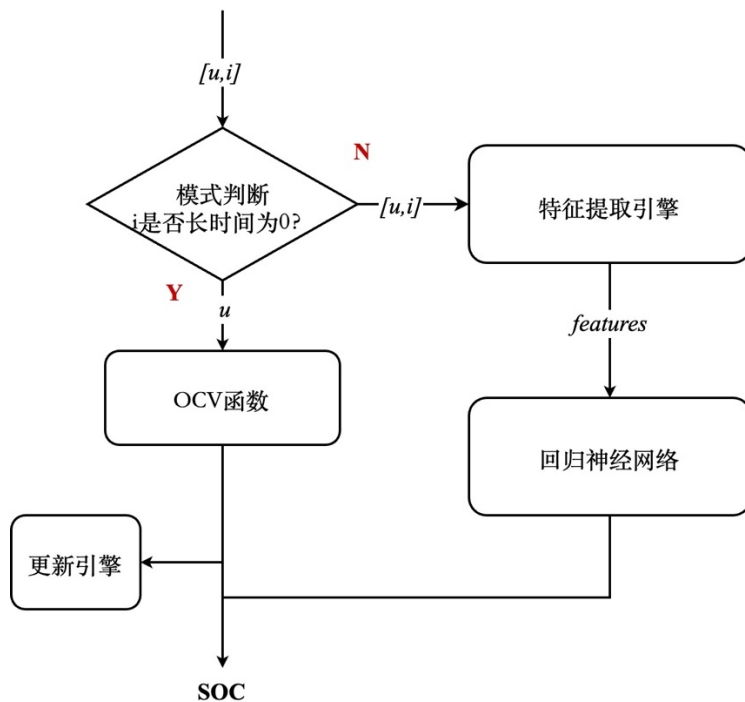


图 3-1 算法框架

前文已经说明，开路电压法提出了锂电池的一个重要特性，即其开路电压 OCV 与 SOC 具有可靠的对应关系，基于这一特性，本设计将输入接入模式判断引擎，将后续算法分成两路模式，一路是动态模式，即在电池的工作状态下启用的模式，其具有复杂的算法，较高的功耗，能够实现在动态工况下的准确 SOC 估计；另一路是静态模式，或者睡眠模式，该模式在电池长时间闲置无负载的情况下启用。在动态模式中，算法由特征提取算法以及基于长短时记忆网络（LSTM）的 SOC 估计算法两个步骤组成。静态模式则由 OCV 函数以及更新特征提取引擎组成。

本文所提出的算法有两点创新。其一，相比于传统的 SOC 估计算法类研究，本文从硬件设计的角度出发，提出了双模式的算法设计，考虑到在实际的可移动电子产品使用情况下，处于休眠状态占据了大部分的时间，这种设计有利于在产品中大幅降低功耗。其二，本文所提出的特征提取算法引入了安时积分模块，有

助于提高估计的精度，结合神经网络高鲁棒性的优点，能够在保证精度的情况下提高鲁棒性。

本章主要介绍提出算法中的动态模式算法，由于该算法使用了神经网络，因此首先使用 Python 语言实现，代码基于 Pytorch 库编写，编译器版本为 3.10。在完成网络的训练和优化后在后文使用 RTL 进行实现。

3.2 特征提取算法

3.2.1 数据引入噪声

在本节提出一个观点，即便是同样的神经网络，但对于不同的算法，预测不同的对象，训练和验证的方式也应该因地制宜。

前面提到安时积分法的一大缺点就是会累积误差，而在实验数据中这一缺点被极大掩盖，对实验数据而言，安时积分法的结果与真实 SOC 的值十分接近，因为数据集或者部分文献的实测数据均来自高精度电池充放电实验设备，相比于实际产品中常用的采集芯片，其采集的电压或者电流具有非常小的误差。因此有必要对前端采集的电流添加噪声来反映真实的安时积分法的预测效果。

并且对于神经网络的训练来说，添加噪声可以提高算法模型的鲁棒性，即使脱离数据集也能更好地兼容。对于 LSTM 网络的训练与验证所使用数据的处理，本文所采取的做法如表 3-1 所示。

表 3-1 训练与验证所使用的数据

	使用数据	覆盖率
训练	原始数据+随机噪声 1	80%
验证	原始数据+随机噪声 2	100%

训练仅使用 80%的数据集是为了防止过拟合，训练数据采取交叠划分随机打乱的方式，交叠划分是指，假设输入是 10s 的时间序列，那么对于一个 100s 的数据集，输入则划分为 1-10、2-11、3-12，以此类推，共计可以产生 91 个 10s 的时间序列，这样做的好处是可以扩大样本数量。验证集添加的噪声 2 与噪声 1 是服从同一分布的噪声，目的是测试模型的鲁棒性，验证集为全集顺序切片验证，因为验证没有打乱和交叠的必要，电池的电量本身是一个连续的量，直接按照时间顺序输入数据得到输出即与真实使用场景一致。

本文噪声的添加方式分两种，一种是固定标准差，一种是自适应标准差。固定标准差是叠加一个服从期望 μ 为 0、固定标准差 σ 的正态分布的噪声，正态分布的数据落在 $[\mu + 3\sigma, \mu - 3\sigma]$ 这一区间的概率为 99.74%。自适应标准差则是根据原始数据的大小按照一定的比例确定 σ 的值。对于电流和电压，添加自适应标准差噪声，对于 SOC 则添加固定标准差噪声。

添加噪声后的电压电流数据与原数据对比如图 3-2 所示，不同的颜色代表不同的标准差，数字代表标准差的百分比，如 0.01 代表标准差为数据值的 0.01%。为了看清图片内容，横轴是没有设置坐标，而是随机选取了 100 个点。在左边两幅对比图可以看到添加 0.01%噪声的数据与原数据重合度较高，添加 1%噪声的数据则有明显的偏离。从右边的误差图可以更直观地看出，以图 3-2(b)为例，1%的噪声误差在 0-50mV 居多，属于噪声较大的情况，0.1%的噪声误差则在 0-5mV，属于噪声较正常的情况，0.01%的噪声误差接近 0，属于比较理想的情况。

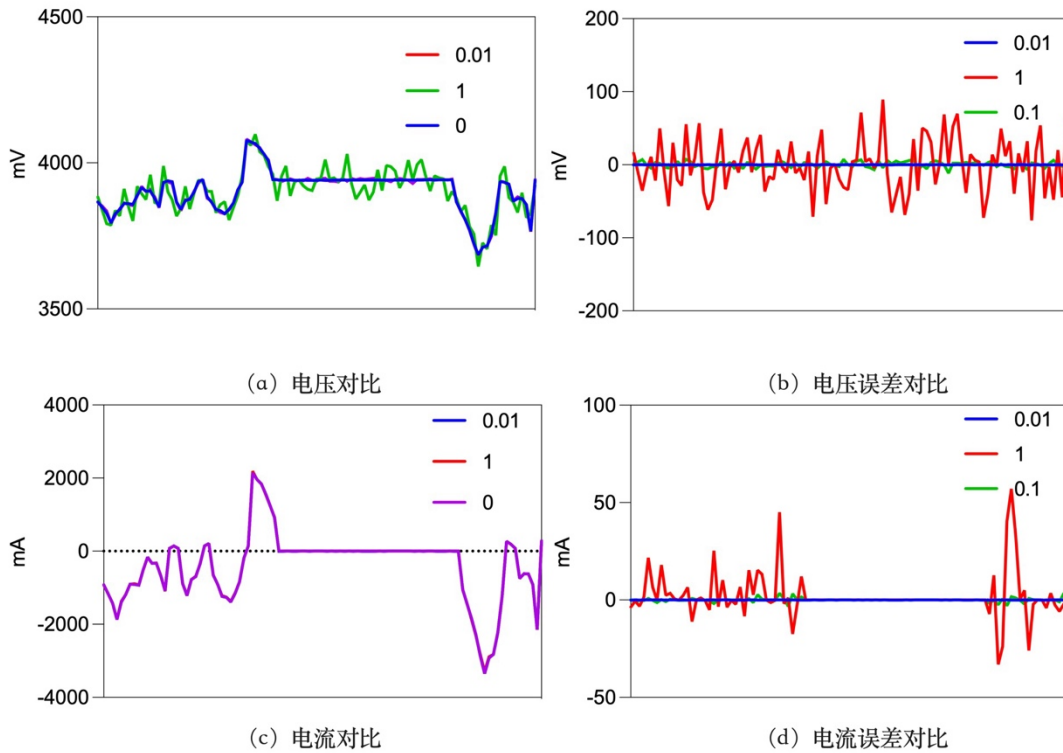


图 3-2 不同标准差大小的噪声添加后与原数据对比

3.2.2 特征矩阵构造

LSTM 网络的输入的特征 X 是一个矩阵，其大小为 $\text{seq_len} \times \text{input_size}$ ，其中 seq_len 是时间步长， input_size 是特征维度。顾名思义，时间步长是输入的时间序列在时间上持续的长度，而特征维度则是在某一时间电池的特征。

先分析特征维度，在本文的第二章中，已经对电池进行建模，电池的电路模型在一定程度上可以反映电池的内部状态，尤其是内阻，与电池的电压变化有较强的关联性，因此从电路模型中提取元件参数作为特征，电池模型的元件参数的查找则通过当前的 SOC 确定，因此将特征提取的过程分以下三个步骤。

1. 特征 1-2。前 2 维特征使用最为关键的测量数据电压 U 和电流 I 。在训练网络时这 2 个特征已经经过上一小节的添加噪声处理。

2. 特征 3。第 3 维特征使用电池 SOC_{pre} ，区别于将要估计的 SOC， SOC_{pre} 是 SOC 这一状态变量的旧值，电池 SOC 的变化可以看作马尔可夫过程，时刻 $t+1$ 的 SOC 与时刻 t 的 SOC 存在较强的关联性，按时积分法的计算公式也体现了这种关联性。要获取 SOC_{pre} ，有两种方法，其 SOC 估计模型如图 3-3 所示：

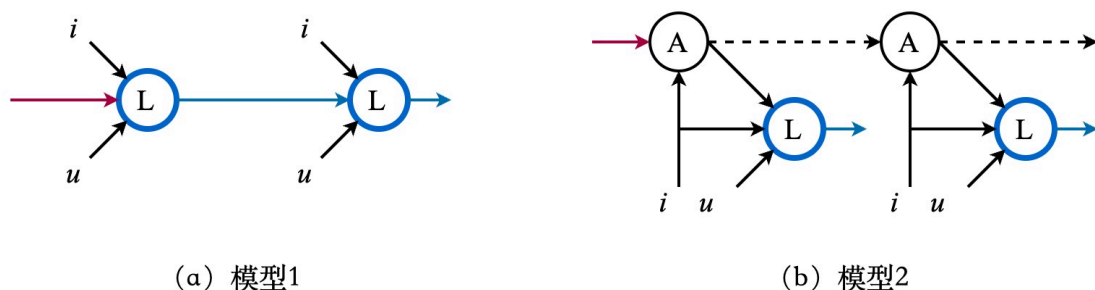


图 3-3 两种 SOC 估计模型示意图

方法一是将上一次估计的结果拿来使用，这种情况下估计 SOC 的模型如图 3-3

(a) 所示，黑色线代表了已知的特征，蓝色圆圈代表了 LSTM 网络，红色线为一个假定常量——SOC 初始值，用来启动整个算法，蓝线代表了 LSTM 网络输出的估计值。实际上，按时积分法、卡尔曼滤波法等采用的都是这种算法，在这种模型中，红线不属于特征，因为它只在一开始被用到，之后每个蓝圈的输入都是黑线与蓝线。然而蓝线作为神经网络的输入则存在弊端，因为他本身是蓝圈的输出，神经网络的训练方式是事先标记好，所有输入与输出一一对应，只输入第一个显然是无法进行训练的，因为训练过程中无法自己产生输入。尽管数据集中有任意时刻的 SOC，可以直接把当前时刻的 SOC 划入输入特征，但真实情况是，这个输入应当是 SOC 的估计值，而非数据集的 SOC 真实值，两者的差别在于 SOC 的估计值是存在偏差的，而 SOC 真实值则建立在当前估计无偏差的情况下训练的，将训练好的模型用于估计时只能用上一次的模型估计值作为输入，会累积偏差，训练时若不考虑这一点，会造成模型的估计结果发散。本文采用的方法

二则是用另一独立算法计算出 SOC_{pre} ，此时 SOC_{pre} 不宜再称作旧值，而是称为先验估计值。这种方法的估计模型如图 3-3 (b) 所示，引入一个特征提取模块圈 A，将黑线与红线输入圈 A 产生一个先验估计 SOC_{pre} ， SOC_{pre} 的计算不依赖蓝圈，因此是黑线， SOC_{pre} 输出到下一个圈 A 又可以产生下一次的 SOC_{pre} ，在这样的结构下，蓝圈的输入全都是黑线，符合神经网络的训练逻辑。这种模型结构常在神经网络与其他一些算法相结合的时候使用，本文中圈 A 使用的算法是安时积分法，因此把圈 A 叫做 AH 模块。

3. 特征 4—8。基于第二章所建立的参数表 2-2，通过前面所计算的 SOC_{pre} 作为索引查找对应的元件参数。

值得一提的是，由于整体算法采用双模式，AH 模块还具备校准算法。AH 模块基于安时积分法，因此其内部的计算初始变量 SOC_{ini} 会用 SOC_{pre} 进行迭代更新，而在睡眠模式下则使用 OCV 模块的计算结果对 AH 模块中的 SOC_{ini} 进行校准。

综上，本文所提出的特征提取算法流程为，电压 U 与电流 I 作为特征 1-2，然后电流 I 输入 AH 模块，输出 SOC_{pre} 作为特征 3，最后 SOC_{pre} 作为自变量输入电路模型参数函数得到特征 4—8。共计 8 个特征，即 $input_size=8$ ，所有特征在表 3-2 中列出。

表 3-2 提取特征列表

维度	特征	注释
1	U	电压
2	I	电流
3	SOC_{pre}	荷电状态先验值
4	Em	电动势
5	R_0	内阻 0
6	R_1	内阻 1
7	τ_1	时间常数 1
8	τ_2	时间常数 2

然后分析时间步长，理论上设置成 1 也是可以的，但作为时间序列，只用一步存在较大的偶然性，时间步设置长一些则可以正确反映状态的趋势，因此本文根据实际情况选择设置为 30，也即 $seq_len=30$ ，量纲是数据集的采样间隔 1s。因此特征提取引擎最终将构造一个 30×8 的特征矩阵。

3.2.3 归一化

归一化（Normalization）是一种常见的处理神经网络输入数据的方法，目的是将数据转换为特定范围或分布，以便更好地适应模型的需求或提高数据的可比性。归一化的方式有很多，有零均值化（Zero-mean normalization）、单位方差化（Unit variance normalization）和最大最小值缩放（Min-max scaling）等。本文采用最常用的最大最小值归一化，其公式如下（3-1）：

$$x = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (3-1)$$

式中 x —待归一化的特征；

x_{max} —该特征在训练集内的最大值；

x_{min} —该特征在训练集内的最小值。

归一化前的数据由于代表的特征各不相同，数值差距较大，归一化使得所有的特征都在一个区间范围内，图 3-4 显示了一组特征归一化前后的数据情况。

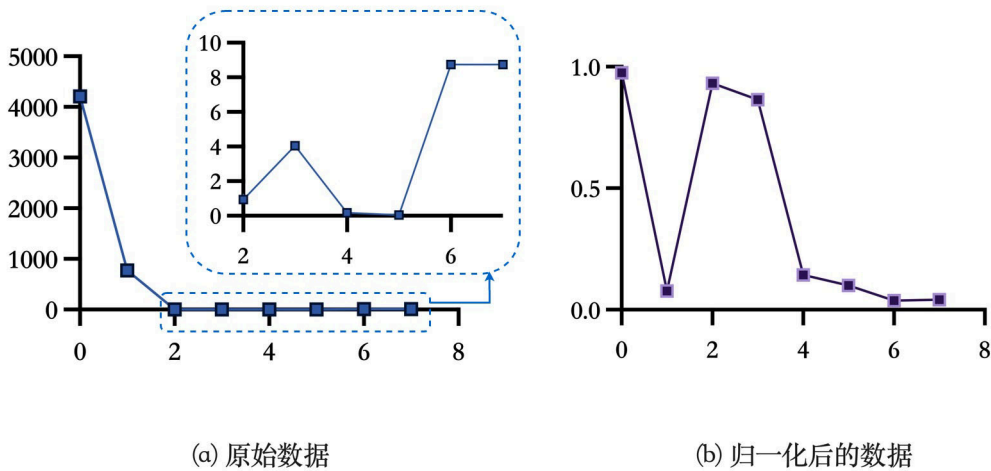


图 3-4 归一化前后对比

归一化的关键在于确定 x_{max} 和 x_{min} ，对于电池充放电数据来说，其工况有无穷多种，数据集只实验了几种典型工况，在有限几种工况中，不一定能涵盖电压电流的全部取值范围，也就是说在实际工况中，电流电压很可能会超出数据集中的最大最小值区间，从而不能归一化在 $[0, 1]$ 这个区间内。并且在数据集中，多数都是放电数据，电流的正负值也呈现不对等的数量关系，针对数据集的特殊性，本文具体情况具体分析，重新定义 x_{max} 和 x_{min} 的确定方式。

受电池本身特性影响，在电池保护系统中设置有放电截至电压和充电截至电压，而将数据集扩展到无穷大时电压的最大最小值将就是这两个值，因此选取这

两个值进行最大最小归一化。同样的，对电流而言，也存在过电流保护阈值，将这个值设置为电池的 $I_{\max}$ ，而最小值设置为 0，这样可以使电流保留正负号，并且在正负的归一化区间上具有对等的关系。而 SOC 本身的取值范围就是 0-1，无需进行归一化处理。对于电池模型特征参数，其所有取值范围也已经固定，因此可以直接导出归一化后的数据取代原数据，无需再进行归一化运算。最终确定的归一化区间如下表 3-3 所示。

表 3-3 归一化参数表

	U(mV)	I(mA)	SOC _{pre}	Em(V)	R ₀ (Ohm)	R ₁ (Ohm)	Tau ₁ (s)	Tau ₂ (s)
MAX	4200	10000	1	4.178	0.284	0.089	211.547	211.548
MIN	2500	0	0	3.234	0.149	0.032	0.828	0.003

3.3 深度学习算法

3.3.1 网络结构

本文提出的算法选择基于 LSTM 小改的网络进行 SOC 估计。理由是对于电池剩余电量这种可以算作时间序列的数据而言，相比于卷积神经网络 CNN，循环神经网络 RNN 具有更好的效果，而作为普通的 RNN 的改进模型 LSTM，因为有记忆单元，其效果更好，代价则是其网络更复杂，参数量更多。式 (3-2) ~ (3-7) 为 LSTM 的核心公式。

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (3-2)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (3-3)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (3-4)$$

$$g_t = \tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g) \quad (3-5)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (3-6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (3-7)$$

式中相乘代表的是矩阵乘法， $\odot$ 代表元素乘法， σ 为 sigmoid 函数。 W_x 与 W_h 都是矩阵，而 x 、 h 、 b 、 c 为向量， x 与 h 的计算可以看成矩阵块与向量的乘积，最终得到一个向量块，如图 3-5 所示。矩阵块的厚度为 4，分别代表 4 个门，即式 2-1 ~ 2-4 的四个 W_x （或 W_h ）。矩阵块的高度称为隐藏层数，记作 hidden_size，其决定

了最后的结果向量块的高度。矩阵块亦由多个向量组成（蓝色柱体），每个向量与 x （或者 h ）作内积得到一个元素（绿色方块），多个元素则构成一个结果向量。而图中蓝色向量的长度对 x 和 h 是不同的，对于 x ，长度为特征的维度 `input_size`，对于 h ，长度则为 `hidden_size`。

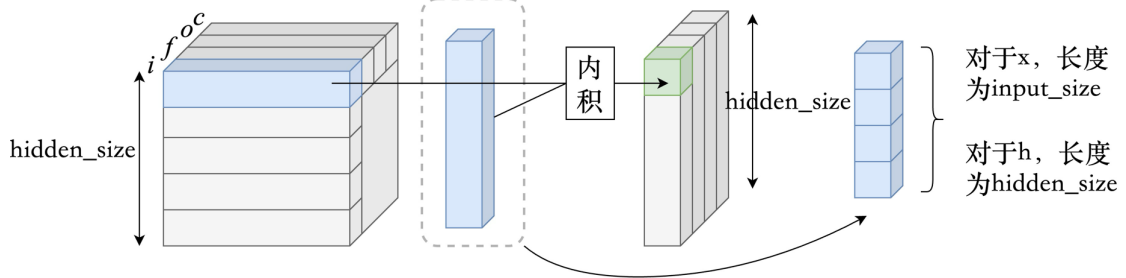


图 3-5 计算过程

x 与 h 计算得到的向量块与四层 b 所构成的向量块具有相同的尺寸，相加得到的四层向量块，即式中的 i_t 、 f_t 、 g_t 、 o_t 。四个向量以及 c 向量之间再进行元素乘法，与内积不同，两个向量的元素积与向量本身的大小相同，仍旧是一个向量，如图所示。因此最后的结果 c 与 h 均为长度为 `hidden_size` 的向量。

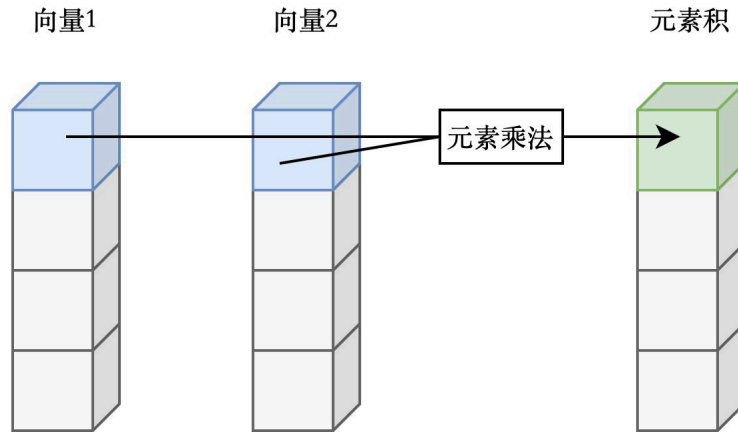


图 3-6 向量的元素乘法

3.3.2 激活函数

由于激活函数 $\tanh$ 和 sigmoid 为复杂的非线性函数，硬件实现代价较大，因此本文在该神经网络的基础上，将激活函数替换为了适合硬件实现的 hardsigmoid 和 hardtanh ，实现方法为采用分段线性函数对激活函数进行近似，公式如（3-8）和（3-9）所示：

$$\text{hardsigmoid}(x) = \begin{cases} 0, & x < -2 \\ 0.25 \times x + 0.5, & -2 \leq x \leq 2 \\ 1, & x > 2 \end{cases} \quad (3-8)$$

$$\text{hardtanh}(x) = \begin{cases} -1, & x < -1 \\ x, & -1 \leq x \leq 1 \\ 1, & x > 1 \end{cases} \quad (3-9)$$

在 hardsigmoid 函数中，之所以采用 0.25 为斜率， $[-2, 2]$ 为分段区间，是因为 0.25 是 2 的负整数次幂，可以直接用移位操作实现，而整数 2 易于比较大小。用于替换的分段函数与原激活函数对比图如 3-7 所示。

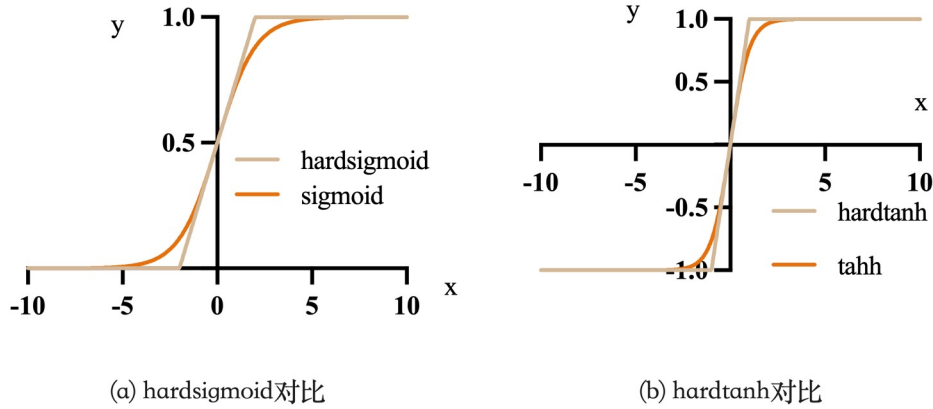


图 3-7 分段函数与激活函数对比

3.2.3 超参数的设置

网络最终确定的各项超参数如下表 3-4 所示：

表 3-4 神经网络参数设置表

参数名	值	注释
input_size	8	输入特征维度
hidden_size	64	隐藏层数
seq_len	30	时间步长
num_layer	2	网络层数
batch_size	64	训练包数
epochs	100	训练回合数
loss function	MSELoss	损失函数
num_linear	1	线性层层数

其中的损失函数公式如下（3-10）：

$$\text{MSELoss} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (3-10)$$

式中 $\hat{y}_i$ 指的是模型预测值， y_i 指的是真值。常用的损失函数还有二元交叉熵、多

元交叉熵等，这些损失函数更适合分类任务，本文选用了最适合执行回归任务时使用的均方差函数 `MSELoss`。

线性层层数 `num_linear` 为 1 层，线性层也称作全连接层（Full Connection）。上一小节已经说明，LSTM 网络的输出 h 是一个长度为 `hidden_size` 的向量，线性层的功能则是紧接在 LSTM 层后，将这一输出向量转换为一个常数。实现该功能仅需要一个长度为 `hidden_size` 的权重向量，以及一个偏置常数，权重向量与输入向量作内积，加上偏置常数输出即得最终结果。

除了所提出的 `input_size=8` 的 LSTM 网络（记作 `ah-lstm`），本文另外还对 `input_size=2` 的 LSTM 网络，即无特征提取引擎，前端数据直接连接的算法（记作 `lstm`）进行了训练测试以供对比，两种算法的训练情况如下表 3-5 所示。

表 3-5 训练情况

平台:Apple M1 Pro 10 Core 3200 MHz		
算法	参数量	训练时长
<code>lstm</code>	50241	1h7m
<code>ah-lstm</code>	51777	1h2m

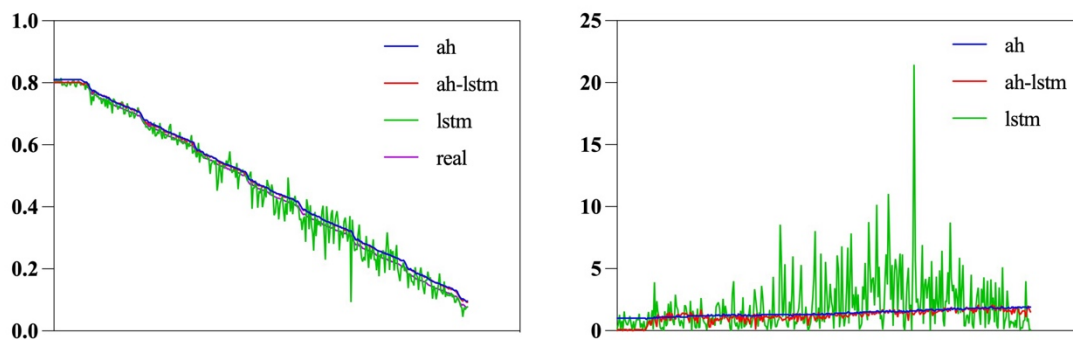
3.4 算法效果分析

3.4.2 估计结果分析

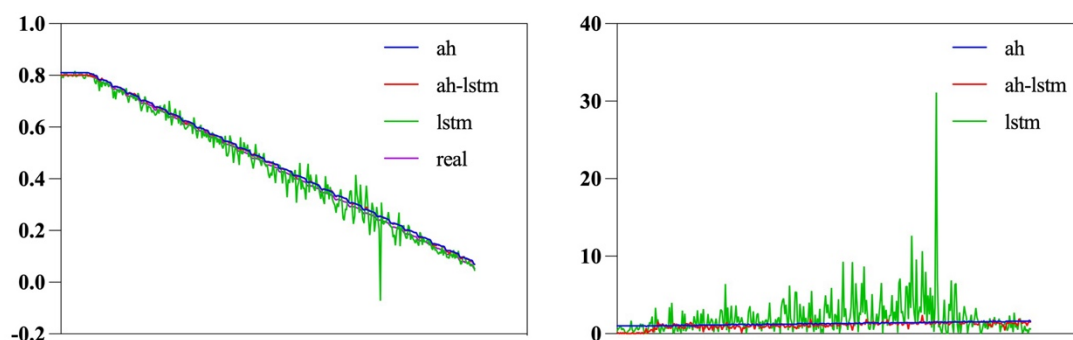
算法性能分析方案如下：

1. 结果分析横向对比三种模型：安时积分法（ah）、单 LSTM 网络（lstm）、本文提出的算法（ah-lstm，因为主要由安时积分与 LSTM 网络构成）。
2. 每种模型分三种训练方式：自适应噪声百分比（用 `sigma` 代替称呼）分别为 0.01、0.001、0.0001。
3. 测试环境为：初始值误差 1%，四种工况 FUDS、DST、BJDST、US06 分别测试，噪声标准差与训练一致。
4. 对比的内容为：三种训练情况下三个模型在四种工况下的估计结果以及估计误差。

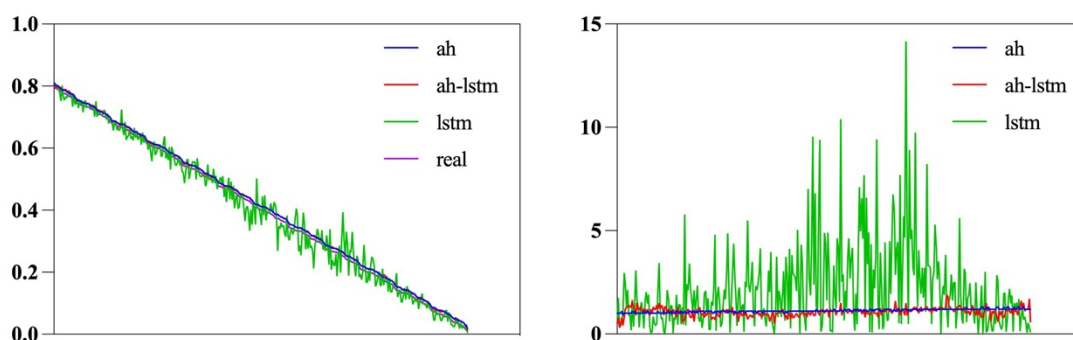
图 3-8~3-10 分别展示了不同训练条件的对比图。为了方便看清线条，将坐标轴都隐去了，横坐标实际为连续的时间，间隔为 30s，纵坐标左图均为 SOC，无量纲，右图则为百分比。



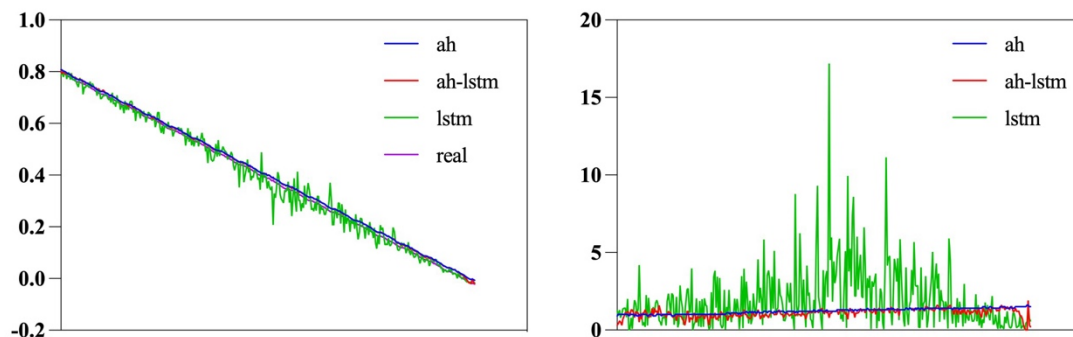
(a) FUDS工况下不同算法估计结果（左）及误差（右）



(b) DST工况下不同算法估计结果（左）及误差（右）

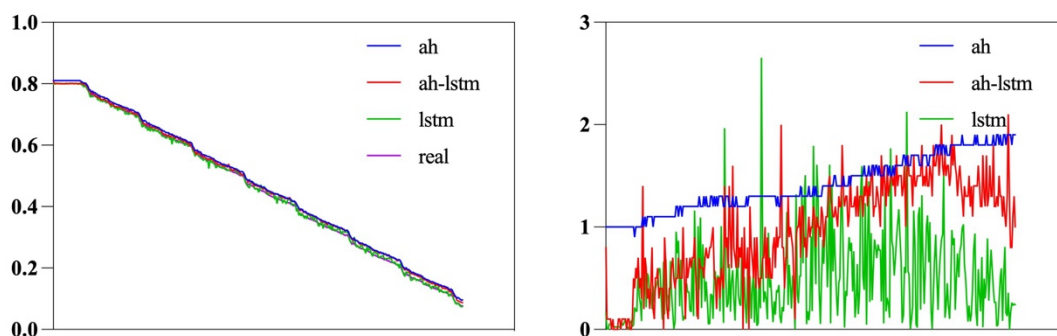


(c) BJDST工况下不同算法估计结果（左）及误差（右）

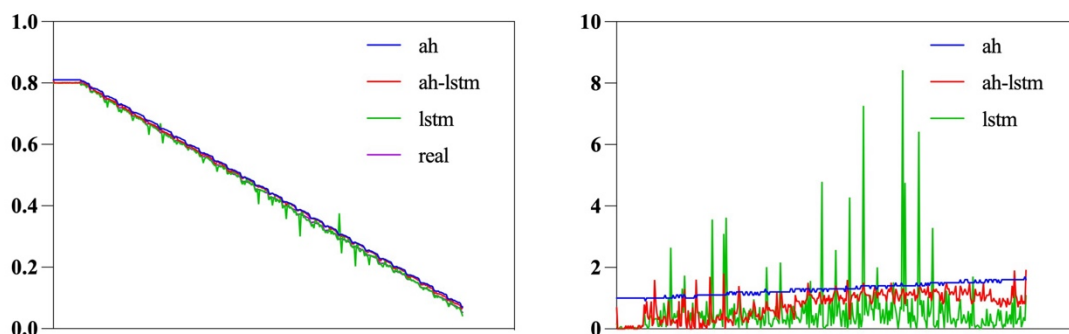


(d) US06工况下不同算法估计结果（左）及误差（右）

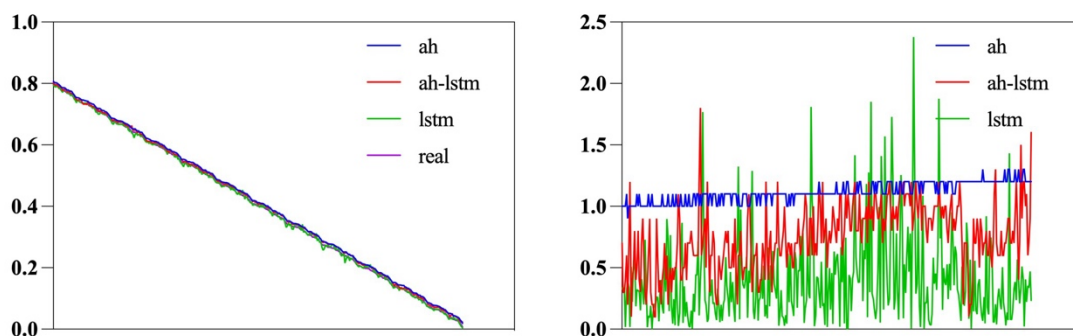
图 3-8 $\sigma=0.01$ 时不同算法的估计结果与误差



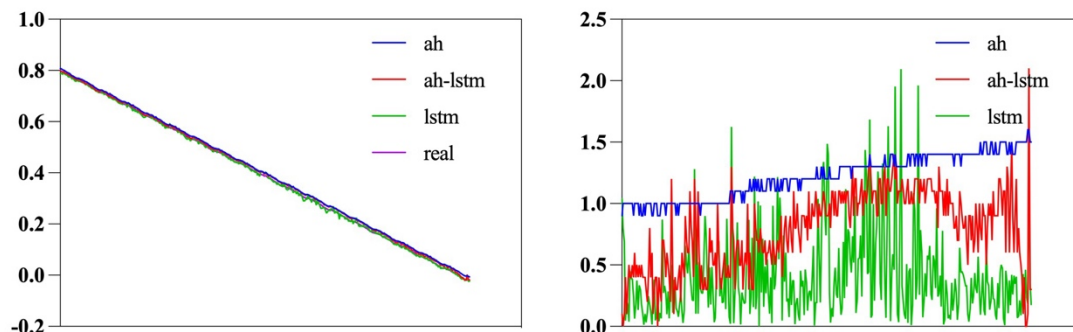
(a) FUDS工况下不同算法估计结果（左）及误差（右）



(b) DST工况下不同算法估计结果（左）及误差（右）

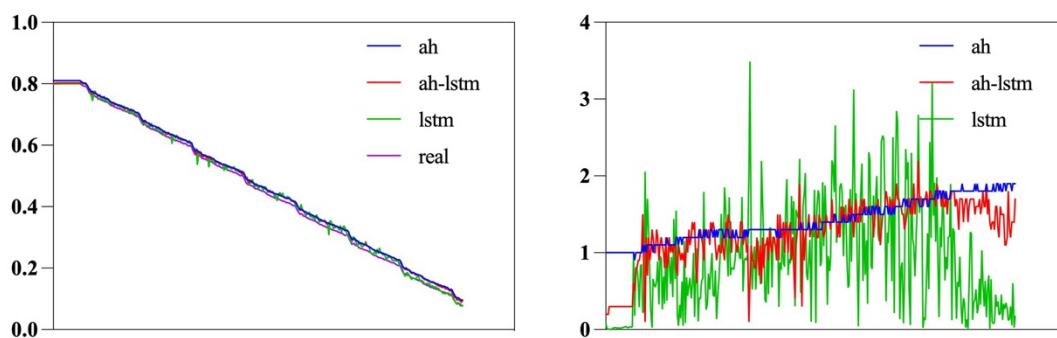


(c) BJDST工况下不同算法估计结果（左）及误差（右）

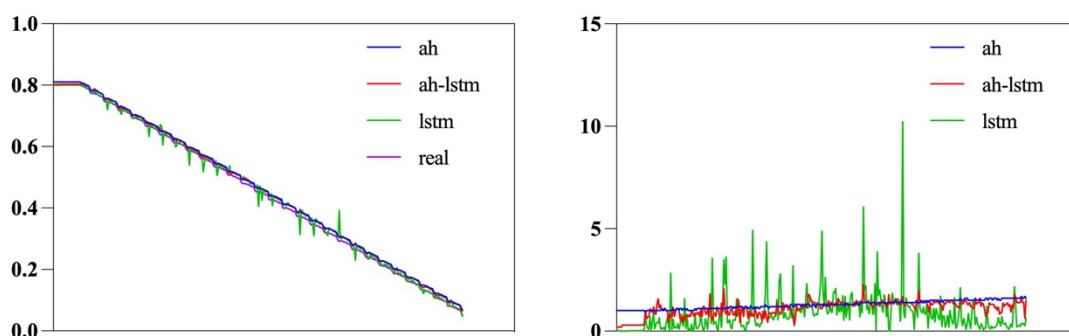


(d) US06工况下不同算法估计结果（左）及误差（右）

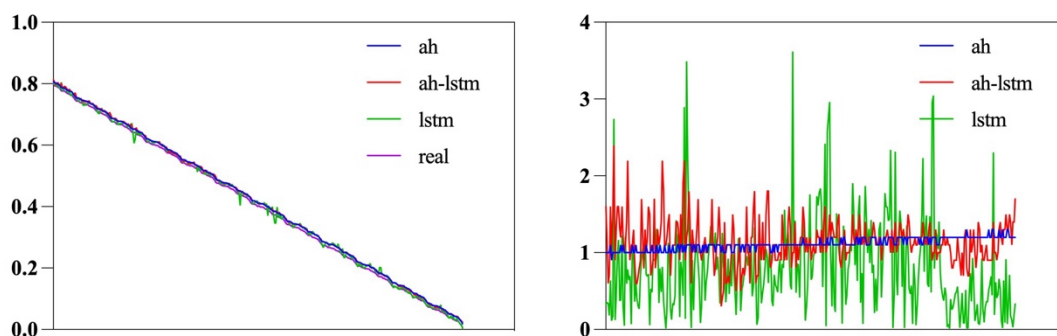
图 3-9 $\sigma=0.001$ 时不同算法的估计结果与误差



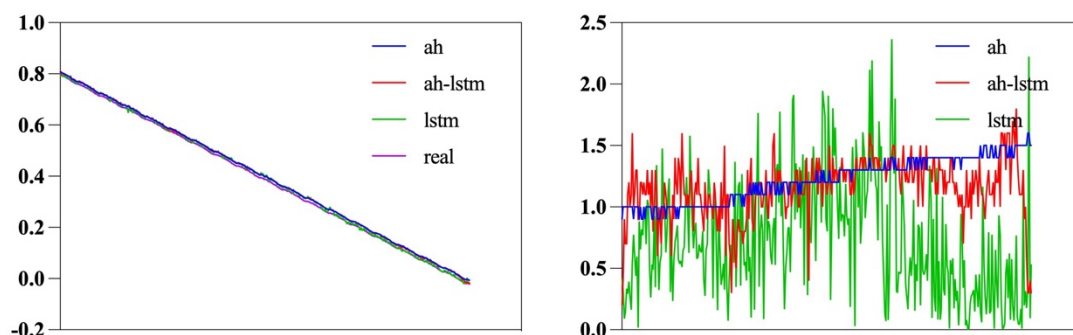
(a) FUDS工况下不同算法估计结果（左）及误差（右）



(b) DST工况下不同算法估计结果（左）及误差（右）



(c) BJDST工况下不同算法估计结果（左）及误差（右）



(d) US06工况下不同算法估计结果（左）及误差（右）

图 3-10 $\sigma=0.0001$ 时不同算法的估计结果与误差

下面从这三幅图中分析各算法的特点。

1. 可以看到安时积分法的优点在于稳定，误差曲线较为平稳，且具有缓慢递增的趋势，这与之前所提到的安时积分法累积误差的缺点相符，其最小误差约等于初始误差 1%，最大误差也不超过 2%。

2. 而单 LSTM 网络波动较大，其误差曲线的大部分都在 ah 的下面，但少部分点具有极大的误差，在训练噪声越大的情况下，lstm 的最大误差也越大。

3. 本文提出的特征提取引擎结合 LSTM 算法 ah-lstm 则兼具了上述两种算法的特点，其大部分点的误差小于 ah，且最大误差仅略大于 ah，从图上看，ah-lstm 的曲线总体介于 ah 与真实曲线 real 之间。与 lstm 相反的是，在添加噪声较大的情况下，ah-lstm 甚至具有更低的整体估计误差。

3.4.1 量化指标分析

式 (3-11) ~ (3-13) 列出了用来评估回归任务性能的三个常用量化指标及其公式：

$$MAE = \frac{1}{n} \sum_{i=1}^n |\widehat{soc}_i - soc_i| \quad (3-11)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\widehat{soc}_i - soc_i)^2} \quad (3-12)$$

$$ME = (|\widehat{soc}_i - soc_i|) \quad (3-13)$$

其中 MAE 为平均绝对误差，RMSE 为均方误差，ME 为最大绝对误差。表 3-6 ~ 3-8 列出了三种不同训练条件的指标结果。每个表包含 4 种不同工况下 3 种算法的三个性能指标，所有指标均为数值小的为优。下面根据这三个表分析各算法的特点。

1. 安时积分法 (ah) 即便在噪声最大的 0.01 时最大误差 ME 也未超过 2%，且其 ME 受噪声的影响较小，ah 的 ME 在所有情况下都取得了最优值。其缺点在于，在设置了 1% 的初始误差情况下，安时积分法从一开始误差就达到了 1% 且缓慢增加，因此 MAE 和 RMSE 在所有情况下都大于 1%，且在 sigma 较小的两

种情况下，ah 的 MAE 与 RMSE 这两个指标均为最差值。

2. 单 LSTM 网络算法 (lstm)，该算法为不提取其他特征，只输入电流与电压归一化的值进行训练，在 SOC 估计的过程中，神经网络的高鲁棒性高体现在是其不受初值误差的影响，具有较小的 MAE 和 RMSE。lstm 的 ME 在所有情况下均为最差，推测这种结果是由于，在该算法模型中，未来的 SOC 与当前的 SOC 无关，只取决于负载情况。在误差最大的情况下，lstm 在所有指标上取得了最差结果，仅在误差最小的情况下，lstm 存在一部分优势，这是由于 lstm 纯粹基于数据驱动，受训练数据的影响较大。误差较大时测试数据与训练数据差别较大，且受限于样本数量，训练过程有可能存在过拟合的情况，只有通过更大的训练样本才能改善这种情况。

3. 本文提出的算法 (ah-lstm)，从表中可以看到所有情况下，ah-lstm 的指标均为最优或者次优，没有出现最差的指标。且在误差较大的两种情况下 ah-lstm 的 MSE 与 RMSE 均为最优。缺点在于，其具有与 lstm 相似的特点，即存在少数点偏大的情况，导致 ME 指标不如 ah，但与 ah 的 ME 十分接近，总体来说是三种算法中最优的。

表 3-6 sigma=0.01

Working Conditions	Algorithm	MAE(%)	RMSE(%)	ME(%)
FUDS	ah	1.461	1.490	1.900
	lstm	1.918	2.877	21.448
	ah-lstm	1.256	1.340	2.100
DST	ah	1.302	1.318	1.700
	lstm	1.989	3.135	31.131
	ah-lstm	1.091	1.168	2.400
BJDST	ah	1.128	1.131	1.300
	lstm	2.077	2.874	14.152
	ah-lstm	1.055	1.079	1.900
US06	ah	1.215	1.228	1.600
	lstm	1.917	2.723	17.189
	ah-lstm	1.092	1.126	1.900

表 3-7 $\sigma=0.001$

Working Conditions	Algorithm	MAE(%)	RMSE(%)	ME(%)
FUDS	ah	1.455	1.485	1.900
	lstm	0.471	0.619	2.653
	ah-lstm	0.999	1.107	2.100
DST	ah	1.303	1.320	1.700
	lstm	0.538	1.026	8.417
	ah-lstm	0.797	0.906	1.900
BJDST	ah	1.120	1.123	1.300
	lstm	0.389	0.525	2.379
	ah-lstm	0.735	0.787	1.800
US06	ah	1.218	1.231	1.600
	lstm	0.406	0.543	2.095
	ah-lstm	0.785	0.849	2.100

表 3-8 $\sigma=0.0001$

Working Conditions	Algorithm	MAE(%)	RMSE(%)	ME(%)
FUDS	ah	1.456	1.485	1.900
	lstm	0.875	1.107	3.487
	ah-lstm	1.294	1.357	2.200
DST	ah	1.304	1.321	1.700
	lstm	0.821	1.265	10.242
	ah-lstm	1.124	1.183	2.300
BJDST	ah	1.120	1.123	1.300
	lstm	0.782	0.982	3.617
	ah-lstm	1.126	1.164	2.400
US06	ah	1.219	1.232	1.600
	lstm	0.734	0.875	2.366
	ah-lstm	1.151	1.177	1.800

另外需要说明的是, 由于不同的研究人员对电池型号、实验数据工况的选取等不尽相同, 加之数据的处理方式、测试的方式也多有不同, 很难统一变量进行算法的性能对比, 因此本文选择完全相同条件下的三种算法进行对比, 而非与国内外其他研究对比。但大多研究都选取了 RMSE 这一指标作为算法的性能评判指标, 因此在不考虑上述变量的情况下, 本文进行一个不严谨的对比, 如表 3-9 所示, 以供参考。三种噪声条件下的本文提出算法的平均 RMSE 分别为 1.571%、0.912%、1.174%, 表中将本文算法记作 AH-LSTM, RMSE 为 0.912%。

表 3-9 与其他研究对比

文献 ^[53]							文献 ^[50]	文献 ^[48]	本文
算法	EKF	AEKF	UKF	AUKF	UKF- NN	AUKF- NN	Nonlinear Observer	ANN	AH- LSTM
RMSE(%)	0.79	1.317	1.48	4.165	1.128	0.7	2.23	1.36	0.912

3.5 网络所占用存储空间

神经网络的参数（权重、偏置）在硬件设计中是存储在 Memory 中的，其需要的存储空间大小与神经网络的超参数选取相关，本节对其进行计算，为方便描述，本节中对涉及的超参数用首字母表示，详见表 3-10。

表 3-10 本节使用的缩写符号

参数	含义	符号	值
hidden_size	隐藏层数	H	64
input_size	输入特征维度	I	8
num_layer	LSTM Cell 层数	N	2
data_width	数据位宽	D	16

对于多层 LSTM，第二层的输入 x 是第一层的 h ，因此两层的 W_x 大小不同，要分开计算，但二层及以上与二层均相同。对第一层来说，每个门的参数量 $PPG1$ （Parameters per gate of layer1）计算公式为式（3-14）：

$$PPG1 = H \times I + H \times H + H \quad (3-14)$$

式中三个加数分别为 x 的权重、 h 的权重、偏置，一层共有 4 个门，因此第一层的参数总量 $PL1$ （Parameters of layer 1）计算公式为式（3-15）：

$$PL1 = 4 \times PPG1 = 4H^2 + 4(I + 1)H \quad (3-15)$$

第二层的 x 的权重与 h 相同，因此第二层的每个门参数量 $PPG2$ 与总参数量 $PL2$ 公式分别为下式（3-16）与（3-17）：

$$PPG2 = H \times H + H \times H + H \quad (3-16)$$

$$PL2 = 4 \times PPG2 = 8H^2 + 4H \quad (3-17)$$

整个网络总参数量 P （Parameters of network）还取决于线性层参数量 L （Parameters of linear），本文中线性层为 1 层，共 64 个权重，1 个偏置，因此 $L=65$ ， P 的计算公式为第 1 层、第 2~N 层、线性层的参数之和，如下式（3-18）：

$$\begin{aligned}
P &= PL1 + (N - 1) \times PL2 + L \\
&= 4[(2N - 1)H^2 + (N + I)H] + L
\end{aligned} \tag{3-18}$$

将表中的值代入计算得 $P=51777$ ，与表 4-5 中 Python 计算的一致，从表达式也可以看出 P 受 H 的影响较大，受 I 的影响较小，因此 2 维特征与 8 维特征在参数量上相差不到 3%。占用的存储空间则取决于 D ， P 乘以 D 则是比特数，再除以 8 就是占用的字节数 NB (Number of bytes)，其计算公式为下式 (3-19)，最终算得 $NB=103554\text{Byte}$ ，即需要 103KB 的 Memory。

$$NB = \frac{1}{8}PD \tag{3-19}$$

3.6 本章小结

本章详细阐述了动态模式算法，首先提出了更加符合实际的数据的处理方式，包括给数据集数据加入噪声、归一化。接着阐述了特征提取算法，该算法由安时积分法提取特征 SOC_{pre} ，然后构造特征矩阵。最后介绍 LSTM 网络的结构、训练方式、测试结果。本章通过横向对比，评价了所提出算法的性能，分析说明了本文提出的算法相比于其他两种算法具有更好的效果。

第四章 电路设计

4.1 本章引言

上一章提出了双模式的算法，并在 Python 平台环境下对动态模式算法进行实现与优化。但由 Python 语言所编写的算法抽象级别较高，只能运行在通用计算机上，为了将算法部署到本地 ASIC，本章将对行为级模型进行修改和转换，实现基于 Verilog 硬件描述语言的寄存器传输级设计。

本章根据设计过程撰写，首先对基于 Python 模型的各项参数进行定点化处理，然后介绍了芯片的顶层架构，接着对每个模块进行详细的阐述，最后进行小结。

4.2 定点化

4.2.1 定点化方法

将基于 Python 代码的算法转换为硬件电路的第一步便是定点化，在 Python 中，数据都是用 32 位浮点数表示，而硬件电路使用二进制整数。定点化的方法有多种，本文使用四舍五入法。这种方法需要确定两个参数：定点数的总位宽 $data_width$ 与小数位宽 $frac_width$ 。

定点化的流程为，浮点数 x_{float} 乘以 2^{frac_width} 再四舍五入取整便得到其定点数 x_{fix} ，此时 x_{fix} 转换成 $data_width$ 位二进制数，则其后 $frac_width$ 位为小数部分，若要还原为浮点数，将 x_{fix} 除以 2^{frac_width} 即可，显然还原后会与原来的数存在误差，误差的来源则是定点化时的取整操作，取整操作会丢掉小数部分，而丢掉的这个小数部分的大小取决于 $frac_width$ ，因为丢掉的小数部分最大不超过 $1/frac_width$ ，因此更大的位宽意味着更高的精度，但同时意味着更大的硬件开销。式 (4-1) 与 (4-2) 定义了定点化函数 f_{fix} ：

$$a_{fix} = f_{fix}(a_{float}) \quad (4-1)$$

$$f_{fix}(x, scale) = round(x \cdot 2^{scale}) \quad (4-2)$$

其中 $round$ 为四舍五入操作，自变量 x 为待定点化的数， $scale$ 为量化尺度，与 $frac_width$ 相同。

对 $data_width=16$ ， $frac_width$ 分别为 12 位和 13 位的两个定点模型，随机选

了 100 个输入，进行结果对比，如图 4-1 所示。从图 4-1 (b) 可以看到 12 位的平均量化误差在 4% 左右，而 13 位的平均量化误差在 2% 左右，量化尺度也决定了定点数的位宽 `data_width`，要求能保证所有的参数定点化后不溢出。本文使用 12 位的量化尺度，加上 1 位符号位，3 位整数位，数据正好不会溢出，共计 16 位，若使用 13 位则 `data_width` 的最低要求为 17 位，`frac_width` 与 `data_width` 在设计中均为可配置的参数。

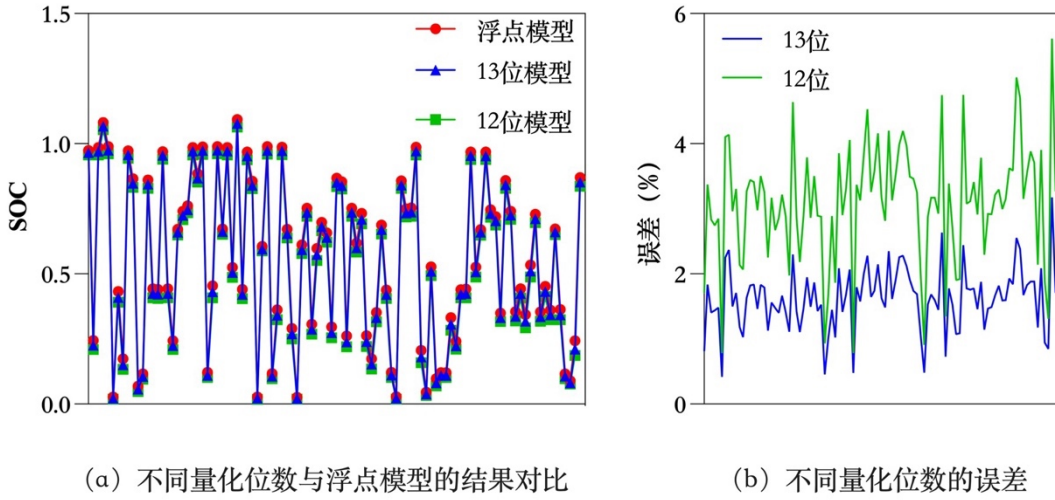


图 4-1 不同的量化尺度的精度对比

4.2.2 定点数乘法

固定小数点位置后，乘法的运算方式也要做出相应的改变。设计一个定点乘法模块来代替所有的乘号操作，浮点乘法的公式为式 (4-3)，替换为硬件乘法后的公式为式 (4-4)：

$$c_{float} \approx a_{float} \times b_{float} \quad (4-3)$$

$$c_{fix} = \text{hardmul}(a_{fix}, b_{fix}) \quad (4-4)$$

硬件乘法的计算过程如图 4-2 所示，虚线框内的部分是硬件乘法模块 `hardmul`。

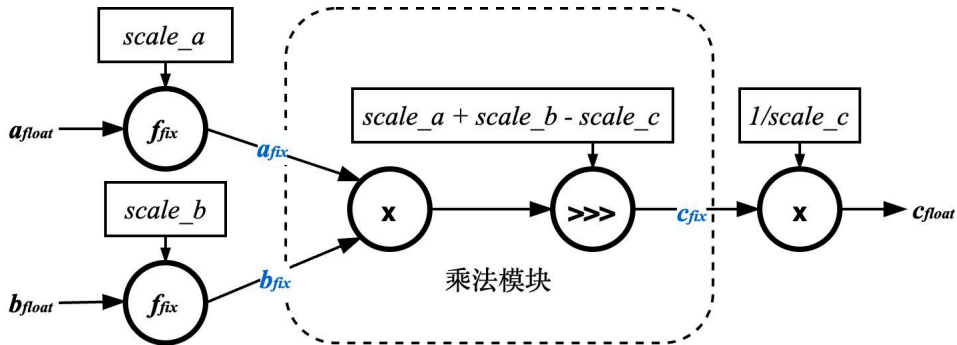


图 4-2 定点乘法模块示意图

从图中可以看到在乘法过程中,首先对浮点数进行量化,然后输入乘法模块,输出量化结果再还原为浮点数,虚线框外部分为外部实现,硬件只负责定点数的运算。总的来说,乘法模块的主要功能是完成乘法并把结果转换到指定的量化尺度。该模块的详细接口如下表 4-2 所示。

表 4-2 乘法模块信号列表

信号	类型	注释
a_{fix}	input	乘数 1
b_{fix}	input	乘数 2
c_{fix}	output	乘积
$scale_a$	parameter	乘数 1 的量化尺度
$scale_b$	parameter	乘数 2 的量化尺度
$scale_c$	parameter	乘积的量化尺度

4.3 整体架构

在确定了算法框架的情况下,设计方式仍有多种,针对 PPA (Power、Performance、Area) 的平衡与取舍,大致可以分为面向性能设计或者面向能效设计,通常我们认为高性能和低功耗一般处于对立面,但在本文的设计中,该芯片的工作情况是以固定的频率重复执行一次计算任务,并且受限于前端采集的精度(过高频率的采集对缓慢变化的数据是无意义的),这个频率比较低,本文选取的频率为 0.03Hz,这种情况下整个设计具有较大的灵活性,可以采用高性能的设计缩短执行时间,也可以采用低性能的设计减小晶体管数量,将功耗耗用 $W = pt$ 表示,当功率 $p_1 > p_2$, 时间 $t_1 < t_2$, 具体 W_1 和 W_2 的关系无从得知,但考虑到低性能的设计方式可以节省面积,本文采用了的设计原则为,在关键的计算步骤采用并行计算和流水线设计,其他大部分则采用串行计算。

图 4-3 给出了芯片顶层的架构图,模块之间属于串联运行的关系,每个模块通过给出有效脉冲信号驱动下一模块,其驱动路径如图中彩色箭头所示,编号表示该路径的顺序,首先由模式选择模块向 SPI 驱动模块发起读取数据信号, SPI 驱动模块会向外部读取数据并在读取完成发送数据有效信号回到模式选择模块,模式选择模块判断完成后发出脉冲信号,该信号在橙色与紫色路径二选一,橙色路径下启动 OCV 模块计算 SOC,完成后触发 SPI 驱动模块将数据发送到外部;

紫色路径下启动特征构造模块，特征将写入神经网络模块的输入特征 buffer 中，写入完成后发送一个脉冲信号启动神经网络计算 SOC，计算完成同样触发 SPI 驱动模块送出数据。

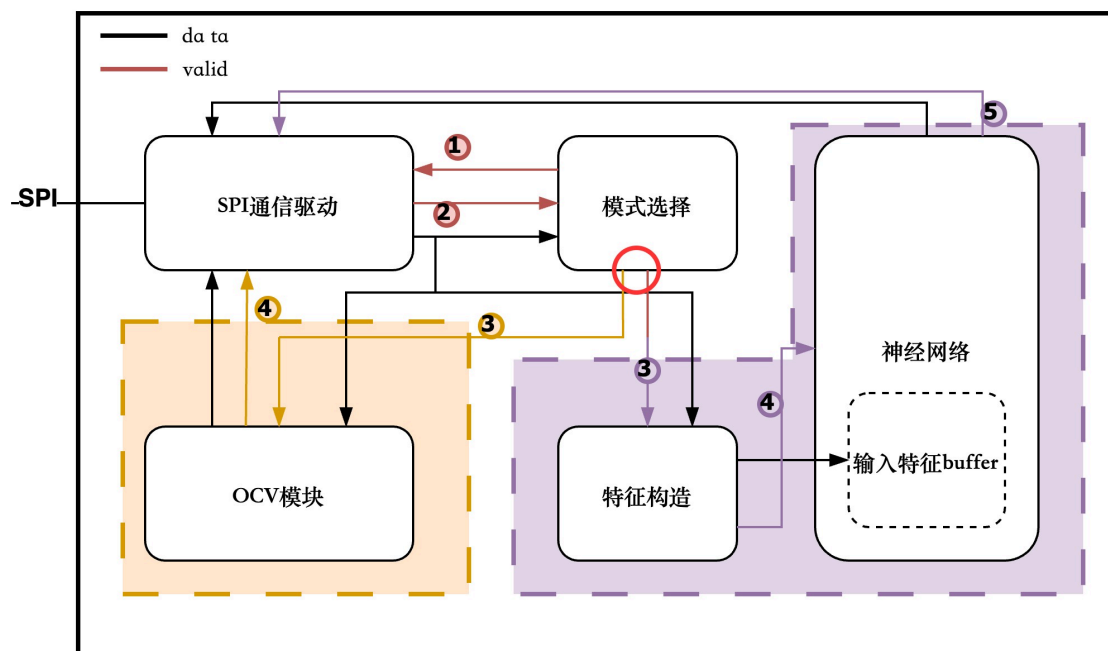


图 4-3 顶层连接

4.4 各模块设计

4.4.1 SPI 通信模块

为了节省芯片的 IO 口数量，本文采用 SPI 串行通信协议与外部交换数据，因此设计了一个 SPI 通信模块，负责发送读写数据的指令以及将要发送的数据并转串输出、将要读取的数据串转并输出，由于该模块产生协议时钟 $sclk$ 以及有效信号 csn ，因此是 SPI 主机。下面分别介绍该模块在读取和发送模式下的时序。

SPI 主机模块的读时序如图 4-4 所示，在时钟上升沿检测到读取数据指令时，将状态更改为读数据，在读数据状态下，计数器在时钟上升沿计数，同时拉低 csn 信号、产生 $sclk$ 信号，通知从机开始接收数据。 $sclk$ 信号为时钟的二分频，在 $sclk$ 的第一个下降沿拍出 $mosi$ 信号 0，代表要求读取数据，从机收到后会准备数据，并在第 3 个下降沿拍出第一 bit 数据，主机则第 3 个上升沿采样该数据，在第 4 个下降沿从机拍出第 2bit 数据，第 4 个上升沿主机采样，如此循环往复，直到从机发送完毕（图中为读取 2bit）， csn 信号拉高， $sclk$ 不再翻转，状态更改为空闲，表明这一次读操作结束。

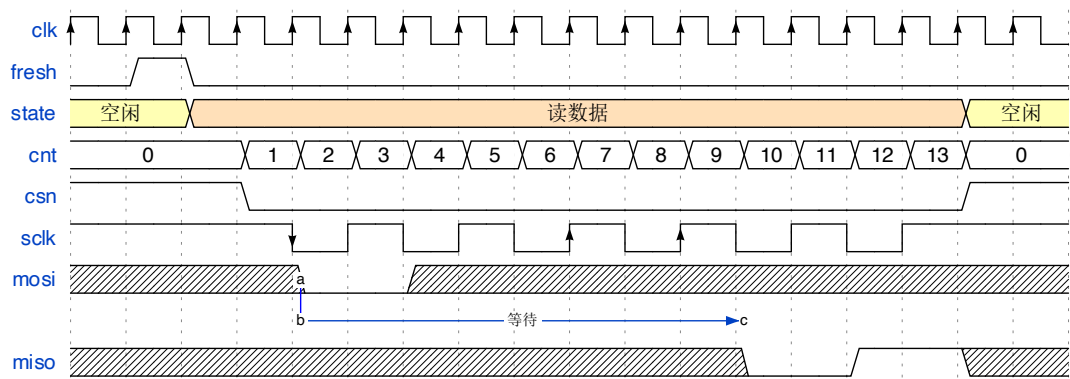


图 4-4 SPI 主机模块读时序

SPI 主机的写时序如图 4-5 所示，与读所不同的是，写数据时对 miso 信号直接忽略，在第 1 个下降沿拍出 1，代表要求写入数据，在之后的每个下降沿拍出要写入的 1bit 数据，直至发送完毕（图中为发送 3bit）。从机则在每个上升沿采样，不发送任何数据。

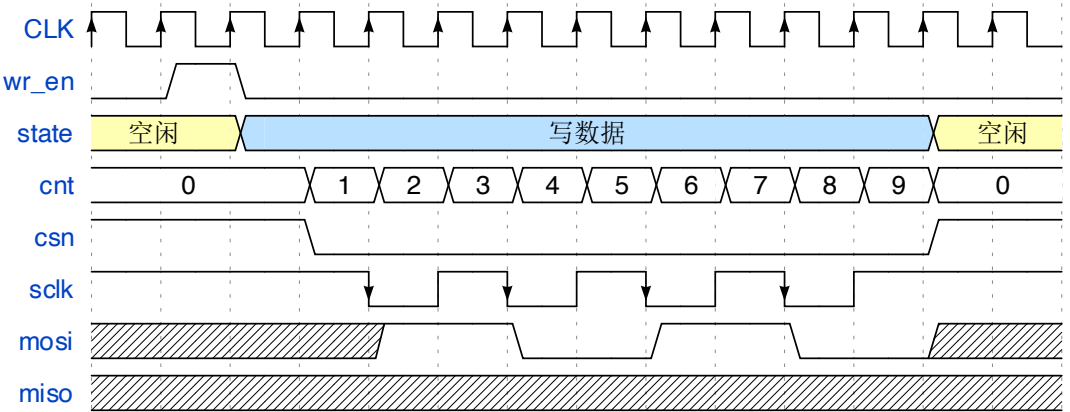


图 4-5 SPI 主机模块写时序

4.4.1 模式选择模块

此模块判断芯片工作在动态模式还是睡眠模式，如图 4-3 所示，其输入为电流 I，输出为两个有效信号 valid0 和 valid1，分别连接到动态模式的特征构造模块和静态模式的开路电压模块（OCV 模块）。

该模块内部对输入的电流 I 进行判别并计数，根据设置的两个参数：小电流阈值、最大计数值进行判别。具体的当电流小于该阈值时，计数器加 1，如果大于该阈值，计数器清零，当计数器计数到最大计数值，判定设备处于空闲状态，芯片进入低功耗模式，拉高 valid1，启动 OCV 模块，反之拉高 valid0，启动特征构造模块。

4.4.2 特征构造模块

特征构造模块的功能包括：对输入电压 U 和电流 I 进行归一化得到 u_norm 和 i_norm 、用安时积分法计算一个 soc_ah 、使用 soc_ah 产生地址读取出其他特征，最后将特征按照顺序逐个输出。整个数据流如图 4-6 所示。

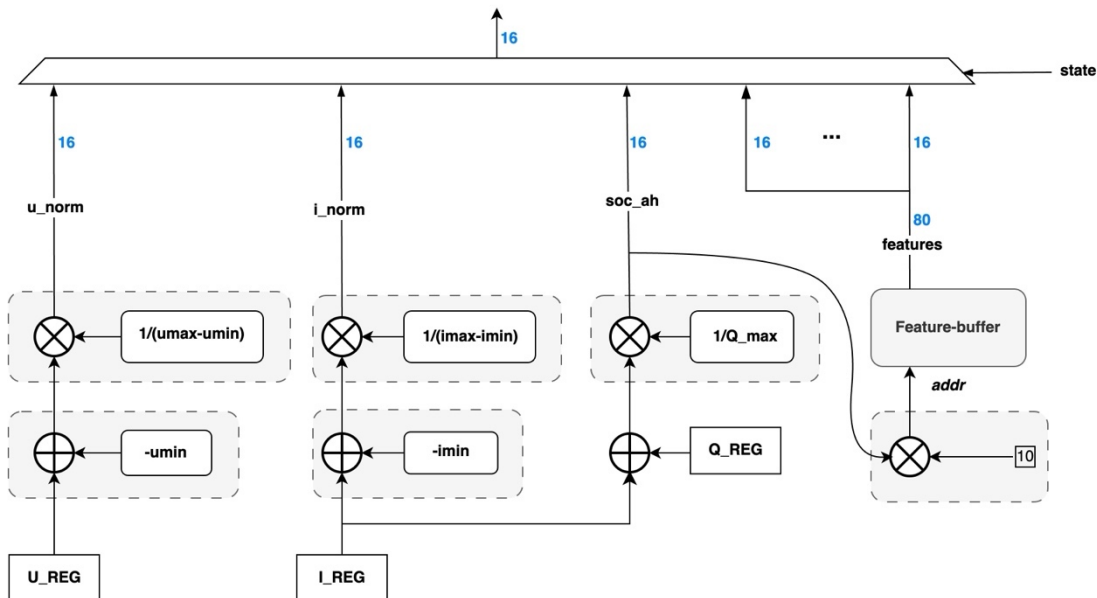


图 4-6 特征构造数据流

从图 4-6 中可以看到，此模块共有四条计算分支，从左至右分别是电压归一化、电流归一化、安时积分以及特征查找。并且左边三条计算分支上均使用了加乘结构，因此采取对一个加乘单元多次复用的方法，从左至右按顺序串行执行四条计算分支。四条分支的结果在 $state$ 信号的选择下按顺序输出到下一级模块中。

需要注意的是对于不同的参数，量化尺度差距非常大，例如 u_{norm} 参数是 2^{-10} 数量级，而 Q_{max} 是 2^{-20} 数量级，而一个乘法器的量化变化尺度是固定的，因此对三条支路来说乘法器输出的结果的量化尺度是不相同的，需对结果再各自进行不同位数的右移缩小使得量化尺度归到统一标准。

4.4.3 神经网络模块

4.4.3.1 整体结构

这一部分是整个算法的核心部分，也是最复杂的部分，因为涉及大量的数学运算，运算的内容实质就是上一章介绍的 LSTM 公式，从公式可以看出，LSTM 由以下几种运算组成：矩阵乘法、元素乘法、加法、激活函数。实现 LSTM 网络的方式也不止一种，设计的核心问题在于乘法、加法等大量使用的单元如何分时

复用。本文提出的 LSTM 网络模块架构如图 4-7 所示。整个模块按功能划分为三个部分：控制逻辑、存储、数据通路。控制逻辑即图中 FSM 部分，存储包括 Memory Bank 以及 3 个 Buffer，数据通路为 LSTM Cell 部分。

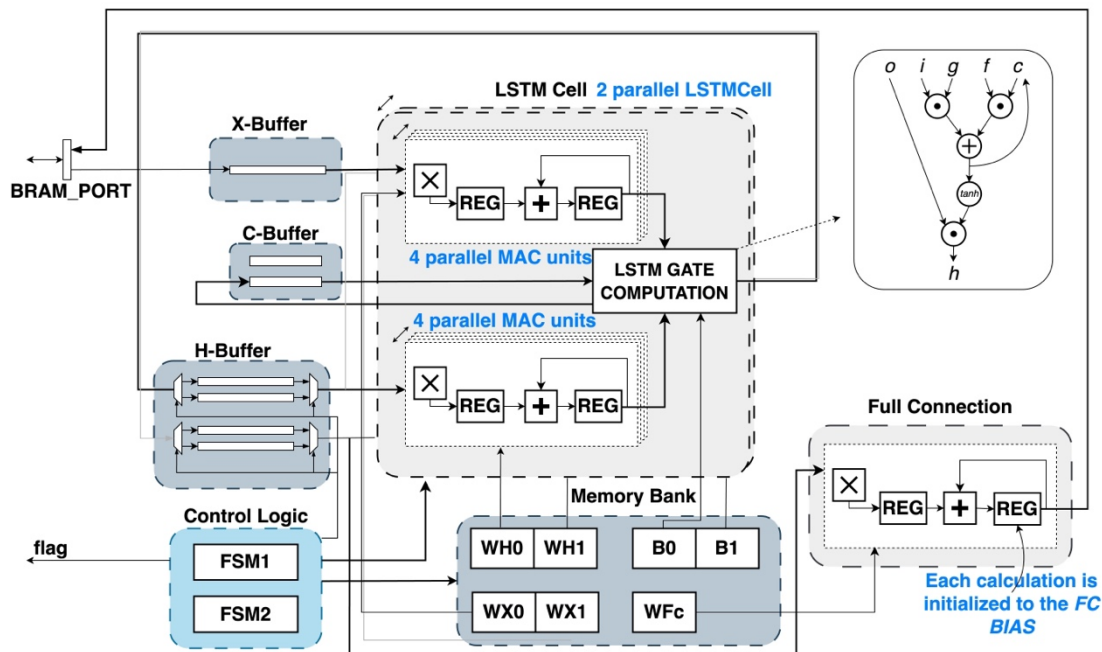


图 4-7 神经网络模块架构

4.4.3.2 控制逻辑

控制逻辑部分负责控制整个计算的进程，给数据通路控制信号，给存储地址信号，而数据则只在存储和数据通路之间流动，如下图 4-8（a）所示。

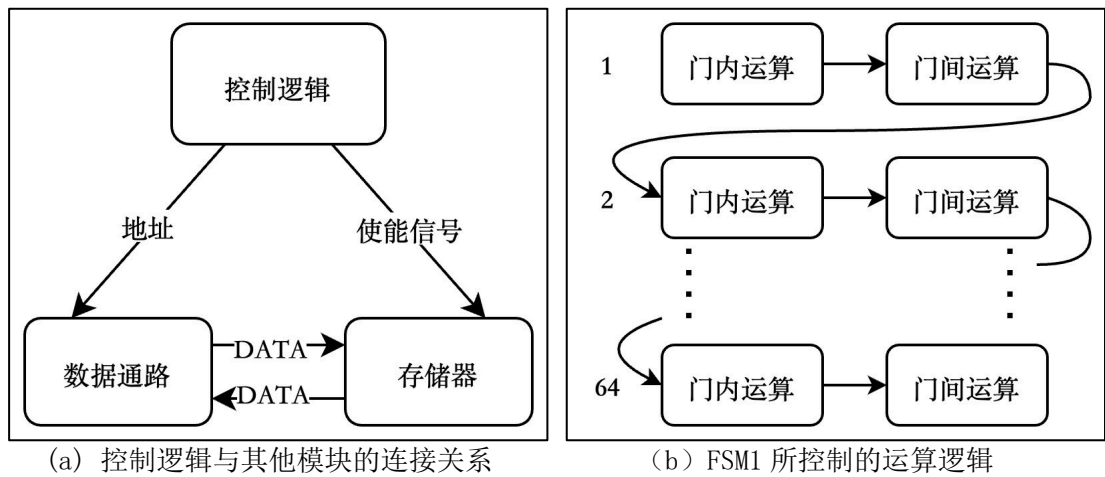


图 4-8 控制逻辑

代码上控制逻辑由两个状态机驱动。状态机 FSM1 控制 LSTM Cell 内部的运算进程，FSM2 控制两个 LSTM Cell 间的运算进程。从图 4-7 可以看到 Cell 的

内部结构包 8 个 MAC (乘累加) 单元与一个门计算单元, 从公式可以看到 LSTM 网络共有 4 个门, 每 2 个 MAC 单元负责一个门的计算, 计算门的过程称作门内计算, 而 4 个门之间的相互运算称作门间运算。

FSM1 控制运算的顺序如图 4-8 (b) 所示, 首先是门内运算, 接着是门间运算, 之后又是门内运算, 循环往复 64 次之后便完成了一个 Cell 的内部计算, 其中 64 代表的是网络的隐藏层数 `hidden_size`。其次是 FSM2 控制的门间运算逻辑, 对于双层 LSTM 网络, 第二层的 x 向量就是第一层的输出 h 向量, 因此第二层的运算要等到第一层运算结束, 本文采用两级流水的方式, 总共需要 31 个 `cell_cycle` (CELL 完成一次计算称作一个 `cell_cycle`), 相比串行计算的 60 个 `cell_cycle`, 可以节约 48% 的时间。如图 4-9 所示, 在第一个 `cell_cycle`, 第一层 LSTM 网络 (简称 L1) 单独计算, 第 2~30 个 `cycle` 里 L1 与 L2 同时计算, 但 L1 计算的是领先 L2 一个时间步, 在第 31 个 `cycle`, L2 单独完成最后一个时间步的计算, 最后结束输出给线性层 Linear Layer。

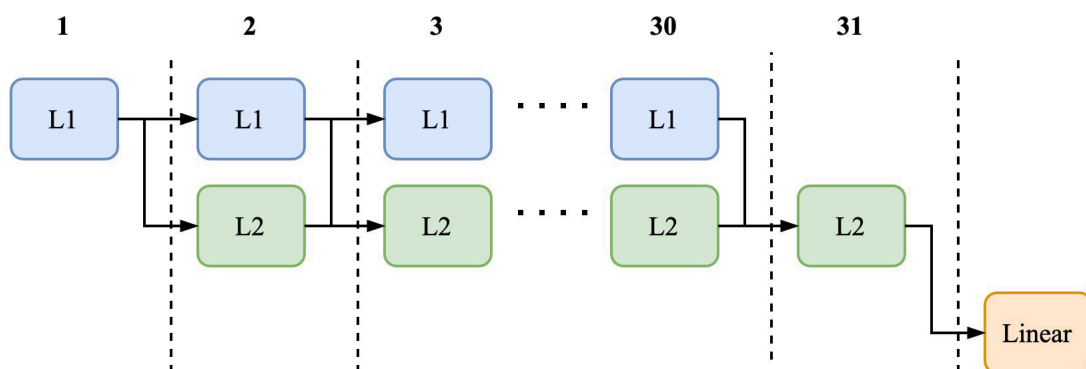


图 4-9 FSM2 所控制的时序

4.4.3.3 数据通路

上一小结阐述了数据通路的运算顺序, 本节将说明门内运算与门间运算的实现方式。

门内运算指的本该是 LSTM 网络中一个门的运算, 以遗忘门为例, 其公式如下式 (4-5):

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (4-5)$$

计算由两个矩阵乘法与两个加法构成, 但在实际的实现中, 矩阵乘法需要多次循环计算, 而加法只需要计算一次, 因此将加法也纳入门间运算的范畴, 门内运算则仅包括两个矩阵乘法, 各由一个 MAC 单元负责。MAC 单元是实现矩阵乘法

的最小单元，其包含一个乘法器与一个加法器，如图 4-7 中所示，本文还在乘法器与加法器之后各插入了一个寄存器，目的是为了加一级流水线，与上一节提到的 FSM2 流水线设计类似，可以将时间缩短近一倍。因为本文中 W_h 是 64×64 ， W_x 是 64×8 ，因此 $W_x \cdot x$ 的计算过程和 $W_h \cdot h$ 的一部分计算差不多，下面以 $W_h \cdot h$ 为例，先给出两个公式：

$$m_w \cdot m_x = \begin{bmatrix} wx_{11} & wx_{12} & \dots & wx_{1\ 64} \\ wx_{21} & wx_{12} & \dots & wx_{1\ 64} \\ \dots & \dots & \ddots & \dots \\ wx_{64\ 1} & wx_{64\ 2} & \dots & wx_{64\ 64} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_{64} \end{bmatrix} \quad (4-6)$$

$$= \begin{bmatrix} v_{w1} \cdot v_x \\ v_{w2} \cdot v_x \\ \dots \\ v_{w64} \cdot v_x \end{bmatrix}$$

$$v_w \cdot v_x = [w_1 \ w_2 \ \dots \ w_{64}] \cdot [x_1 \ x_2 \ \dots \ x_{64}] \quad (4-7)$$

$$= w_1x_1 + w_2x_2 + \dots + w_{64}x_{64}$$

根据矩阵运算的公式（4-6），可以拆解为 64 次向量乘法。根据向量乘法公式（4-7），其可以拆解为 64 个乘积之和，因此 MAC 单元通过循环对乘积结果累加，从而实现向量乘法。而且不难发现，矩阵乘法中的 64 个向量乘法相互独立，可以并行计算提高速度，但那意味着 MAC 单元的数量也要增加到 64 倍，在 Deepak Kadetotad 等提出的设计中^[66]，就采用了矩阵乘法中的所有向量乘法并行计算、4 个门单元串行计算的方式，如果采用这种结构，完成这样一个矩阵乘法，共计使用 72 个 MAC 单元，耗时 256 个周期。本文则采用了更加节省资源的结构，4 个门单元并行计算，64 个向量乘法串行计算，因此总共有 8 个 MAC 单元，但耗时 4096 个周期，相比之下资源开销为 11%，耗费的时间 1600%，速度为 6.25%。之所以资源使用不是达到了 11% 是因为两个矩阵乘法的维度不同， $W_x \cdot x$ 仅有 8 个向量乘法，因此在并行设计的时候， $W_x \cdot x$ 只需要使用 8 个 MAC 单元。同时这也意味着采用串行计算产生了时间上的浪费，也就是在经过 8 次 Cell 的运算后，负责 $W_x \cdot x$ 的 MAC 单元会进入空档期。

门间运算则包括了向量的对位加法（对应位置相加）、对位乘法（对应位置相乘）以及激活函数。前面已经说明门内运算每一次是完成了一个向量乘法，门间运算则紧接在门内运算之后进行，否则数据会被下一次运算的结果覆盖，正是由于矩阵乘法以及对位运算在第一个维度上的数据都是独立的，才有了这种设计

的思路，即在 `hidden_size` 这一维度上逐条运算，而非完成全部的矩阵乘法才能进行门间运算，从而节省了缓存矩阵乘法结果的空间，而这一空间所需的深度为 128。

4.4.3.4 存储

从图 4-1 中可以看到，存储包括了 Bank 和 Buffer 两种类型，Bank 存放的是权重、偏置等常量，使用 ROM 进行实现。Buffer 则用来缓存计算过程中的中间变量，一个是 c ，一个是 h ，使用 RAM 进行实现。

值得注意的是，C-Buffer 只用了 2 块 RAM，而 H-Buffer 用了 4 块，这是因为 h 向量参与每一次向量乘法，因此第 1 个门计算单元的输出不能直接覆盖 H-Buffer 中的值，否则会导致下一次向量乘法的输入错误，因此采用乒乓结构的双 Buffer，一个读一个写，Cell 单元从 Buffer0 中读入数据进行运算，门计算单元的结果写入 Buffer1，等到下一个时间步，Cell 从 Buffer1 中读入数据，结果则写入 Buffer0，如此依次交替读写。而 c 向量在每次门单元的计算中只有对应位置的元素参与计算，因此在该元素参与计算后便可以用新的结果直接覆盖。

使用乒乓结构的好处是，对比用缓存存下 h 的结果，等计算结束再写入 H-Buffer，可以省去这一写入过程所耗费的时间，约为 64 个时钟周期。这里额外说明，需要对 h 进行缓存也是逐条运算所带来的副作用之一，但相比将矩阵乘法的结果缓存下来再进行门间运算（无需缓存 h ，可直接覆盖），这一方案只需增加深度为 64 的存储空间。

4.4.4 开路电压模块

开路电压模块（OCV 模块）的实现较为简单，其本质上是一个查找表，查找表的输入是开路电压 OCV，输出则是 SOC。在前文中已经拟合了 SOC-OCV 曲线，但这里的查找表是该函数的反函数，所以把拟合时的 x 与 y 对调再以同样的方式拟合得到 OCV-SOC 函数。查找表的输入范围为 $U_{\min} \sim U_{\max}$ ，在本文中为 3298~4183，在这个范围内的每个电压 U （均为整数）对应一个 SOC，用 case 语句即可实现。

总体来说，当模式选择模块判定进入休眠模式时，`valid1` 信号触发本模块，根据当前电压确定对应的 SOC 值。同时将这一值作为校准数据写入特征构造模块的 `soc-ah` 寄存器中，校准机制在上一章也已经说明过。

4.5 本章小结

综上所述,本章从面积优先级大于速度的角度出发提出了整个电路的设计方案。资源开销大幅缩减所的同时也带来了速度上的落后,尽管基于本文提出算法的硬件设计对速度要求极低,但考虑到模块作为独立的神经网络电路的性能上限,本设计也采用了多种方法来弥补速度上的不足,包括对 4 个门内运算采取并行、对顶层采用两级流水、MAC 单元多插入一级寄存器,提高了能实现的最大频率,以及使用乒乓 Buffer 结构取消数据重写时间等。

第五章 FPGA 验证与后端设计

5.1 本章引言

上一章所提出的电路设计使用 Verilog HDL 语言实现，本章对该设计进行逻辑仿真、搭建 FPGA 原型验证平台对设计进行部署测试、对设计进行后端设计。

其中 FPGA 原型验证是数字芯片主流的验证方法，相比于仿真软件，FPGA 更接近实际情况，能够达到充分验证芯片的目的，确保功能的可靠实现，且相比于流片周期短，验证快。

5.2 逻辑仿真

设计完成后，对其进行逻辑仿真验证功能的正确性是必不可少的。原理上即设计作为待测试模型，对其施加输入，然后将输出与原抽象模型的输出比对，在 ASIC 设计中，一般采用编写 tb (testbench) 来施加激励，然后查看输出波形是否符合预期的方法。需要注意的是，仿真与专门的验证不同，仿真主要是针对设计人员，目的是检查代码是否有错误，代码的逻辑功能是否与预期一致；而验证是验证人员的工作，其行为更加复杂，目的是验证设计是否满足需求。

本文采用 VCS (Verilog Compiled Simulator) 进行仿真，VCS 是 Synopsys 公司的仿真工具，其具有出色的内存管理，能够支持大规模 ASIC 设计，性能十分出色，是数字 IC 设计的主流仿真工具。本文的设计采用自顶向上的设计方法，每一个子模块在设计完成后都会编写相应的 tb，子模块并没有现成的激励与响应，因为子模块在 Python 中并没有对应的模型。但可以在 Python 中编写子模块模型或者仅查看波形是否与设计前所画的时序图一致。

由 VCS 编译 tb 后产生后缀为 fsdb 的波形文件，该文件使用 Verdi 查看，Verdi 是 Synopsys 公司的波形查看工具，无法自己产生波形，往往和 VCS 搭配使用。所有的子模块编写完成后，连接到顶层，这时编写 tb，激励可以使用与 Python 所编写的算法模型相同的激励以验证 Python 模型是否正确映射到了 RTL 设计中。顶层的 tb 采用 SV (SystemVerilog) 编写，相比 Verilog，SV 的优势一在于使用了接口，使得修改设计后的 testbench 修改更为简便，其次在于使用时钟块可以方便对 tb 中的信号进行同步。

```

summer@yian:~/Desktop/my_prj/sim
File Edit View Search Terminal Help
calculate done !
x_in: 3996
x_in: 317
x_in: 3816
x_in: 3538
x_in: 588
x_in: 411
x_in: 154
x_in: 169
x_in: 4000
x_in: 316
x_in: 3816
x_in: 3538
x_in: 588
x_in: 411
x_in: 154
x_in: 169
result: 3919
finish at simulation time 92861910.0ns
VCS Simulation Report
Time: 92861910000 ps
CPU Time: 27.570 seconds; Data structure size: 0.5Mb
Fri Jan 19 23:17:23 2024
CPU time: .600 seconds to compile + .249 seconds to elab + .200 seconds to link
+ 27.596 seconds in simulation
  
```

(a) RTL仿真结果输出

```

In [26]: x_ts * scale
Out[26]: tensor([[[[3996.3677, 317.0304, 3816.0208, 3538.7053, 588.6820, 411.5098,
                    154.0025, 169.3419],
                    [4000.7957, 316.2112, 3816.0217, 3538.7053, 588.6820, 411.5098,
                    154.0025, 169.3419],
                    [4000.7957, 316.2112, 3816.0217, 3538.7053, 588.6820, 411.5098,
                    154.0025, 169.3419]]]])

In [31]: compare('result')
Out[31]: 浮点模型: [0.92576253], 定点模型: [0.9570125], 误差: 3.38%

In [32]: 0.957 * scale
Out[32]: 3919.872
  
```

(b) Python模型输出

图 5-1 RTL 仿真输出与 Python 模型输出对比

图 5-1 (a) 为 VCS 仿真过程打印在 Terminal 上的信息，包括 2 个时间步的特征向量、最终的计算结果。图 5-1 (b) 为 Python 模型的结果，Python 中有浮点和定点两个模型，通过与图 5-1 (a) 的结果对比，得出结论为 RTL 设计对定点模型的还原度为 99.9%，对浮点模型的还原度为 96.62%。

此外，仿真的运算时间总共 142976 个时钟周期 (2.86ms)，如图 5-2 所示。因此最低频率为 150KHz (最大允许计算时长为 1s) 即可完成设计需求的功能。

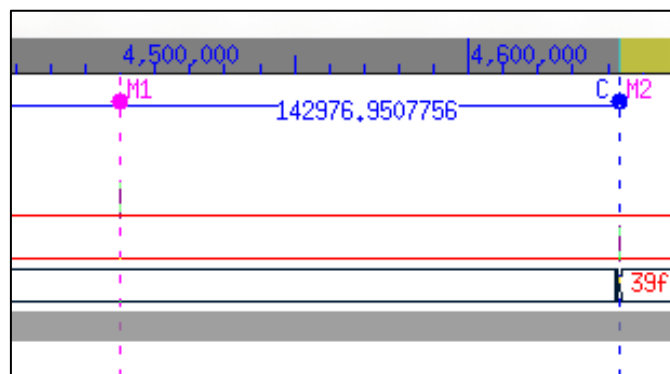


图 5-2 仿真中计算所用时间

5.3 原型验证

5.3.1 测试平台搭建

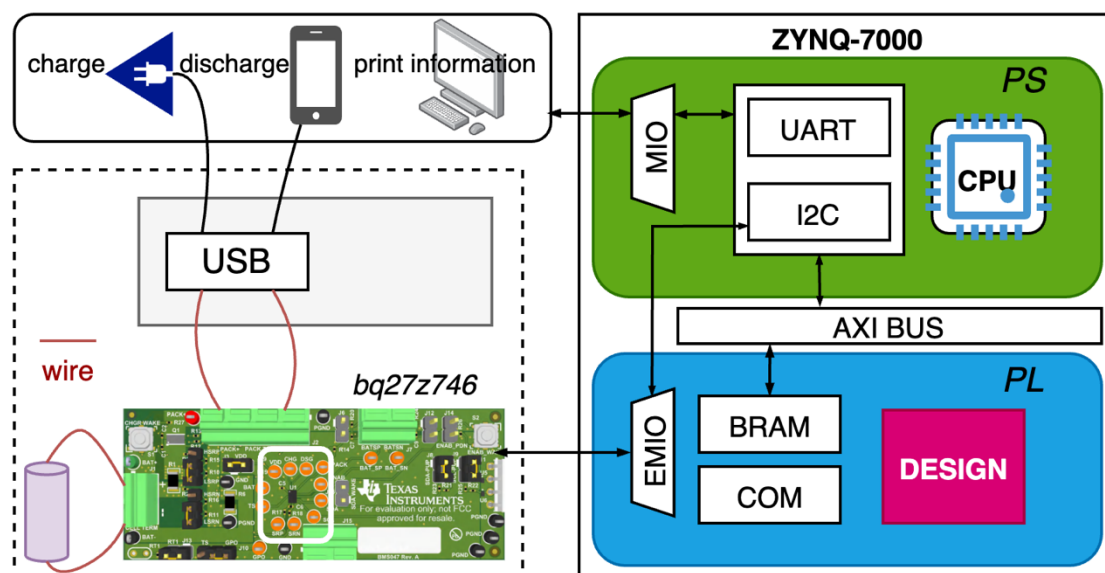


图 5-3 测试平台连接图

测试平台连接图如 5-3 所示，由以下四个部分组成：

1. ZYNQ，见图 5-3 右边方框部分。本文所使用的 ZYNQ-7000 是 Xilinx 推出的 SoC-FPGA 器件。Xilinx（赛灵思）公司是 AMD（美国超威半导体）公司旗下的可编程器件厂商。相较于传统 FPGA，ZYNQ 系列具有一颗 ARM 架构的 SoC 芯片，该芯片提供了许多外设，可以通过 C 语言编程配置这些外设，这一部分简称 PS，FPGA 部分则称作 PL，两部分软硬结合，可以大大提高开发效率。

2. 前端采集芯片，见图 5-3 左边虚线框内的绿色框。本文采用的前端数据采集芯片是 TI 的 BQ27Z746evm 套件，BQ27z746evm 由 BQ27z746 芯片及其外围电路组成。该芯片不仅具有电流电压采集 ADC，同时其也具有电池保护和电

量估计功能，可以有效保护电池，同时确保测试平台的安全性。该芯片支持与外部设备通讯，协议为 IIC。

3. 充放电模块，见图 5-3 左侧虚线框内黄色部分。充放电主板是梦莹科技的 22.5W 快充主板，可以对电池侧的电压进行转换，通过标准的 USB-C 接口或 USB-A 进行充电和放电，方便连接各种电子设备与电源适配器。其同样有防止过电流等保护功能。

4. 电芯，见图 5-3 左侧紫色柱体。本位采取 4 节三星 INR18650-30Q 并联，该电池额定容量为 3000mAh，材料为三元锂电池，其额定电压 3.7V，满电电压 4.2V，持续输出电压 20A，最大输出电流达 40A，内阻 12m 欧左右。

本文为了方便测试，设计了一个外壳，将电芯、充放电模块、采集芯片整合为一个电池数据采集系统，如图 5-4 所示。电池数据采集系统提供 I2C 数据通信接口与 USB 充放电接口，除了导出数据用于实验分析外，还可以作为充电宝使用。电池数据采集系统的外壳设计为单层方形外壳，三个组件及外壳的建模如图 5-4（a）所示，对外壳侧面接口位置进行了开槽处理，3D 渲染图如 5-4（b）所示。电池以及主板采用可调节角码进行固定，实物如图 5-4（c）所示。

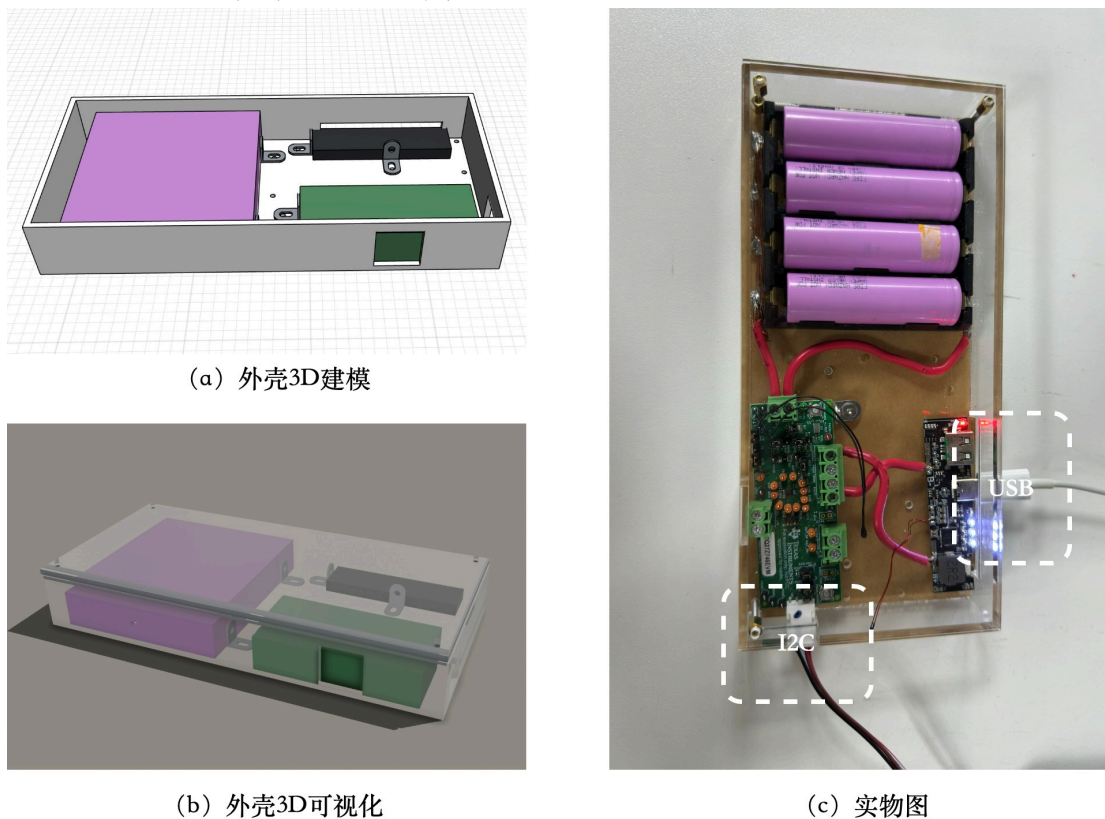


图 5-4 电池数据采集系统

将电池数据采集系统连接到 ZYNQ 组成整个测试平台，实物如下图 5-5 所示，PC 连接到 ZYNQ，主要是用来查看串口打印信息、对 PS 编程、为 ZYNQ 烧写比特流文件等。

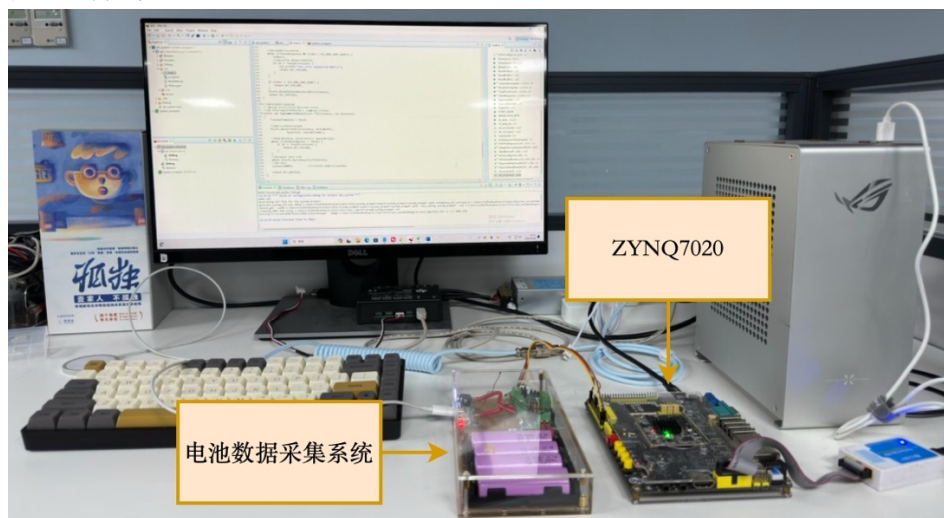


图 5-5 实物测试平台

5.3.2 FPGA 综合

5.3.2.1 设计移植

为了在 FPGA 中实现该算法功能，需要将 Verilog 代码移植到 Vivado 中，Vivado 是 Xilinx 为自己 FPGA 产品推出的一款集成电路设计工具套件，它是一个全面的硬件设计工具，用于设计、开发和部署 FPGA。Vivado 提供了一系列的功能和工具，用于实现数字电路设计、综合、IP 集成、仿真、布局与布线、时序分析、验证和生成比特流文件等任务。

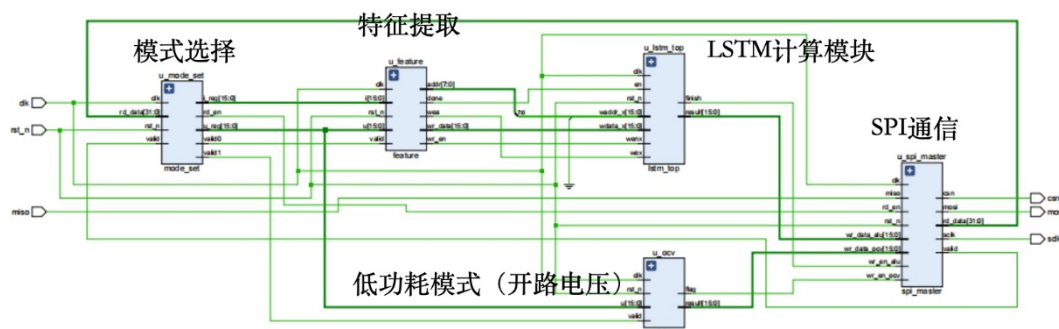


图 5-6 Vivado 分析的 Schematic

图 5-6 是设计的框架图。与上一章提出的设计不同的地方在于，移植的 Verilog 主要对 RAM 部分进行了修改，即用 Vivado 的 Blockram IP(简称 BRAM)

替换原先的 RAM 代码，一是使用 IP 对电路进行优化，二是权重与偏置等参数需要用该 IP 的初始化功能在上电时写入 ROM 当中，在 FPGA 中，RAM 和 ROM 均使用该 IP。该 IP 的初始化功能支持使用后缀为 coe 的文件导入数据，数据由原先的二进制文本文件按照 Vivado 所要求的格式进行修改得到。其次是加入了一个简单的 LED 模块，用板上的两颗灯 LED0 和 LED1 指示电路中的数据刷新情况，LED0 连接读数据信号 rd_en，每读取一次前端信息闪烁一次；LED1 则连接完成信号 finish，每次运算完成闪烁一次。

将 Vivado 中的顶层设计称作 Top Design，对 Top Design 同样跑一遍与之前相同的 testbench，得到相同的结果证明逻辑功能没有问题，Vivado 的仿真结果如图 5-7（a）所示，VCS 的仿真结果如图 5-7（b）所示。

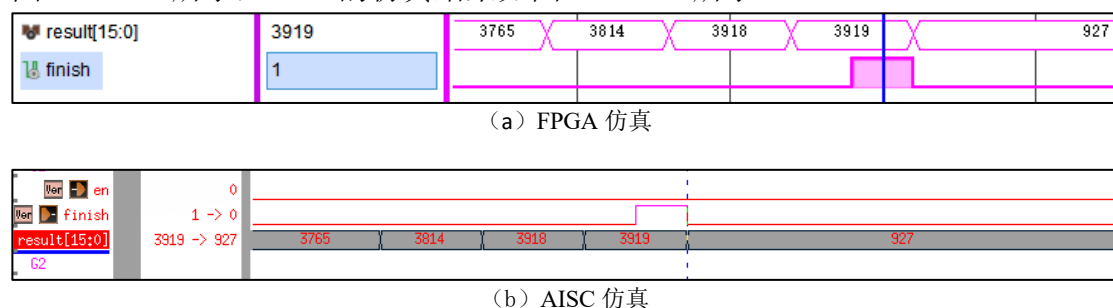


图 5-7 移植到 Vivado 中的仿真

5.3.2.2 前端数据传输

在前面的图 5-3 中可以看到，前端采集芯片 BQ27z746 的数据由 I2C 接口与 ZYNQ 的 PS 部分通信，I2C 是由飞利浦公司发明的多主从串行通讯总线，仅有两条线 SCL（时钟）、SDA（数据）就可以完成通讯，实际的 I2C 共有 4pin，另外 2 线为 VDD（高电平）和 VSS（地）。

BQ27z746 通过 I2C 接口连接到 ZYNQ PL 端的扩展 IO，该 IO 经 EMIO 连接到 PS 端的 I2C 外设，由该外设驱动，之所以绕一圈使用 PS 部分的 I2C 外设，而不使用 PL 硬件实现 I2C 驱动模块的原因是 Clock Stretching(时钟延展)机制。简单的 I2C 设备如 e2prom，发送字地址事实上就是存储器地址，该设备可以立即应答，但对于 BQ27z746 这样一款复杂的 SoC（片上系统）芯片，发送字地址其实是发送指令，需要等待该芯片中的 CPU 对该指令执行完成才返回数据，而在数据准备期间，该芯片会拉低时钟线，这种功能便称作 Clock Stretching。大部分情况下，该芯片时钟延展只需要等待几 ms，但在某些写入的情况下，需要较

长时间，最长达 40ms。BQ27z746 通讯时通过示波器抓取的波形如图 5-8 所示，可以看到期间共有三次时钟延展，第一次是接收到字地址后，芯片解析指令拉低了 SCL，另外两次则是在发送数据之前。

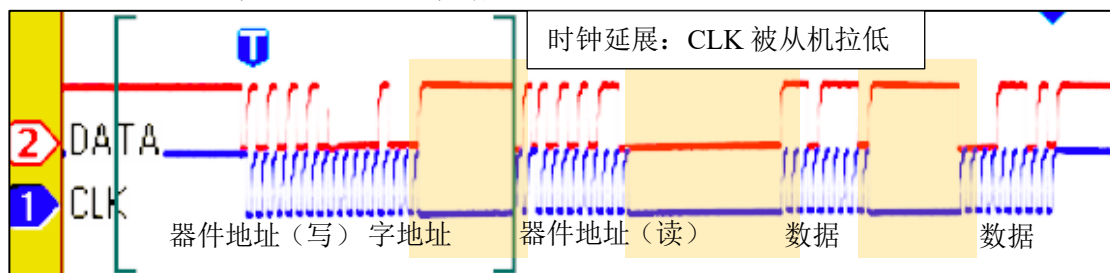


图 5-8 时钟延展机制

针对 eeprom 设计的 I2C 驱动模块，仅有 SDA 为 inout 类型，SCL 完全由主机控制产生，从机无法拉低 SCL，因此不具有 Clock Stretching 功能，无法与 BQ27z746 实现通信，而编写具有 Clock Stretching 功能的 I2C 驱动模块较为复杂，因此使用 PS 部分的 I2C 外设，通过编程实现 I2C 外设定读取数据的功能。

5.3.2.3 板内数据传输

通过 PS 端 I2C 外设所读取的数据直接使用 AXI-BRAM IP，按照规定的时间内写入 PL 的双口 RAM。Top Design 则从该 RAM 中读出数据，因为接口不同，无法直接连接，需要通过一个 SPI 从机模块进行连接。读数据时，该模块从 RAM 中将数据读出，再串行发送给电 Top Design，写数据则相反。

表 5-1 BRAM_PORT 信号列表

信号名称	位宽	方向	功能
RAM_CLK	1	OUT	ram 读写时钟
RAM_RST	1	OUT	ram 复位
EN	1	OUT	时钟使能
WE	4	OUT	写使能
ADDR	32	OUT	地址
DIN	32	IN	写入数据
DOUT	32	OUT	读出数据

该 SPI 从机模块与 RAM 相连的一端是 BRAM_PORT 接口，与 Top Design 相连的一段则是 SPI 接口。其中 BRAM_PORT 是 Xilinx 提供的 Block ram IP 的标准接口，包括了七个信号，在表 5-1 中列出。

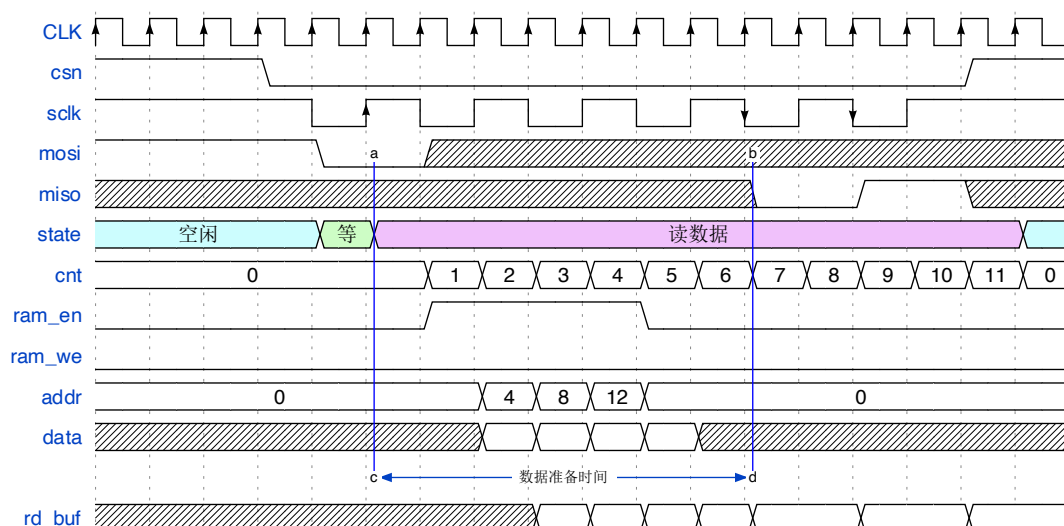


图 5-9 SPI 从机模块读时序

上一章详细介绍过 SPI 主机的时序，下面介绍 SPI 从机的时序。读取的时序如图 5-9 所示，从机模块时钟采用与主机同一时钟，在从机检测到 `csn` 拉低后进入等待状态，采样到 `mosi` 为 0 则进入主机请求读取数据状态，该状态下计数器开始计数，同时拉高时钟使能信号、更新地址，向 RAM 请求数据。如图所示，在本文中主机每次请求 4 个地址的数据，分别是 0、4、8、12，从机将这四个地址读取的数据暂存。从第一个 `sclk` 上升沿读取到 `mosi` 的请求到数据准备完毕的第一个下降沿的时间为数据准备时间，共计 7 个 CLK 周期，准备时间结束后将暂存的数据向主机逐位发送，发送多少位则取决于 `csn` 信号，图 5-9 共计发送 2bit 数据。

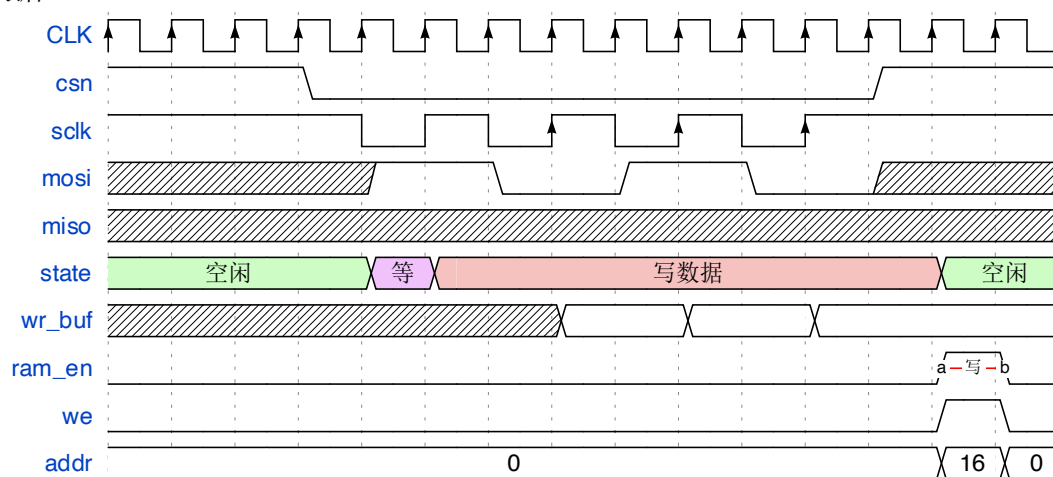


图 5-10 SPI 从机模块写时序

SPI 从机的写时序如图 5-10 所示，等待状态采样到 `mosi` 为 1，则进入主机请求写数据状态，在该状态下每个 `sclk` 的上升沿读取 `mosi` 并进行缓存，`csn` 拉

高后退回空闲状态，同时产生一个使能脉冲将缓存的数据写入 RAM，写入时间仅 1 个 CLK 周期。

双口 RAM 中的所有数据可以由 PS 读取并通过其串口外设输出，在 PC 上查看串口打印的信息，且 PS 的串口外设可由 C 语言编程打印信息，能轻松实现复杂的文字打印。

5.3.3 验证结果

图 5-11 是整个 ZYNQ 的设计框架图，其中绿色框为 PS 实现的部分，虽然看上去模块较多，但均为 Vivado 所提供的 IP，蓝色框为 PL 实现的部分，包括芯片设计 Top Design 与保存前端数据的双端口 RAM。

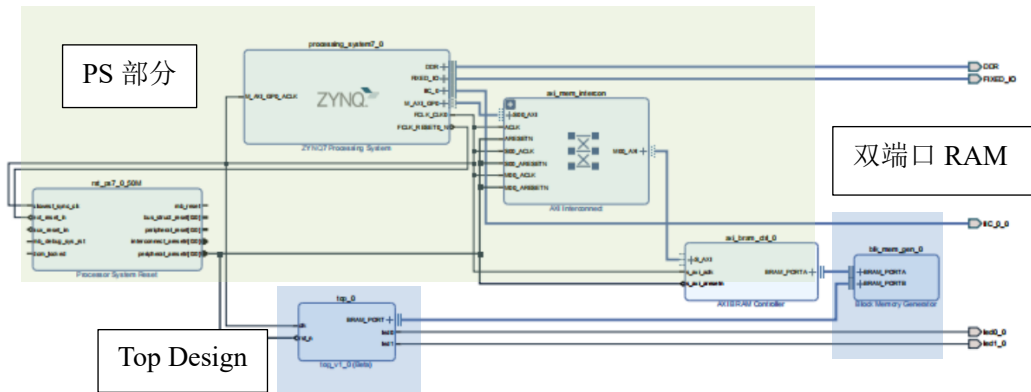


图 5-11 BLOCK DESIGN 框架图

图 5-12 为 FPGA 后端后的总结报告，报告主要给出了整个电路设计完成布局布线后的资源使用情况以及功耗分析。资源使用报告表明，对于 LUT、FF、DSP 以及 BRAM 的利用率均为 1%，IO 的利用率为 18%，BUFG 利用率为 3%。功耗分析报告表明，总功耗为 7.6W，其中动态功耗为 7.27W，占比 96%，静态功耗为 0.32W，占比 4%。在动态功耗中，Signals 功耗为 2.1W，占比 29%，Logic 功耗为 2W，占比 28%，DSP 为 1.93W，占比 27%，BRAM 和 IO 功耗占比则较低。

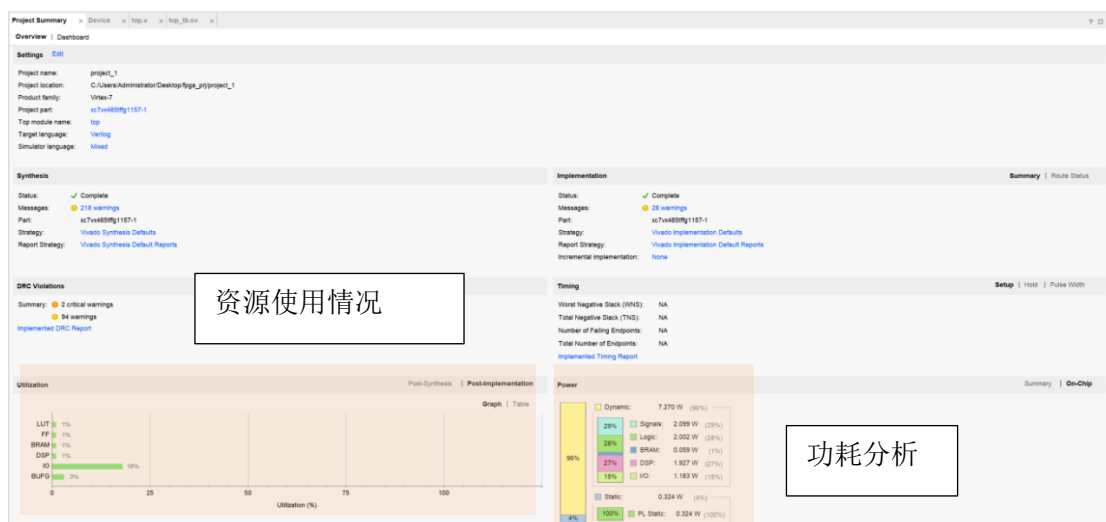


图 5-12 FPGA Implementation 报告

图 5-13 是上位机串口所接收的打印信息，其中报告了当前 I2C 模块所读取的电压电流数据以及 RAM 中当前存储的数据，RAM 中地址 6 存放的即芯片所计算的 SOC。可以看到从前端读取数据到计算 SOC 的整个功能正确实现。窗口显示计算的 SOC 为 2976，这是量化后的数据，除以量化尺度 2 的 12 次方即 4096 得到结果为 72.6%。由于该电芯与训练所使用的数据测试的电芯不同，因此结果有一定误差，但足以说明 RTL 设计的功能被正确地实现。

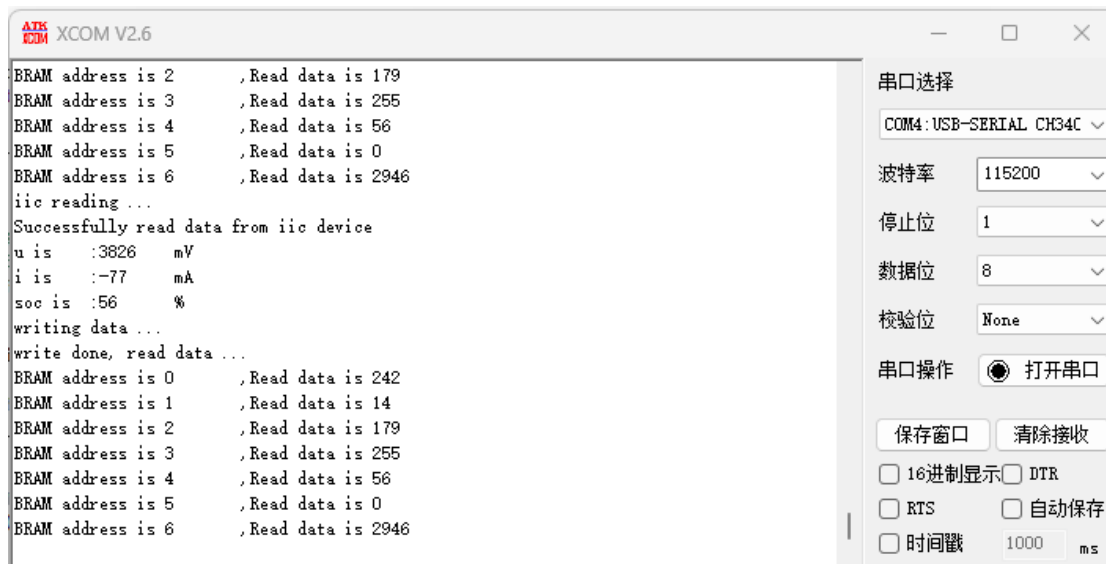


图 5-13 串口打印信息

5.4 后端设计

通过 FPGA 验证了设计的可行性后对 RTL 进行后端设计。本节后端设计基于 SAED28/32nm 工艺库。

前面提到设计在 FPGA 中实现时对代码做了相应的修改,在后端设计的过程中同样发现有许多可以优化的地方。针对 ASIC 实施,本节对存储结构进行优化,在第 4 章提出的设计中使用了大量的存储,如中间变量缓存使用的双口 RAM、固化的权重偏置使用的 ROM 等,其中使用的 ROM 容量为 103KB, RAM 为 0.9KB,这种存储结构主要存在以下问题。

1. 不同于 FPGA,ASIC 一旦生产出来,ROM 中的数据便无法更改,这意味着无法对该神经网络进行更新,极大限制了芯片的应用场景。

2. 工艺库的文件中只有 SRAM 宏单元可供使用,并没有 ROM,对于这么大容量的 ROM,没有宏单元,直接写代码综合时间过于长。

3. 容量 0.9KB 的 SRAM 单元的面积与运算部分的逻辑单元面积相当,虽然无法得知 ROM 的存储密度比 SRAM 高多少,但可以大致估计到,存储的面积占比会达到 80%以上,这样的情况下,对运算逻辑进行复用减小面积的收益似乎不再可观。

基于以上原因,将权重偏置的存储从片上 ROM 改为片外存储,至于片外存储器,可以选择 Flash,DDR 等,数据的读取方式则改为总线通信,这样芯片可以作为设备挂在 SoC(片上系统)的总线上。在代码上,将原先的 ROM 模块移除,改为总线读取缓存模块,总线协议为自定义,数据通路为 64 位,地址通路为 16 位,运算电路的工作频率为总线读取频率的六分频,这是为了能够在每次计算前有足够的时间将参与计算的数据读取并缓存。

将上述修改后的代码综合,使用 Synopsys 公司的 DC(Design Compiler)工具,综合后的面积报告如图 5-14 所示,从面积报告中可以看到总面积 0.12mm²,其中 SRAM 的面积占了总面积的 42.9%。

Number of ports:	2282
Number of nets:	22104
Number of cells:	16790
Number of combinational cells:	15308
Number of sequential cells:	1440
Number of macros/black boxes:	7
Number of buf/inv:	2565
Number of references:	34
Combinational area:	41648.864209
Buf/Inv area:	3369.949465
Noncombinational area:	10118.743538
Macro/Black Box area:	52820.120117
Net Interconnect area:	18458.213506
Total cell area:	104587.727865
Total area:	123045.941371

图 5-14 DC 综合面积报告

DC 的功耗报告如图 5-15 所示，显示总功耗为 5.4mW。DC 综合功耗仅供参考，本文提出的设计中还使用了两种模式，要进一步分析实际功耗的话需要完成布局布线后用专业软件 PTPX 进行分析。

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	220.0797	0.5233	0.0000	220.6030	(4.03%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	8.6558	5.5867e-03	2.3845e+07	32.5065	(0.59%)	
register	122.8818	9.3654e-02	1.3522e+09	1.4752e+03	(26.95%)	
sequential	-3.2165e-05	1.5109e-05	6.4986e+06	6.4986	(0.12%)	
combinational	229.0650	87.3657	3.4218e+09	3.7382e+03	(68.30%)	
Total	580.6823 uW	87.9883 uW	4.8043e+09 pW	5.4730e+03 uW		

图 5-15 DC 综功耗报告

得到综合后的门级网表，使用 ICC 读入进行布局规划，主要包括 Macro（宏单元）的摆放、PG（电源地）的规划、Core Ring 和 Macro Ring 的创建等。图 5-16 展示了电路完成 Floorplan 后的布局图，以及一些重要指标的总结。

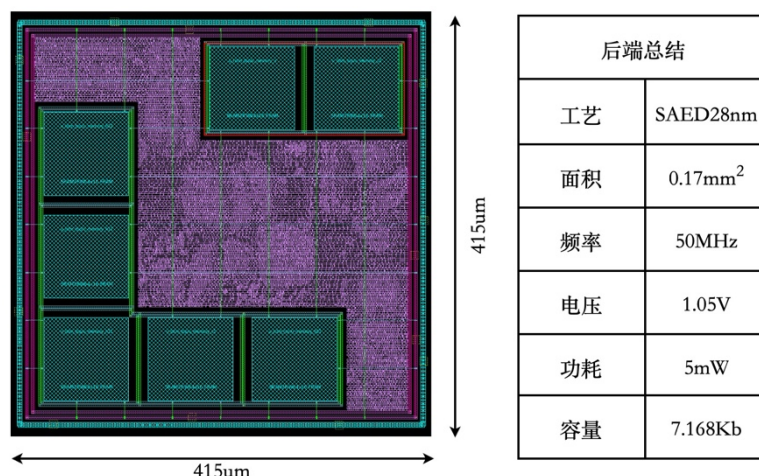


图 5-16 电路设计性能总结与布局图

5.5 本章小结

本章主要阐述了逻辑仿真、FPGA 验证和一部分后端的流程。逻辑仿真验证了 RTL 代码的正确性。在 FPGA 验证过程中，搭建了一个实物验证平台，前端数据采集系统由购买的芯片组成，采集的数据流为：前端采集芯片的 I2C 接口传到 PS 部分的 I2C 外设，PS 通过 AXI 总线传到 PL 端的 RAM，RAM 通过编写的 SPI 从机模块传输到设计 Top Design。后端部分使用 DC 进行综合，得到了面积与参考功耗，使用 ICC 进行布局规划，得到了初步布局图。

第六章 总结与展望

6.1 总结

电力作为我们生活中最不可或缺的能源，以其便利性和环境友好性领先其他能源。锂电池的存在也给我们的生活带来了极大的便利，但它本身固有的缺陷，如充能速度慢、能量释放具有不稳定性等也让人们容易产生“续航焦虑”，因此准确地告知用户还有多少电量非常关键。本文以锂电池 SOC 为研究对象，在以下几个方面开展了一系列研究：准确的锂离子电池模型研究、具有高准确度高鲁棒性的 SOC 估计算法研究、具有小面积低功耗的 SOC 估计芯片研究，以下是本文的主要工作：

1. 第二章从电池的工作原理出发，研究了最能反映电池容量的度量单位，再以此为基础开展研究。提出了能够准确反映锂离子电池动态特性的电池模型，分析对比了不同模型在不同工况下的响应情况，最后选用了二阶电路模型的参数作为后续算法的辅助信息。

2. 第三章研究了准确估计 SOC 的算法。首先提出一种结合了安时积分、开路电压与神经网络的双模式算法模型，接着详细阐述了由特征提取引擎与 LSTM 网络组成的混合算法模型，并将该模型与其他算法模型的估计结果进行对比分析。该混合算法经过训练后，在噪声参数 σ 大小分 0.01、0.001、0.0001 的三种情况下的平均 RMSE 分别为 1.571%、0.912%、1.174%。

3. 第四章研究了实现该算法的数字电路设计。在提出的设计中，量化尺度为 12 位，特征构造模块使用 1 个加乘单元实现数据的归一化，开路电压模块使用查找表实现 SOC 的估计，在 LSTM 网络的硬件设计中，使用 2 个有限状态机 FSM 实现电路的控制逻辑，使用 16 个 MAC 单元实现双层 LSTM Cell，使用 6 个宽度为 16，深度为 64 的双端口 RAM 实现中间变量的存储，使用总容量为 103KB 的 ROM 实现权重与偏置的存储。

4. 第五章首先对编写 RTL 代码进行仿真验证，RTL 模型对定点模型的还原度为 99.9%，对浮点模型的还原度为 96.62%。仿真通过后将该芯片设计部署到了 ZYNQ7020 开发板上，与电池、ADC 芯片、充放电主板连接搭建了测试平台，基于该测试平台的整体设计实现后的功耗为 7.6W，单次计算耗时 2.86ms。验证

完成后，基于 SAED28/32nm 工艺进行后端设计，主体部分时钟约束为 50MHz，在 DC 综合后，该芯片的面积为 0.12mm²，功耗为 5.4mW，在 ICC 布局规划后，面积为 0.17mm²。

6.2 展望

本文以准确估计锂电池 SOC 为目标设计了一款数字芯片，取得一定成果，但限于作者目前的认知水平，研究中仍有许多方面需要改进，以下为后续待进一步完善或优化的工作。

1. 流片测试。目前仅在 FPGA 上进了验证，以及做了初步的后端设计，未来会进一步完善优化前端与后端工作并投片。

2. 数据集的拓展。受限于采集设备的能力，无法对更多条件下的数据进行采集，使得电池建模与神经网络训练的数据覆盖的范围还不够全面，日后若有设备条件则可以将数据集的范围拓展，提升模型的性能。

3. 神经网络的优化。本文选取了普遍认为效果较好的 LSTM 网络作为预测模型，但其规模也较大，相应的实现的代价较大，未来的工作中可以寻找更好的网络模型。其次本文直接对网络进行了硬件实现，未对网络模型进行权重上的优化，未来将对网络进一步优化，在准确度差不多的情况下实现网络规模的缩减。

4. 电路设计的优化。第一点，本文基于高度定制化的思路设计了整个算法模型的硬件电路，其中的 LSTM 网络部分应用上就有些限制，如只支持双层 LSTM 以及输入特征维度不能大于隐藏层数等，后续的工作中会对设计进行优化，让设计向 IP 靠拢，支持各种不同类型的输入。第二点，整个设计仍旧是基于纯逻辑设计，而非 SoC 架构，未来将继续学习，优化设计架构。

参 考 文 献

- [1] Sanguesa J A, Torres-Sanz V, Garrido P, et al. A Review on Electric Vehicles: Technologies and Challenges[J]. *Smart Cities*, 2021, 4(1): 372–404.
- [2] 网易. 2023 年中国新能源汽车销量突破 900 万辆! [EB/OL]. 2024-02-15/2024-03-18. <https://www.163.com/dy/article/IR0IU5QO0552CENS.html>.
- [3] 乘用车市场信息联席会[EB/OL]. /2024-03-25. <http://data.cpcauto.com/>.
- [4] 汽车之家[EB/OL]. /2024-03-25. <https://www.autohome.com.cn/beijing/>.
- [5] Lithium-Ion Battery Pack Prices Hit Record Low of \$139/kWh[J]. *BloombergNEF*, 2023.
- [6] Li M, Lu J, Chen Z, et al. 30 Years of Lithium-Ion Batteries[J]. *Advanced Materials*, 2018, 30(33): 1800561.
- [7] Wang Y, Tian J, Sun Z, et al. A comprehensive review of battery modeling and state estimation approaches for advanced battery management systems[J]. *Renewable and Sustainable Energy Reviews*, 2020, 131: 110015.
- [8] Miao Z, Xu L, Disfani V R, et al. An SOC-Based Battery Management System for Microgrids[J]. *IEEE Transactions on Smart Grid*, 2014, 5(2): 966–973.
- [9] Brandl M, Gall H, Wenger M, et al. Batteries and battery management systems for electric vehicles[A]. 2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)[C]. 2012: 971–976.
- [10] Lawder M T, Suthar B, Northrop P W C, et al. Battery Energy Storage System (BESS) and Battery Management System (BMS) for Grid-Scale Applications[J]. *Proceedings of the IEEE*, 2014, 102(6): 1014–1030.
- [11] Szumanowski A, Chang Y. Battery Management System Based on Battery Nonlinear Dynamics Modeling[J]. *IEEE Transactions on Vehicular Technology*, 2008, 57(3): 1425–1432.
- [12] Rahimi-Eichi H, Ojha U, Baronti F, et al. Battery Management System: An Overview of Its Application in the Smart Grid and Electric Vehicles[J]. *IEEE Industrial Electronics Magazine*, 2013, 7(2): 4–16.
- [13] Cheng K W E, Divakar B P, Wu H, et al. Battery-Management System (BMS) and SOC Development for Electrical Vehicles[J]. *IEEE Transactions on Vehicular Technology*, 2011, 60(1): 76–88.
- [14] Xu D, Wang L, Yang J. Research on Li-ion Battery Management System[A]. 2010 International Conference on Electrical and Control Engineering[C]. 2010: 4106–4109.
- [15] Gabbar H, Othman A, Abdussami M. Review of Battery Management Systems (BMS) Development and Industrial Standards[J]. *Technologies*, 2021, 9(2): 28.
- [16] Xiong R, Li L, Tian J. Towards a smarter battery management system: A critical review on battery state of health monitoring methods[J]. *Journal of Power Sources*, 2018, 405: 18–29.
- [17] Baccouche I, Mlayah A, Jemmali S, et al. Implementation of a Coulomb counting algorithm for SOC estimation of Li-Ion battery for multimedia applications[A]. 2015 IEEE 12th International Multi-Conference on Systems, Signals & Devices (SSD15)[C]. 2015: 1–6.
- [18] Huang H, Meng J, Wang Y, et al. A comprehensively optimized lithium-ion battery state-of-health estimator based on Local Coulomb Counting Curve[J]. *Applied Energy*, 2022, 322: 119469.

-
- [19] He L, Guo D. An Improved Coulomb Counting Approach Based on Numerical Iteration for SOC Estimation With Real-Time Error Correction Ability[J]. IEEE Access, 2019, 7: 74274–74282.
- [20] Kumar K S R, Sastry V V, Sekhar O C, et al. Design and fabrication of coulomb counter for estimation of SOC of battery[A]. 2016 IEEE International Conference on Power Electronics, Drives and Energy Systems (PEDES)[C]. 2016: 1–6.
- [21] Ng K S, Moo C-S, Chen Y-P, et al. Enhanced coulomb counting method for estimating state-of-charge and state-of-health of lithium-ion batteries[J]. Applied Energy, 2009, 86(9): 1506–1511.
- [22] Lee J, Won J. Enhanced Coulomb Counting Method for SoC and SoH Estimation Based on Coulombic Efficiency[J]. IEEE Access, 2023, 11: 15449–15459.
- [23] Zhao L, Lin M, Chen Y. Least-squares based coulomb counting method and its application for state-of-charge (SOC) estimation in electric vehicles[J]. International Journal of Energy Research, 2016, 40(10): 1389–1399.
- [24] Mohammadi F. Lithium-ion battery State-of-Charge estimation based on an improved Coulomb-Counting algorithm and uncertainty evaluation[J]. Journal of Energy Storage, 2022, 48: 104061.
- [25] Movassagh K, Raihan S A, Balasingam B. Performance Analysis of Coulomb Counting Approach for State of Charge Estimation[A]. 2019 IEEE Electrical Power and Energy Conference (EPEC)[C]. Montreal, QC, Canada: IEEE, 2019: 1–6.
- [26] Saji D, Babu P S, Ilango K. SoC Estimation of Lithium Ion Battery Using Combined Coulomb Counting and Fuzzy Logic Method[A]. 2019 4th International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT)[C]. 2019: 948–952.
- [27] Zheng F, Xing Y, Jiang J, et al. Influence of different open circuit voltage tests on state of charge online estimation for lithium-ion batteries[J]. Applied Energy, 2016, 183: 513–525.
- [28] Pattipati B, Balasingam B, Avvari G V, et al. Open circuit voltage characterization of lithium-ion batteries[J]. Journal of Power Sources, 2014, 269: 317–333.
- [29] Xing Y, He W, Pecht M, et al. State of charge estimation of lithium-ion batteries using the open-circuit voltage at various ambient temperatures[J]. Applied Energy, 2014, 113: 106–115.
- [30] Hu C, Youn B D, Chung J. A multiscale framework with extended Kalman filter for lithium-ion battery SOC and capacity estimation[J]. Applied Energy, 2012, 92: 694–704.
- [31] Andre D, Nuhic A, Soczka-Guth T, et al. Comparative study of a structured neural network and an extended Kalman filter for state of health determination of lithium-ion batteries in hybrid electricvehicles[J]. Engineering Applications of Artificial Intelligence, 2013, 26(3): 951–961.
- [32] Baccouche I, Jemmali S, Manai B, et al. Improved OCV Model of a Li-Ion NMC Battery for Online SOC Estimation Using the Extended Kalman Filter[J]. Energies, 2017, 10(6): 764.
- [33] Dong G, Wei J, Chen Z. Kalman filter for onboard state of charge estimation and peak power capability analysis of lithium-ion batteries[J]. Journal of Power Sources, 2016, 328: 615–626.
- [34] Lee J, Nam O, Cho B H. Li-ion battery SOC estimation method based on the reduced order extended Kalman filtering[J]. Journal of Power Sources, 2007, 174(1): 9–15.
- [35] Wei J, Dong G, Chen Z. On-board adaptive model for state of charge estimation of lithium-ion batteries based on Kalman filter with proportional integral-based error adjustment[J]. Journal of Power Sources, 2017, 365: 308–319.
- [36] Peng X, Li Y, Yang W, et al. Real-Time State of Charge Estimation of the Extended Kalman Filter and Unscented Kalman Filter Algorithms Under Different Working Conditions[J]. Journal of Electrochemical Energy Conversion and Storage, 2021, 18(4): 041007.

- [37] Jiang Y, Baoyin H. Robust extended Kalman filter with input estimation for maneuver tracking[J]. Chinese Journal of Aeronautics, 2018, 31(9): 1910–1919.
- [38] Rangegowda P H, Patwardhan S C, Biegler L T, et al. Simultaneous State and Parameter Estimation using Robust Receding-horizon Nonlinear Kalman Filter[J]. 12th IFAC Symposium on Dynamics and Control of Process Systems, including Biosystems DYCOPS 2019, 2019, 52(1): 10–15.
- [39] Zhang Z, Jiang L, Zhang L, et al. State-of-charge estimation of lithium-ion battery pack by using an adaptive extended Kalman filter for electric vehicles[J]. Journal of Energy Storage, 2021, 37: 102457.
- [40] Li J, Ye M, Meng W, et al. A Novel State of Charge Approach of Lithium Ion Battery Using Least Squares Support Vector Machine[J]. IEEE Access, 2020, 8: 195398–195410.
- [41] Sheng H, Xiao J. Electric vehicle state of charge estimation: Nonlinear correlation and fuzzy support vector machine[J]. Journal of Power Sources, 2015, 281: 131–137.
- [42] Yang D, Wang Y, Pan R, et al. State-of-health estimation for the lithium-ion battery based on support vector regression[J]. Applied Energy, 2018, 227: 273–283.
- [43] Hansen T, Wang C-J. Support vector based battery state of charge estimator[J]. Journal of Power Sources, 2005, 141(2): 351–358.
- [44] Álvarez Antón J C, García Nieto P J, Blanco Viejo C, et al. Support Vector Machines Used to Estimate the Battery State of Charge[J]. IEEE Transactions on Power Electronics, 2013, 28(12): 5919–5926.
- [45] Cui Z, Wang L, Li Q, et al. A comprehensive review on the state of charge estimation for lithium-ion battery based on neural network[J]. International Journal of Energy Research, 2022, 46(5): 5423–5440.
- [46] Shen Y. Adaptive online state-of-charge determination based on neuro-controller and neural network[J]. Energy Conversion and Management, 2010, 51(5): 1093–1098.
- [47] Li C, Xiao F, Fan Y. An Approach to State of Charge Estimation of Lithium-Ion Batteries Based on Recurrent Neural Networks with Gated Recurrent Unit[J]. Energies, 2019, 12(9): 1592.
- [48] Liu F, Liu T, Fu Y. An Improved SoC Estimation Algorithm Based on Artificial Neural Network[A]. 2015 8th International Symposium on Computational Intelligence and Design (ISCID)[C]. Hangzhou, China: IEEE, 2015: 152–155.
- [49] Tong S. Battery state of charge estimation using a load-classifying neural network[J]. Journal of Energy Storage, 2016.
- [50] Chen J, Ouyang Q, Xu C, et al. Neural Network-Based State of Charge Observer Design for Lithium-Ion Batteries[J]. IEEE Transactions on Control Systems Technology, 2018, 26(1): 313–320.
- [51] Guo Y, Zhao Z, Huang L. SoC Estimation of Lithium Battery Based on Improved BP Neural Network[J]. Energy Procedia, 2017, 105: 4153–4158.
- [52] He W, Williard N, Chen C, et al. State of charge estimation for Li-ion batteries using neural network modeling and unscented Kalman filter-based error cancellation[J]. International Journal of Electrical Power & Energy Systems, 2014, 62: 783–791.
- [53] Hosseininasab S, Wan Z, Bender T, et al. State-of-Charge Estimation of Lithium-ion Battery Based on a Combined Method of Neural Network and Unscented Kalman filter[A]. 2020 IEEE Vehicle Power and Propulsion Conference (VPPC)[C]. Gijon, Spain: IEEE, 2020: 1–7.
- [54] Duan W, Song C, Peng S, et al. An Improved Gated Recurrent Unit Network Model for State-of-Charge Estimation of Lithium-Ion Battery[J]. Energies, 2020, 13(23): 6366.

-
- [55] Jeong J, Jeong S, Sylvester D, et al. A 42 nJ/Conversion On-Demand State-of-Charge Indicator for Miniature IoT Li-Ion Batteries[J]. 2019, 54(2).
- [56] Neri F, Cimaz L. 40 nm CMOS Ultra-Low Power Integrated Gas Gauge System for Mobile Phone Applications[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2013, 60(4): 836–845.
- [57] Impedance Track Based Fuel Gauging[J]. 2007.
- [58] Kim M, So J. VLSI design and FPGA implementation of state-of-charge and state-of-health estimation for electric vehicle battery management systems[J]. Journal of Energy Storage, 2023, 73: 108876.
- [59] Stighezza M, Bianchi V, De Munari I. FPGA Implementation of an Ant Colony Optimization Based SVM Algorithm for State of Charge Estimation in Li-Ion Batteries[J]. Energies, 2021, 14(21): 7064.
- [60] Kumar B. FPGA-based design of advanced BMS implementing SoC/SoH estimators[J]. Microelectronics Reliability, 2018.
- [61] Jemmali S, Manaï B, Hamouda M. Pure hardware design and implementation on FPGA of an EKF based accelerated SoC estimator for a lithium-ion battery in electric vehicles[J]. IET Power Electronics, 2022, 15(11): 1004–1015.
- [62] Dunn B, Kamath H, Tarascon J-M. Electrical Energy Storage for the Grid: A Battery of Choices[J]. Science, 2011, 334(6058): 928–935.
- [63] Liu D-H, Bai Z, Li M, et al. Developing high safety Li-metal anodes for future high-energy Li-metal batteries: strategies and perspectives[J]. Chemical Society Reviews, 2020, 49(15): 5407–5445.
- [64] Li J, Kong Z, Liu X, et al. Strategies to anode protection in lithium metal battery: A review[J]. InfoMat, 2021, 3(12): 1333–1363.
- [65] Battery Data | Center for Advanced Life Cycle Engineering[EB/OL]. /2024-03-25. <https://calce.umd.edu/battery-data>.
- [66] Kadetotad D, Yin S, Berisha V, et al. An 8.93 TOPS/W LSTM Recurrent Neural Network Accelerator Featuring Hierarchical Coarse-Grain Sparsity for On-Device Speech Recognition[J]. IEEE Journal of Solid-State Circuits, 2020, 55(7): 1877–1887.

致 谢

三年之期转瞬即逝，硕士生涯接近尾声，既然毕业论文给了这么一个小节发表自己的感想，那么我便却之不恭，向生命中的各位贵人表达真挚的谢意。首先我要感谢我的两位室友，他们是我入学第一天认识的人，也是生活中距离最近的人，感谢他们的包容大度，使得生活氛围其乐融融。其次我要感谢我的五位同门和数不清的师兄师弟，他们是我研究工作中接触最多的人，个个都是卧龙凤雏，与之交流常感受良多，感谢他们让我日益精进。最后压轴感谢我的导师，导师不仅在专业上颇有造诣，并且温文尔雅、循循善诱，感谢他的谆谆教诲，使我们的研究生涯一马平川。

最后我要特别感谢我的父亲彭先生和母亲陈女士，他们的支持与理解是我坚持求学的最大动力，也是最大的信心。感谢我的本科同学陈总、李总、邹总，他们事业有成且时常与我共勉，是我的学习榜样，祝他们前程似锦，锦上添花。感谢我的高中同学贺总，去年承蒙厚爱见证了他的幸福，他高尚的品格如同霁月清风，相识多年曲水流觞不曾间断，常使我醍醐灌顶。

最后我想说以上各位都是我生命中的灯塔，为我驱散黑暗，在迷雾中指引道路。硕士生活结束又是一段新的开始，尽管前途未卜，山高路远，但有你们的陪伴就不孤单，感谢一路有你们。还有许多我想要感谢的人受限于篇幅没有具体指出，没错，就是正在看这篇致谢但没在上面列出的你，感谢你们。

但愿人长久，千里共婵娟。

攻读硕士学位期间的科研成果

已获得及申请的专利

[1] 黎冰, 彭一闻. 发明专利: 一种电池电量监测方法、系统及存储介质. 2024101028084

深圳大学

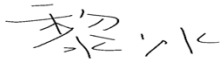
指导教师对研究生学位论文的学术评语

研究生姓名	彭一闻	论文题目	用于锂离子电池SOC估计的数字芯片设计		
学号	2110436222	专业（领域）名称	集成电路工程	学位级别	硕士

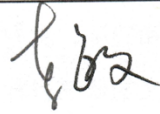
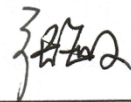
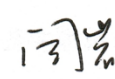
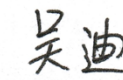
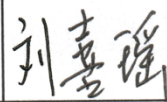
对学位论文的学术评语（评价论文的学术水平和实用价值，论文有无新见解，论据是否充分、可靠，运用基础 和专业理论、知识的实际能力，所体现的科研能力，论文写作是否严谨、科学，是否有学术不端行为，论文存在的主要 缺点和问题等，是否达到学位论文的要求）：

学生的毕业论文展现了令人印象深刻的研究成果。论文中充满了对研究问题的深入思考和系统分析，显示了学生在学术研究方面的扎实基础和独立思考能力。首先，论文展现了对文献的全面综述和理论框架的合理构建，为研究提供了坚实的理论基础。其次，学生在数据收集、分析和解释方面表现出色，使得研究结果具有了较高的可信度和说服力。此外，论文的结构清晰，逻辑严谨，表达流畅，充分展现了学生良好的文笔和组织能力。通过清晰准确地阐述问题、目的、方法和结果，学生成功地向读者传达了自己的研究成果和见解。毕业论文的完成标志着学生在学术生涯中迈出的重要一步，我对学生的优秀表现表示衷心的祝贺，并期待着在未来的学术和职业道路上取得更加辉煌的成就。

指导教师对学位论文的评阅结果： 同意答辩

指导教师（签名）： 日期： 2024-03-28

深圳大学研究生学位（毕业）论文
答辩委员会决议书

研究生姓名	彭一闻		论文题目	用于锂离子电池SOC估计的数字芯片设计		
学号	2110436222		专业名称	085403 集成电路工程	学位级别	硕士
论文答辩委员会出席名单	委员会成员	姓名	职称	工作单位		本人签名
	主席	李敏	教授	天津大学		
	委员	张敏	高级工程师	无锡艾芯泽微电子科技有限公司		
		葛磊	副教授	电子与信息工程学院		
		闫岩	副教授	电子与信息工程学院		
		吴迪	副教授	电子与信息工程学院		
	秘书	刘喜瑶		电子与信息工程学院		
答辩日期:	2024-05-14					
答辩地点:	致真楼1203					

深圳大学研究生学位（毕业）论文
答辩委员会决议书

研究生姓名	彭一闻	论文题目	用于锂离子电池SOC估计的数字芯片设计		
学 号	2110436222	专业名称	085403 集成电路工程	学位级别	硕士
答辩日期	2024-05-14				
答辩委员会对学位论文及答辩情况的评语及明确的结论意见（850字以内, 若推荐参评优秀学位论文, 请给出推荐意见）：					
彭一闻同学硕士论文《用于锂离子电池 SOC 估计的数字芯片设计》，课题来源于导师自拟。论文就如何建立有效的电池 SOC 估计算法模型、如何对算法模型进行高效的硬件实现等问题进行了探讨和验证，具有较好的创新和应用价值。 论文的研究目标是设计一款能够有效估计锂离子电池 SOC 的数字芯片，全文涵盖背景调查、算法建模、RTL 建模与仿真，以及 FPGA 实现至后端设计的整个过程。首先，论文对 SOC 的基础原理进行了深入分析，为整个研究提供了坚实的理论基础。接着，介绍了一种创新的混合架构 SOC 估计算法，该算法整合了基于安时积分法的特征提取引擎以及 LSTM 神经网络，通过与传统算法的比较，展示了其显著的优势。此外，文中详尽阐释了该算法的电路设计和实现过程，提出了一种结合开路电压法的双模式顶层架构，并清晰地划分了各模块的功能，设计过程中遵循逻辑单元最大化复用及模型参数可更新的原则。最后，通过波形仿真和基于 FPGA 的电池电量监测系统实际测试，验证了设计的正确性与有效性，并通过后端设计对 ASIC 设计流程进行了完善。 论文研究内容丰富，论证较为严密，表明该生较好地掌握了系统理论知识且具备一定实践能力，能够独立分析问题、解决问题。在答辩过程中，回答问题条理清楚、表述准确，态度诚恳，虚心接受建议。 答辩委员会一致认为该论文已达到硕士学位论文要求，同意彭一闻同学通过论文答辩，准予毕业，建议授予彭一闻同学电子信息硕士学位。					
答辩委员会	表决项目		计票		结论
	毕业论文（准予毕业）	通过	5		通过 ☒
		未通过	0		未通过 ☐
	学位论文（建议授予学位）	同意	5		通过 ☒
		不同意	0		未通过 ☐
	学位论文答辩不通过，允许其修改论文后重新申请一次答辩 票。				

秘书（签名）：

刘嘉琦

主席（签名）：

杨江